

Εισαγωγή

Σύνοψη βασικών εννοιών
Ανασκόπηση Αρχιτεκτονικής Υπολογιστών

Νεκτάριος Κοζύρης, Διονύσης Πνευματικάτος
& Κωστής Νίκας

{nkoziris,pnevmati,knikas}@cslab.ece.ntua.gr

8ο εξάμηνο ΣΗΜΜΥ – Ακαδημαϊκό Έτος: 2025-26

<https://helios.ntua.gr/course/view.php?id=1039>

Προηγμένα Θέματα Αρχιτεκτονικής Υπολογιστών

- Το μάθημα «Αρχιτεκτονική» σας έμαθε πως φτιάχνονται απλά υπολογιστικά συστήματα
- Εδώ πραγματευόμαστε πως φτιάχνονται τα «εξυπνότερα» και αποδοτικότερα υπολογιστικά συστήματα
- Διαδικαστικά:
 - Ασκήσεις (προαιρετικές): έως +2 μονάδες
IF $B\Delta > 5$ then
 Τελικός Βαθμός = $(1 + 0.2 * \overline{B\Delta} / 10) * B\Delta$
else
 Τελικός Βαθμός = $B\Delta$
όπου $\overline{B\Delta}$ = ΜΟ Βαθμού Ασκήσεων, $B\Delta$ = Βαθμός Διαγωνίσματος
 - Εξέταση με κλειστά βιβλία χωρίς A4 σκονάκι
 - Εργασία εξαμήνου (** προαιρετική)
 - Όσοι ενδιαφέρεστε επικοινωνήστε μαζί μας

Βιβλίο:

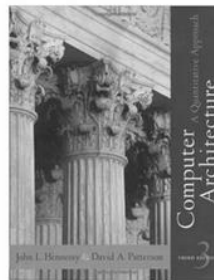
- Αρχιτεκτονική Υπολογιστών – Μια ποσοτική Προσέγγιση, των Hennessy & Patterson



1st Edition



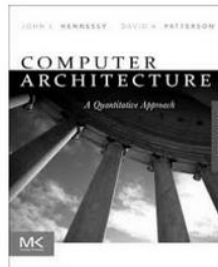
2nd Edition



3rd Edition



4th Edition



5th Edition

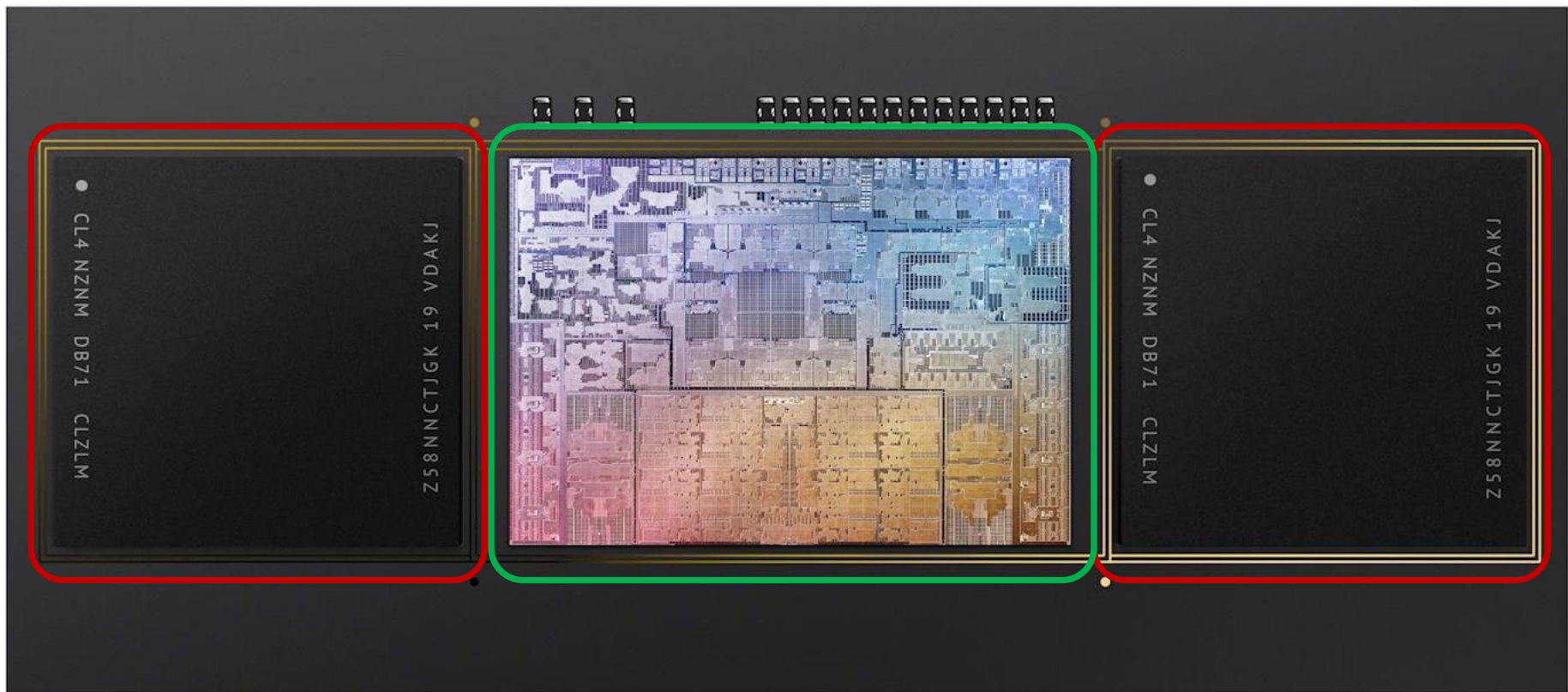


6th Edition

- Νέο: 6η Έκδοση, στον Εύδοξο



What is this?



32GB unified memory

High bandwidth, low latency

256-bit LPDDR5 interface

Apple-designed custom package

Apple M1 Pro
Processor
HBM memory

Apple M1 Pro (2021)

8 high-performance cores

Ultra-wide execution architecture

192KB instruction cache

128KB data cache

24MB L2 cache

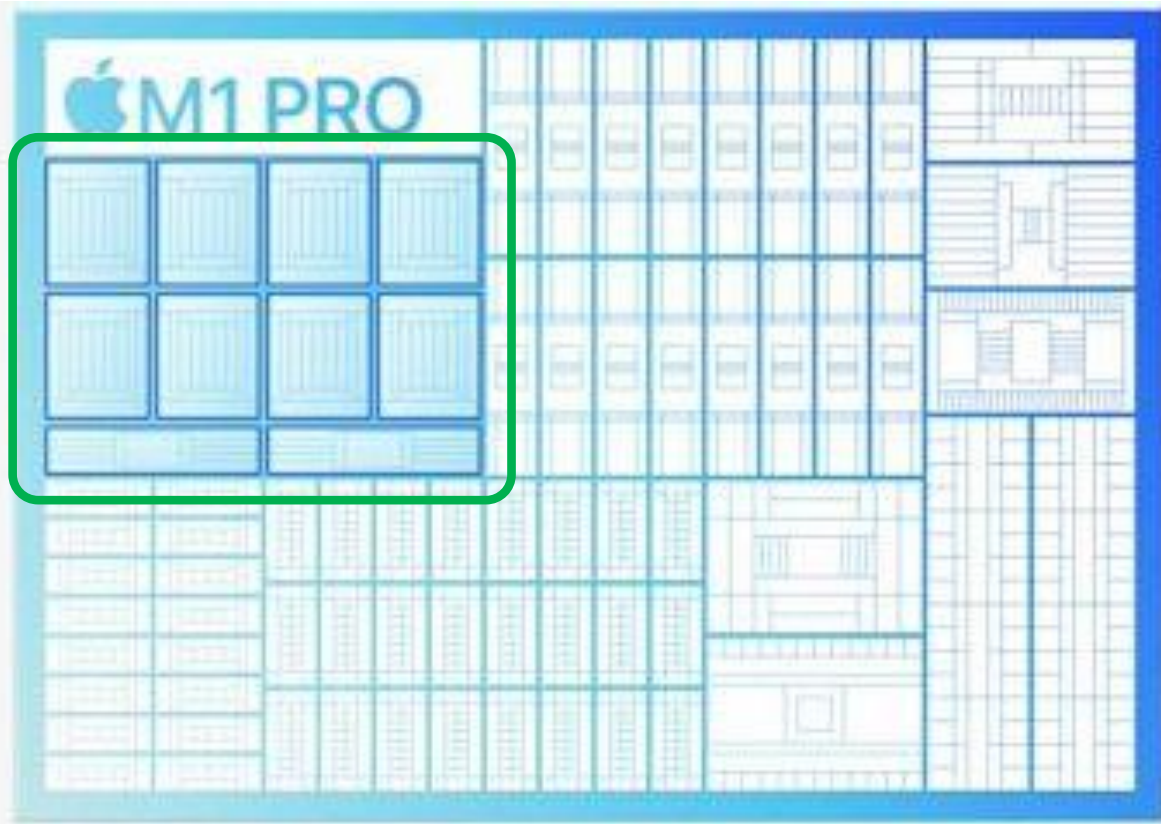
2 high-efficiency cores

Wide execution architecture

128KB instruction cache

64KB data cache

4MB L2 cache



Apple M1 Pro



16-core GPU

2048 execution units

Up to 49,512 concurrent threads

5.2 teraflops

164 gigatexels/second

82 gigapixels/second

Apple M1 Pro



Media Engine

Hardware-accelerated H.264, HEVC, ProRes, and ProRes RAW

Video decode engine

Video encode engine

ProRes encode/decode engine

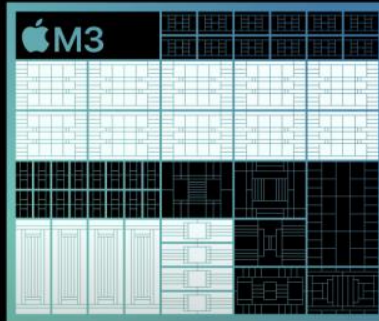
Multiple streams of 4K and 8K ProRes video

Apple M1 Pro



Apple M3, M3 Pro & M3 Max (2023)

Up to 24GB of unified memory
25 billion transistors



8-core CPU

4 performance cores
4 efficiency cores
Up to 35% faster than M1
Up to 20% faster than M2

10-core GPU

Next-generation architecture
Dynamic Caching
Mesh shading
Ray tracing
Up to 65% faster than M1
Up to 20% faster than M2

Up to 36GB of unified memory
37 billion transistors

12-core CPU

6 performance cores
6 efficiency cores
Up to 20% faster than M1 Pro



18-core GPU

Next-generation architecture
Dynamic Caching
Mesh shading
Ray tracing
Up to 40% faster than M1 Pro
Up to 10% faster than M2 Pro

Up to 128GB of unified memory
92 billion transistors

16-core CPU

12 performance cores
4 efficiency cores
Up to 80% faster than M1 Max
Up to 50% faster than M2 Max



40-core GPU

Next-generation architecture
Dynamic Caching
Mesh shading
Ray tracing
Up to 50% faster than M1 Max
Up to 20% faster than M2 Max

Apple M4, M4 Pro & M4 Max (2024)

10-core CPU

4 performance cores

- Improved branch prediction
- 10-wide instruction decode
- 40% larger reorder buffer
- Next-generation ML accelerators

6 efficiency cores

- Improved branch prediction
- Double front-end fetch width
- Wider and lower-latency vector FP
- Next-generation ML accelerators



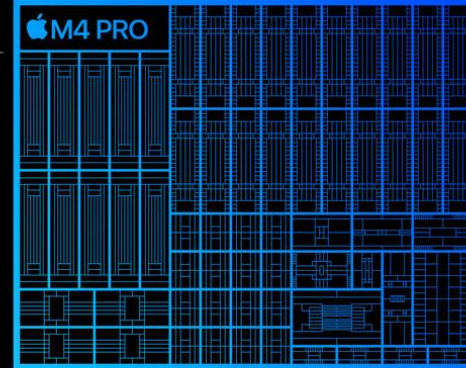
14-core CPU

10 performance cores

- Improved branch prediction
- 10-wide instruction decode
- 40% larger reorder buffer
- Next-generation ML accelerators

4 efficiency cores

- Improved branch prediction
- Double front-end fetch width
- Wider and lower-latency vector FP
- Next-generation ML accelerators



20-core GPU

- Dynamic caching
- Hardware-accelerated mesh shading
- 2x faster ray tracing acceleration
- Improved scheduler

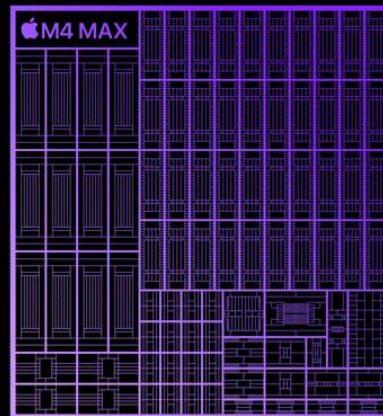
16-core CPU

12 performance cores

- Improved branch prediction
- 10-wide instruction decode
- 40% larger reorder buffer
- Next-generation ML accelerators

4 efficiency cores

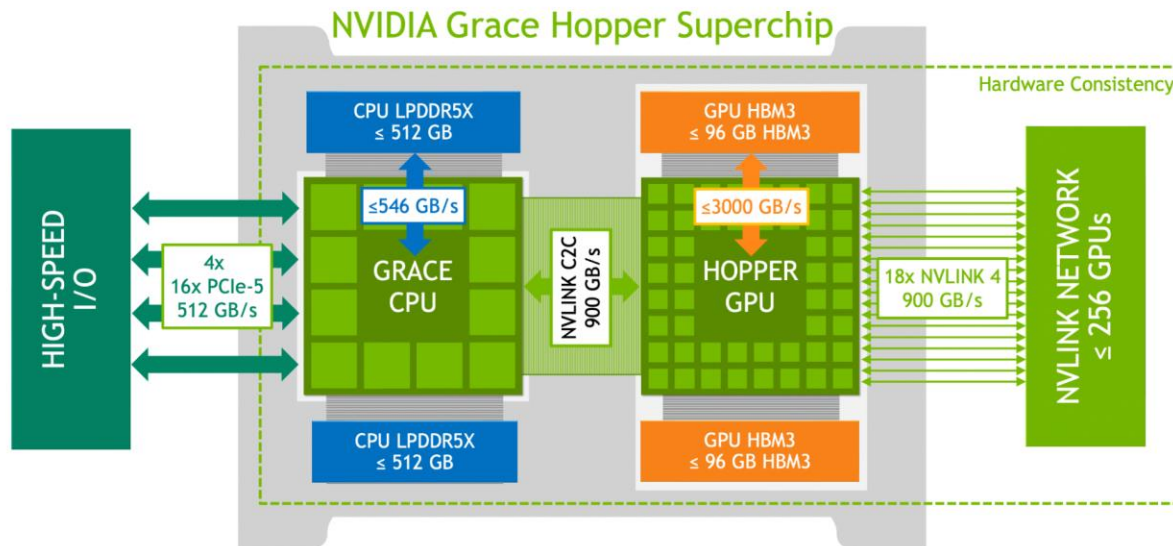
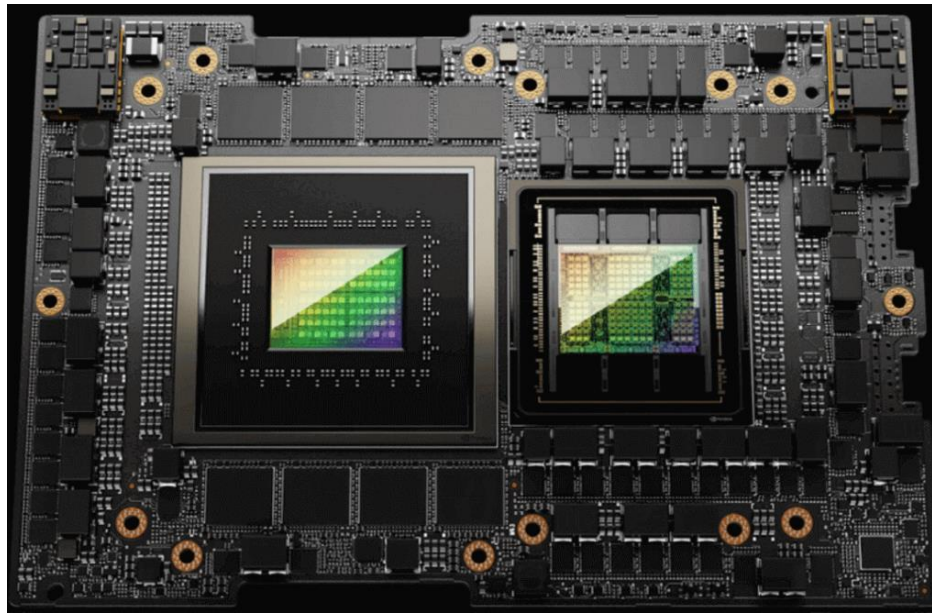
- Improved branch prediction
- Double front-end fetch width
- Wider and lower-latency vector FP
- Next-generation ML accelerators



Apple M4 Max GPU(40 Cores) is a 3nm chip, launched by Apple at 2024. It has 96 GB built-in(On-Board/On-Chip) memory with bandwidth up to 546 GB/s. It has 5120 general-purpose ALUs(CUDA cores/Shader cores)

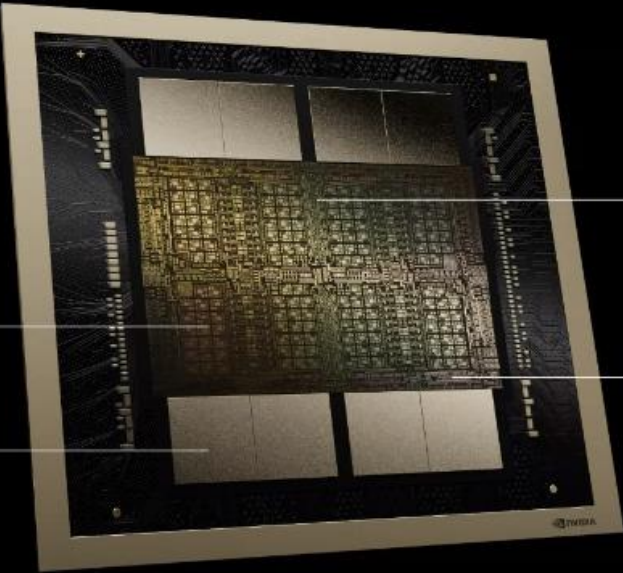
~9,22TF FP64

NVIDIA Gracehopper superchip (GH200) 2022



Blackwell superchip 208 billion transistors

New Class of AI Superchip
The Two Largest Dies Possible—Unified as One GPU



Reticle-sized Die 1

Fast Memory
192GB HBM3e

Reticle-sized Die 2

2 reticle limited dies operate as One Unified CUDA GPU
NV-HBI 10TB/s High Bandwidth Interface
Full performance with no compromises

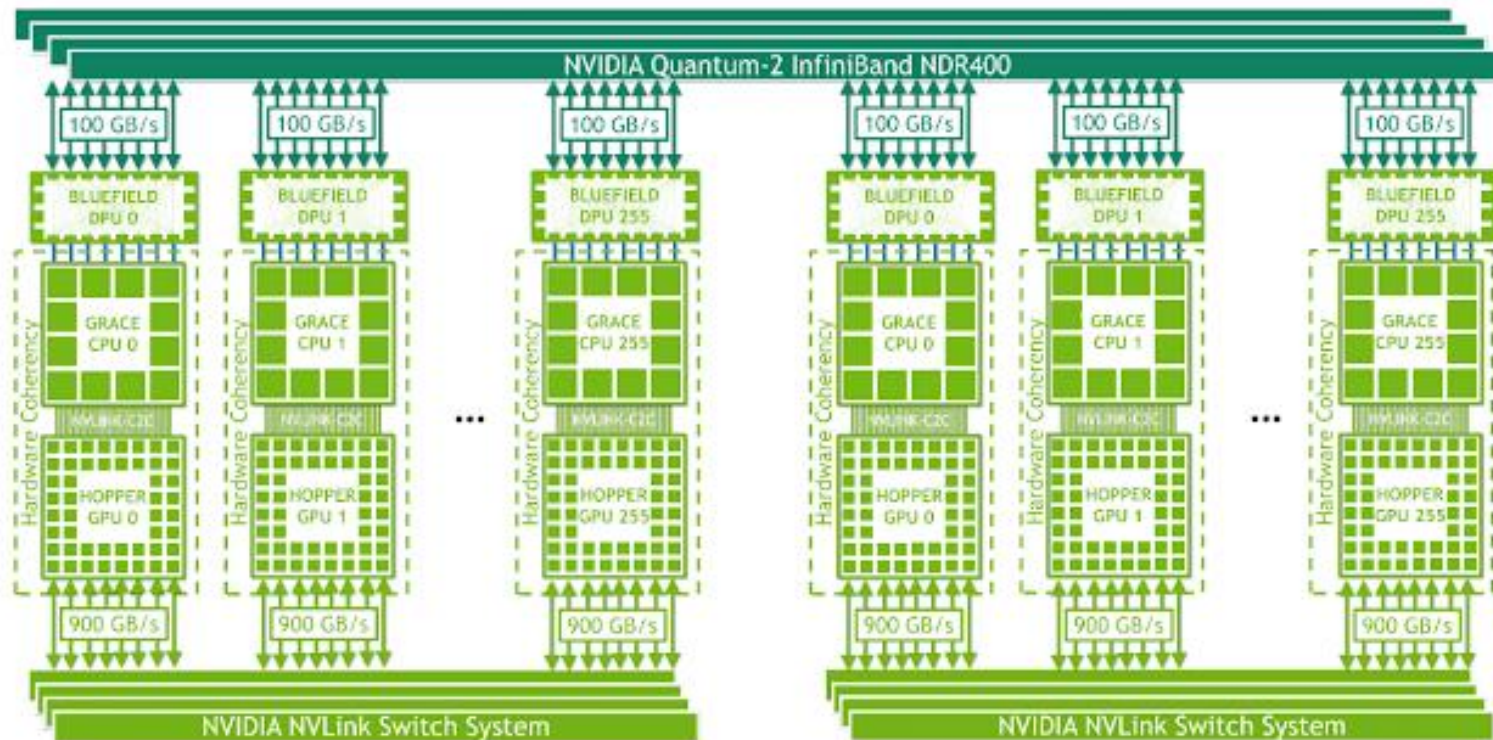
10 PetaFLOPS FP8 | 20 PetaFLOPS FP4
192GB HBM3e | 8 TB/sec HBM Bandwidth | 1.8TB/s NVLink

H100 (Hopper) vs B200 (Blackwell)

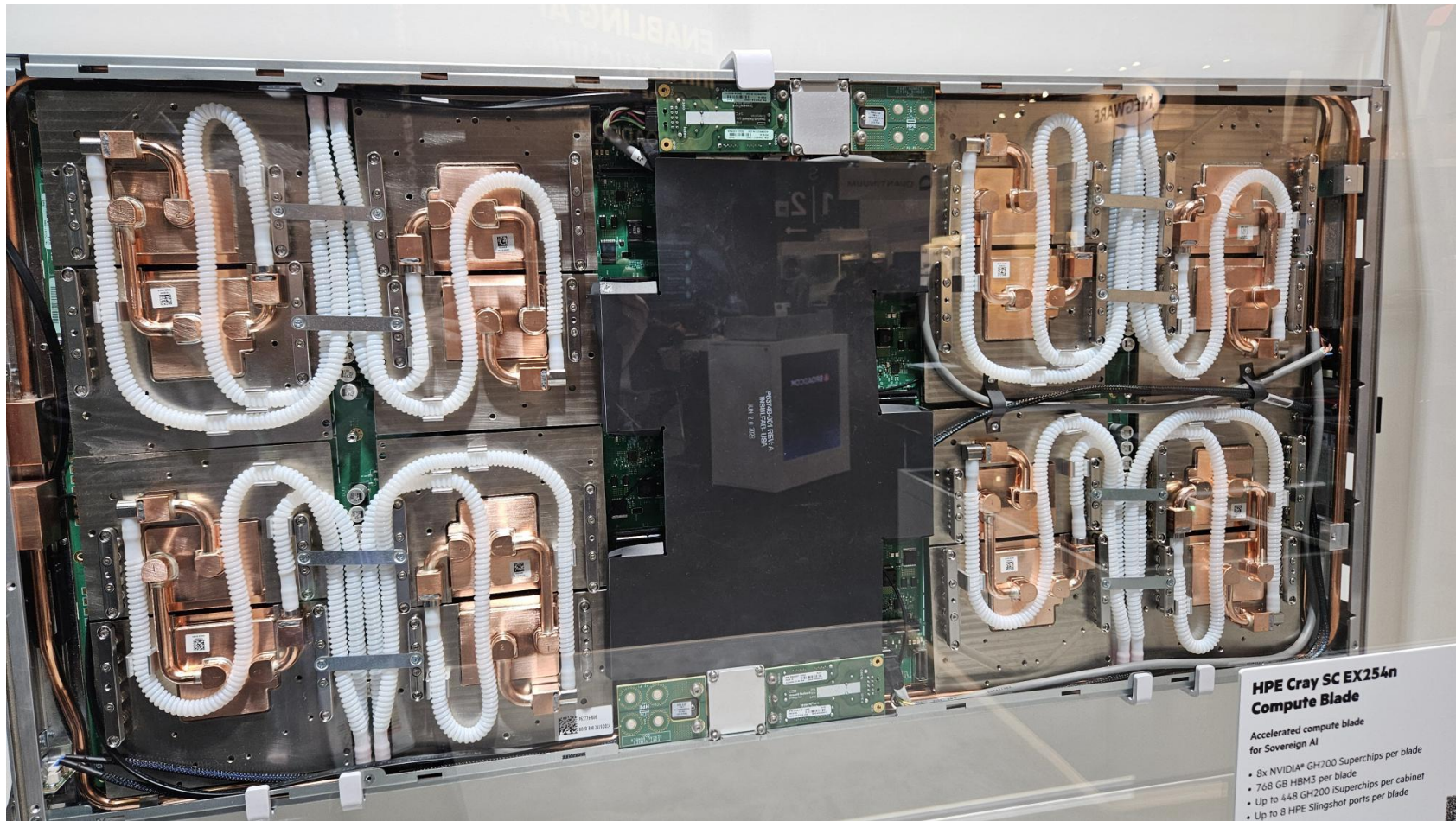
METRIC	HOPPER (H100)	BLACKWELL (B200)
Peak FP16/BF16 Tensor TFLOPS	~2.0 PF	~4.5 PF ⁽¹¹⁾
Peak FP8 Tensor TFLOPS	~4.0 PF	~9.0 PF ⁽¹¹⁾
Peak FP4 Tensor TFLOPS	– (not supported)	~18 PF ⁽¹¹⁾
FP64 Tensor TFLOPS	60 TF ⁽¹²⁾	(Substantially lower) ⁽¹¹⁾
LLM Inference Throughput (LLAMA3-like)	Baseline	+45% (observed, via NVFP4 quant.) ⁽¹⁴⁾
MLPerf Inference (DeepSeek R1)	Baseline	45% higher throughput ⁽¹⁴⁾
Dense GEMM FP32/FP8 (sustained)	High throughput	Up to 4× lower than H100 ⁽¹³⁾
GPU On-chip Memory	80 GB (HBM3) ⁽¹⁷⁾	192 GB (HBM3e) ⁽¹²⁾
Memory Bandwidth	3.35 TB/s ⁽¹⁸⁾	8.0 TB/s ⁽¹⁸⁾
NVLink I/O Bandwidth	900 GB/s	1.8 TB/s ⁽²¹⁾
Power (TGP)	~600–700 W ⁽¹⁷⁾	~1,200–1,400 W ⁽²²⁾

Emphasis on FP8 and FP16 vs FP64

Building systems:



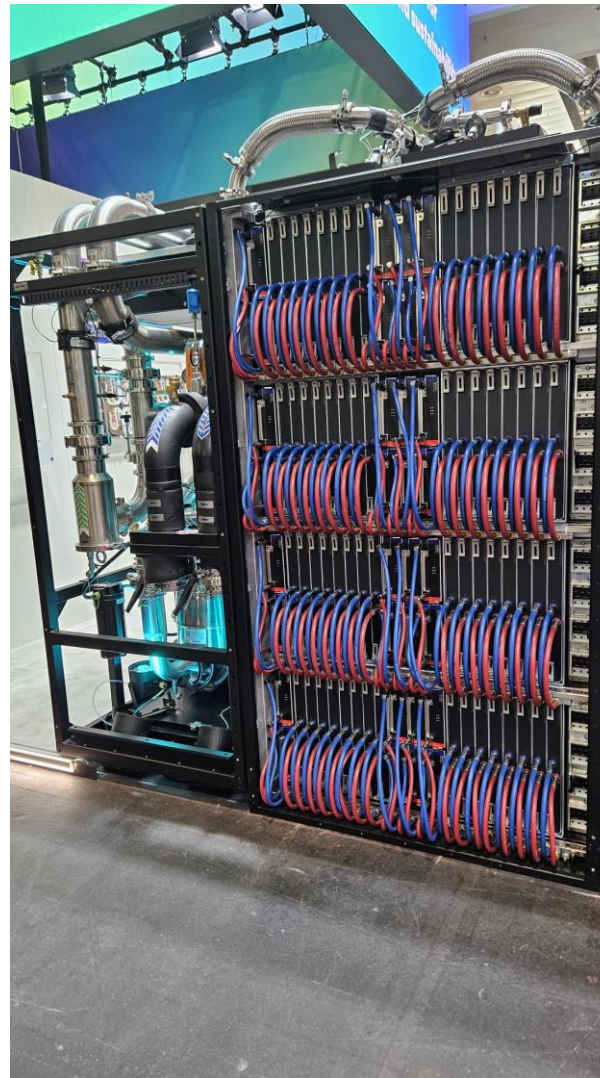
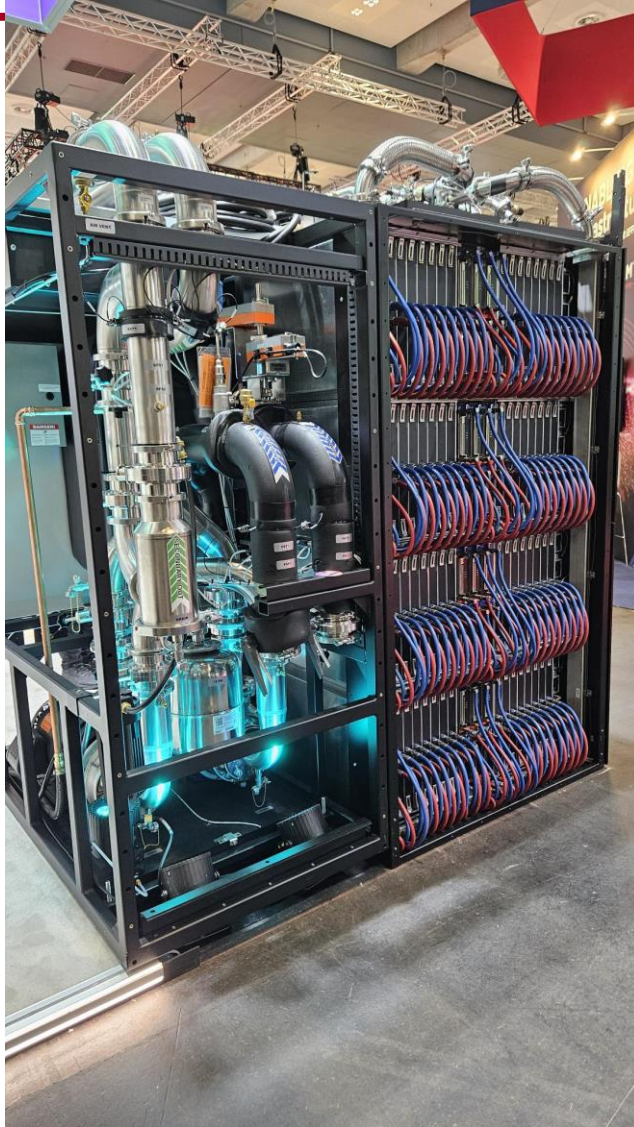
June 2024-HPE Cray EX254n Accelerator Blade (GH200): Daedalus node



8 NVIDIA Grace Hopper modules (72-core Arm Neoverse V2 CPU, LPDDR memory, and a Hopper H100 GPU).

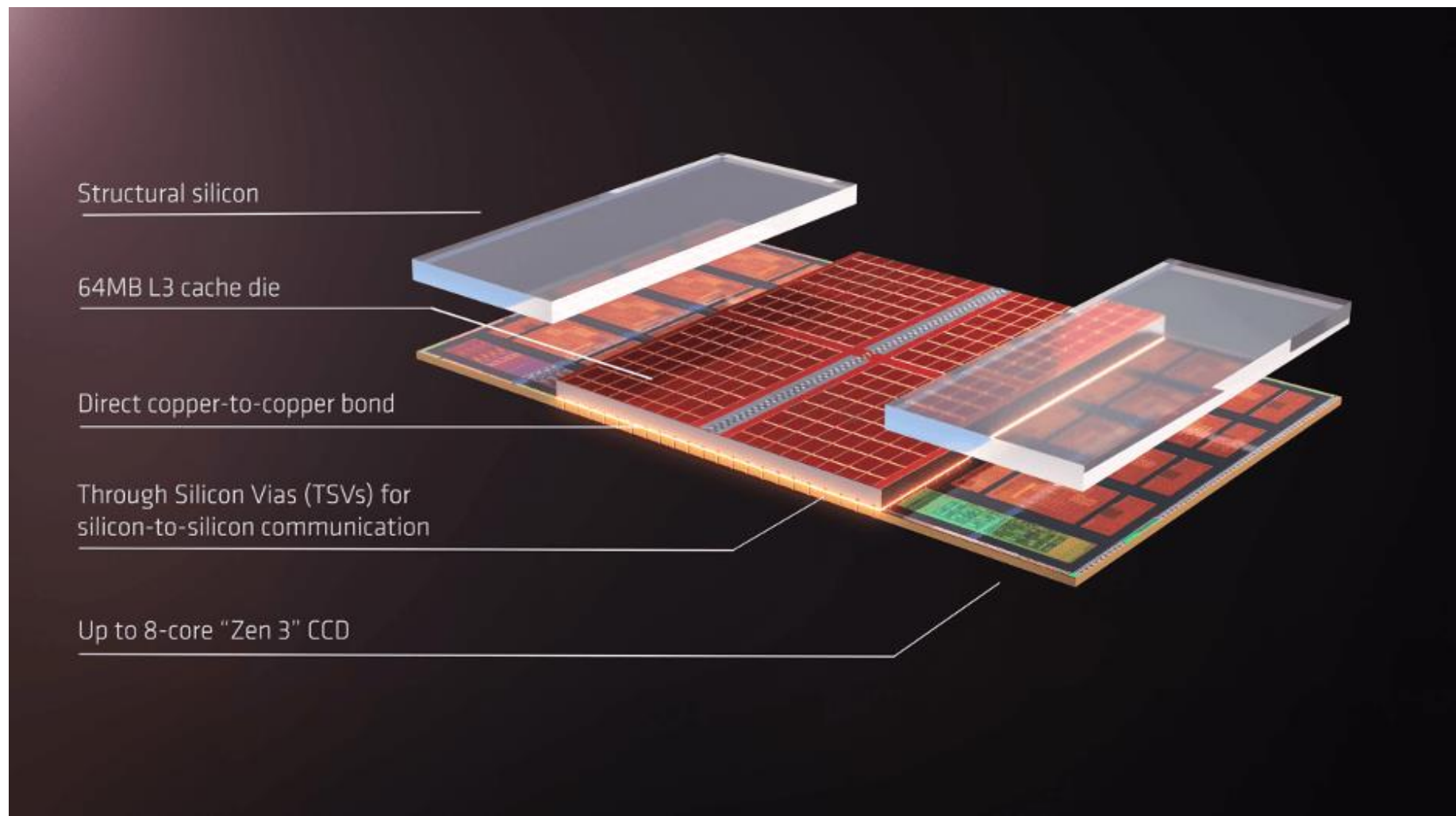
Those eight Grace Hopper modules are split into two four-module carrier boards, and then the node uses Slingshot 11 with four injection ports per node.

Daedalus cabinet EX4000



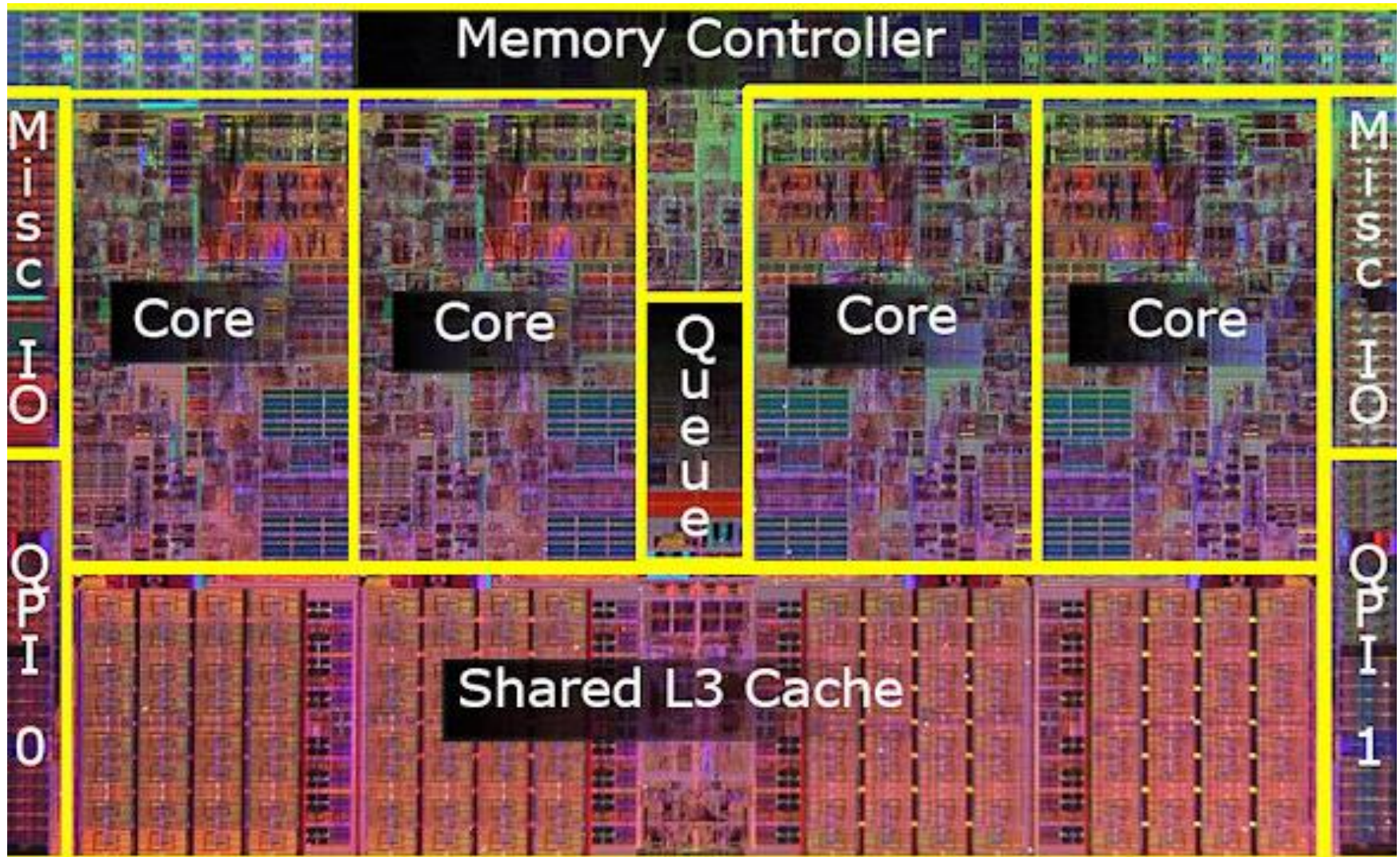
Direct Liquid cooling

AMD 3D integration



Intel Core i7 Processor

Nahalem, 731M xtor, 263mm², 45nm process, ~2008

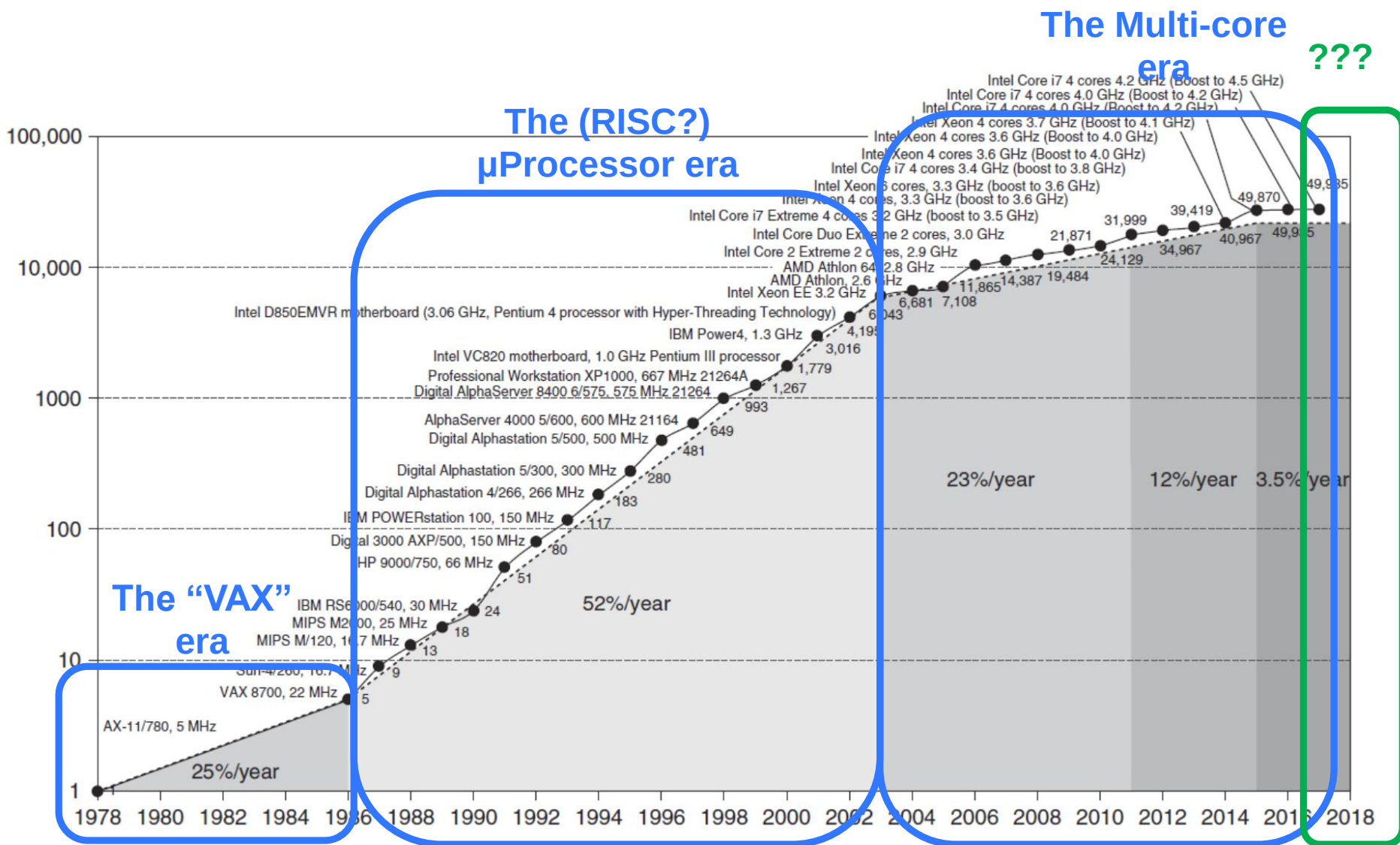


CPU die comparison

CPU	Technology Process	Cores	GPU	Transistor Count	Die Size mm ²
Apple M1 Pro	5nm	8+2	✓□	33,7B	246
AMD Epyc Rome (64-bit, SIMD, caches)	7 & 12nm	32?		39.5B	>1000!
Apple A13 (iphone11+)	7nm	4		8.5B	~99
Haswell GT3 4C	22nm	4	GT3	?	~264
Haswell ULT GT3 2C	22nm	2	GT3	1.3B	181
Intel Ivy Bridge 4C	22nm	4	GT2	1.2B	160
Intel Sandy Bridge E 6C	32nm	6	N/A	2.27B	435
MIPS R2000, 1985	2000nm	1	-	110K	80
Intel 4004, 1971	10000nm	1	-	2250	12

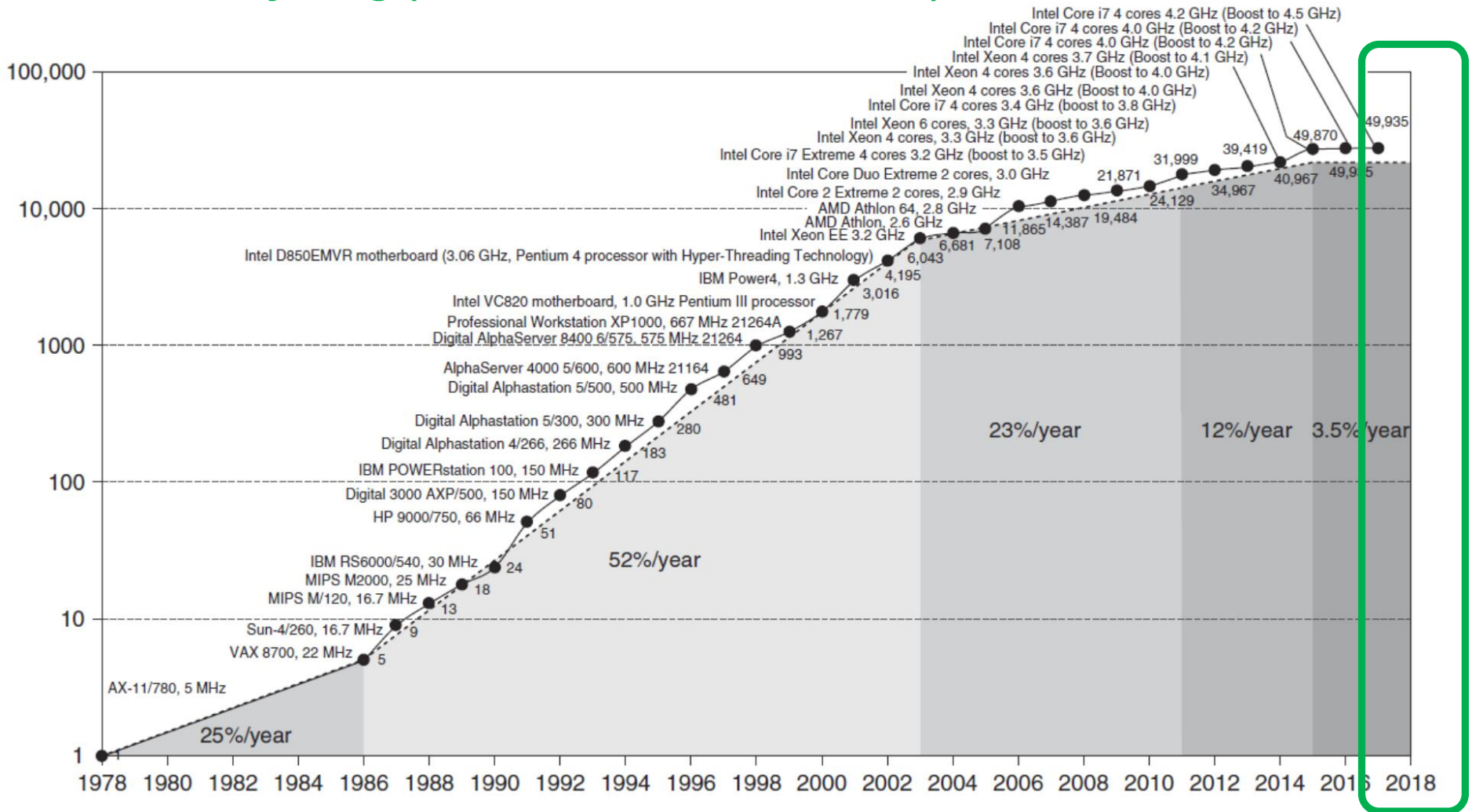
Source: cpu-world.com, anandtech.com, Wikipedia, κλπ.

(Single) Processor Performance



What now?

GPGPU / Accelerator / Open Hardware / Approximate / ML/DL era???
Everything (SW/HW/Platform/FPGA/...) as a service era ???



Outline

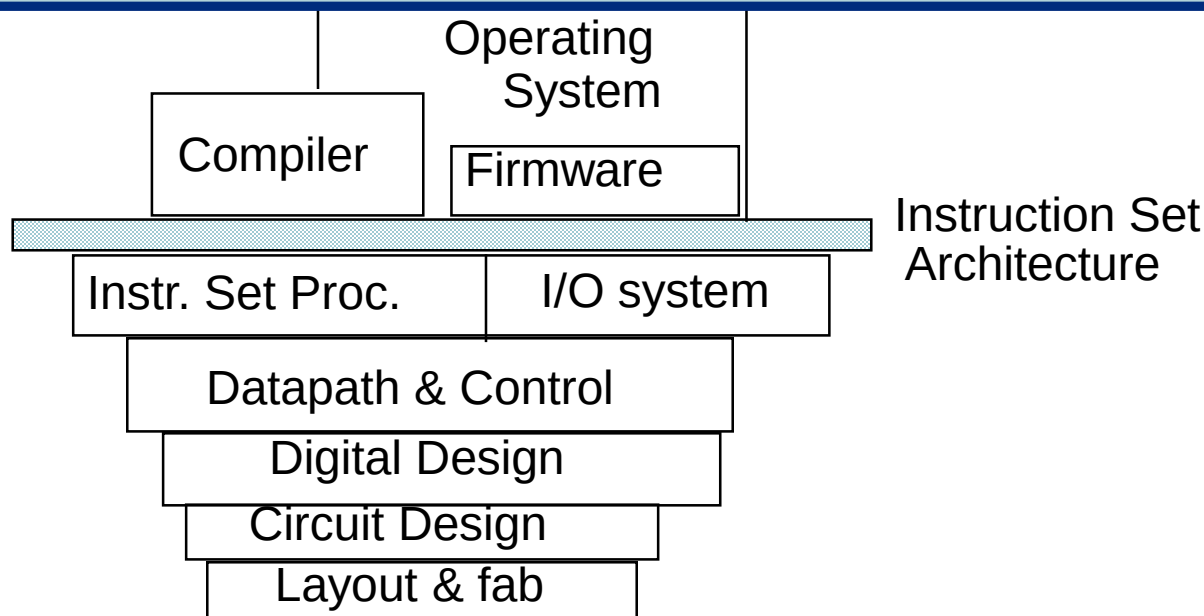
- What is Computer Architecture?
- Computer Instruction Sets – the fundamental abstraction
 - review and set up
- Dramatic Technology Advance
- Beneath the illusion – nothing is as it appears
- Computer (micro)-Architecture Renaissance
 - Heterogeneity
 - Security
 - Processor chips => Systems
 - Different levels of integrations

Key Ideas in Computer Architecture

- Abstraction (Layers of Representation/Interpretation)
- Moore's Law (Designing through trends)
- Principle of Locality (Memory Hierarchy)
- Parallelism
- Mechanisms vs. policies
- Primitives, not solutions
- Speculation
- Efficiency: Heterogeneity – Accelerators
- Dependability via Redundancy

What is “Computer Architecture”?

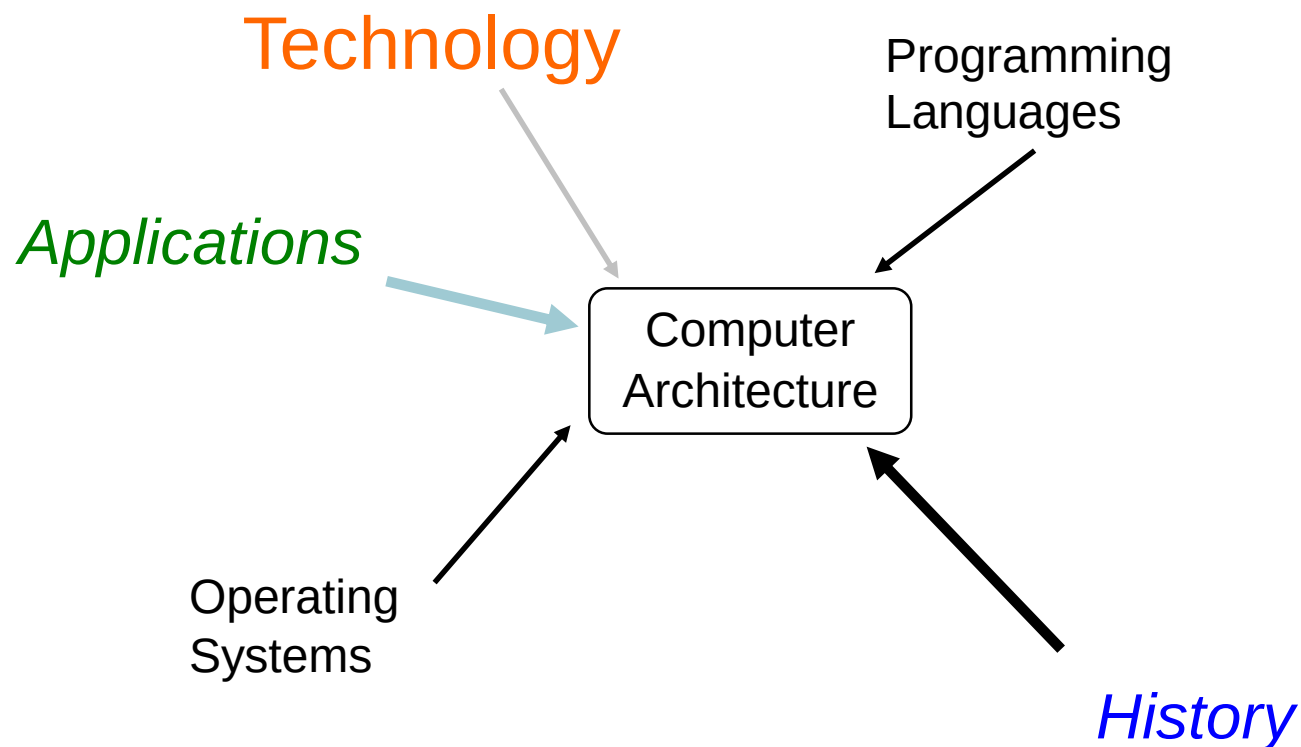
This is a single process/processor/core view
What about parallel/embedded/... systems??



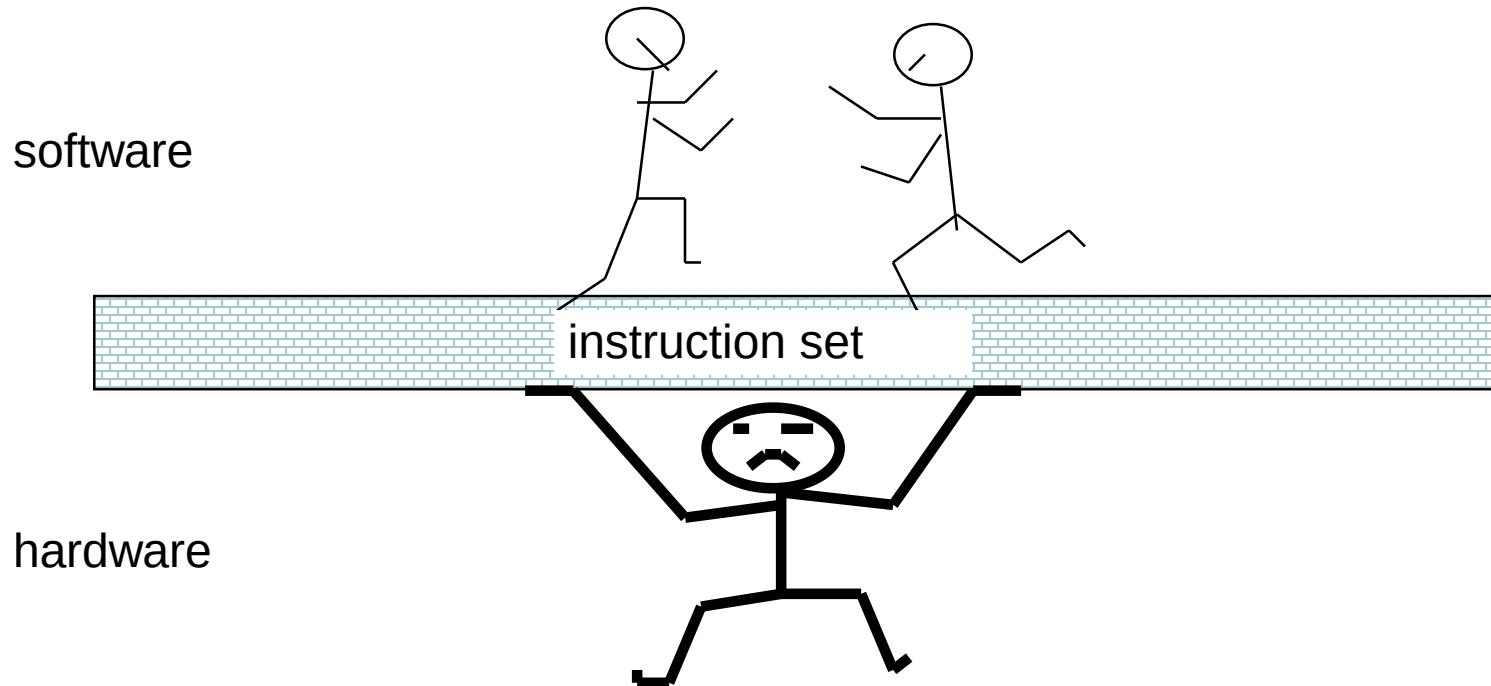
Semiconductor Materials

- Coordination of many *levels of abstraction*
- Under a rapidly *changing set of forces*
- Design, Measurement, *and* Evaluation

Forces on Computer Architecture



The Instruction Set: a Critical Interface



- Properties of a good abstraction
 - Lasts through many generations (portability)
 - Used in many different ways (generality)
 - Provides **convenient** functionality to higher levels
 - Permits an **efficient** implementation at lower levels

Current Trends in Architecture

- Cannot continue to leverage Instruction-Level parallelism (ILP)
 - Single processor performance improvement ended in 2003
 - Latency improvements are difficult => focus on throughput
- “New” models for performance:
 - Data-level *parallelism* (DLP)
 - Thread-level *parallelism* (TLP)
 - Request-level *parallelism* (RLP)
- These require explicit restructuring of the application

Classes of Computers

- Personal Mobile Device (PMD)
 - e.g. smart phones, tablet computers (1.8B @ 2010, 14B @ 2021)
 - Emphasis on energy efficiency and real-time
- Desktop Computing
 - Emphasis on price-performance (0.35B @ 2010, 0.275@2020)
- Servers
 - Emphasis on availability (very costly downtime!), scalability, throughput (10 million @ 2020?)
- Clusters / Warehouse Scale Computers
 - Used for “Software as a Service (SaaS)”, PaaS, IaaS, etc.
 - Emphasis on availability (\$6M/hour-downtime at Amazon.com!) and price-performance (server cost, power, cooling, etc)
 - Sub-class: Supercomputers, emphasis: floating-point performance and fast internal networks, and big data analytics
- Embedded (IoT?) Computers (19 billion in 2010)
 - Emphasis: price. Prediction: 74B @ 2023

Parallelism

- Classes of parallelism in applications:
 - Data-Level Parallelism (DLP)
 - Task-Level Parallelism (TLP)
- Classes of architectural parallelism:
 - Instruction-Level Parallelism (ILP)
 - Vector architectures/Graphic Processor Units (GPUs)
 - Thread-Level Parallelism
 - Request-Level Parallelism

Flynn's Taxonomy

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data streams (SIMD)
 - Vector architectures
 - Multimedia extensions
 - Graphics processor units
- Multiple instruction streams, single data stream (MISD)
 - No commercial implementation
- Multiple instruction streams, multiple data streams (MIMD)
 - Tightly-coupled MIMD
 - Loosely-coupled MIMD

Re-Defining Computer Architecture

- “Old” view of computer architecture:
 - Instruction Set Architecture (ISA) design
 - i.e. **decisions regarding:**
 - Registers#, memory addressing, addressing modes, instruction operands, available operations, control flow instructions, instruction encoding
- “Real” computer architecture:
 - Specific requirements of the target machine
 - Design to maximize performance within constraints: cost, power, and availability
 - **Includes ISA, microarchitecture, hardware**
- Quality of Service? Security?

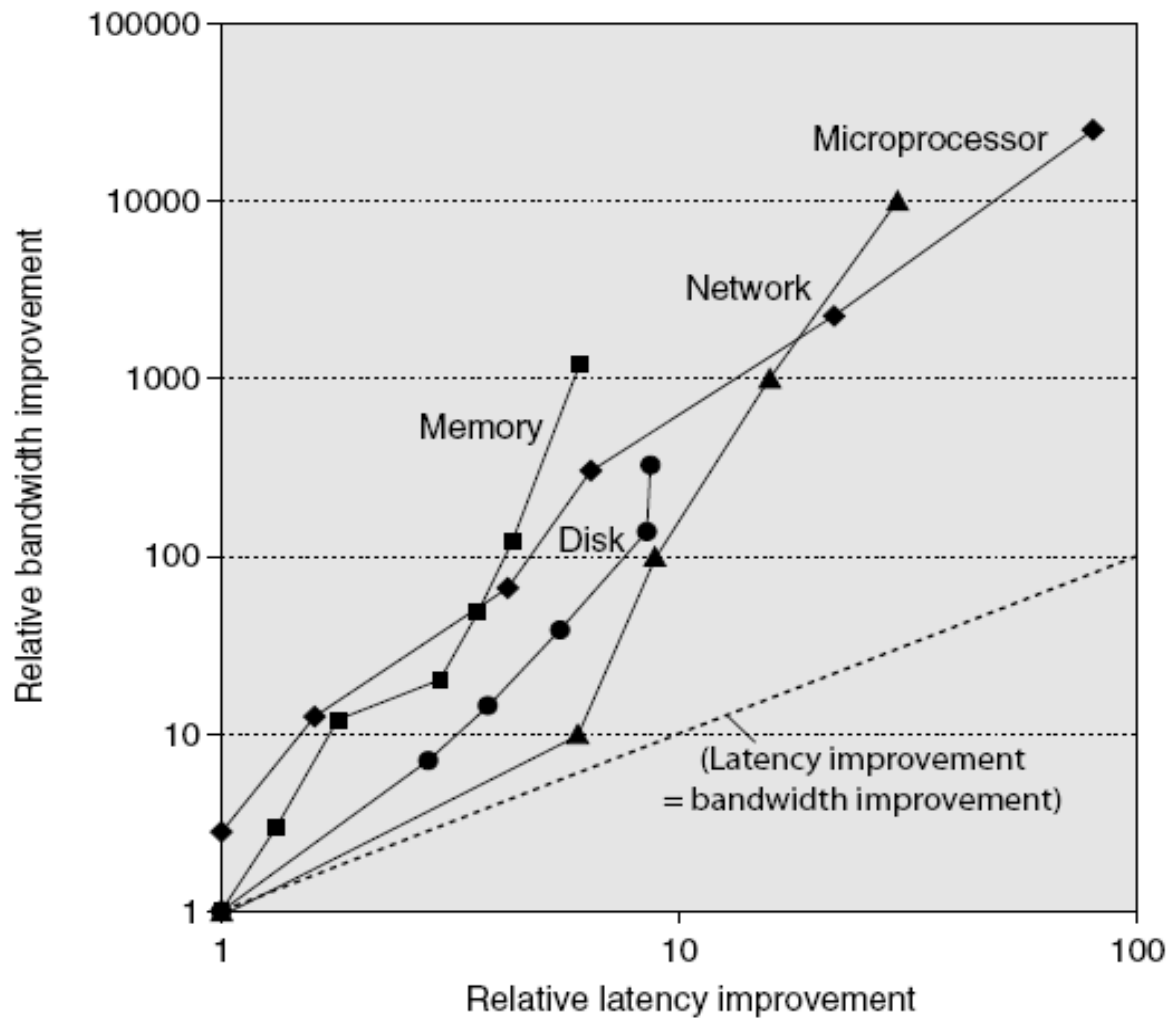
Trends in Technology

- Integrated circuit technology
 - Transistor density: 35%/year (slowing)
 - Die size: 10-20%/year
 - Integration overall: 40-55%/year
- DRAM capacity: 25-40%/year (slowing)
 - 8 Gb (2014), 16 Gb (2019), possibly no 32 Gb?
- Flash capacity: 50-60%/year
 - 8-10X cheaper/bit than DRAM
- Magnetic disk technology: 40%/year, now slowed to 5%/year
 - Density increases no longer possible?, maybe increase from 7 to 9 platters
 - 8-10X cheaper/bit than Flash
 - 200-300X cheaper/bit than DRAM
- New non-volatile technologies: PCM, Intel Optane, ...

Bandwidth and Latency

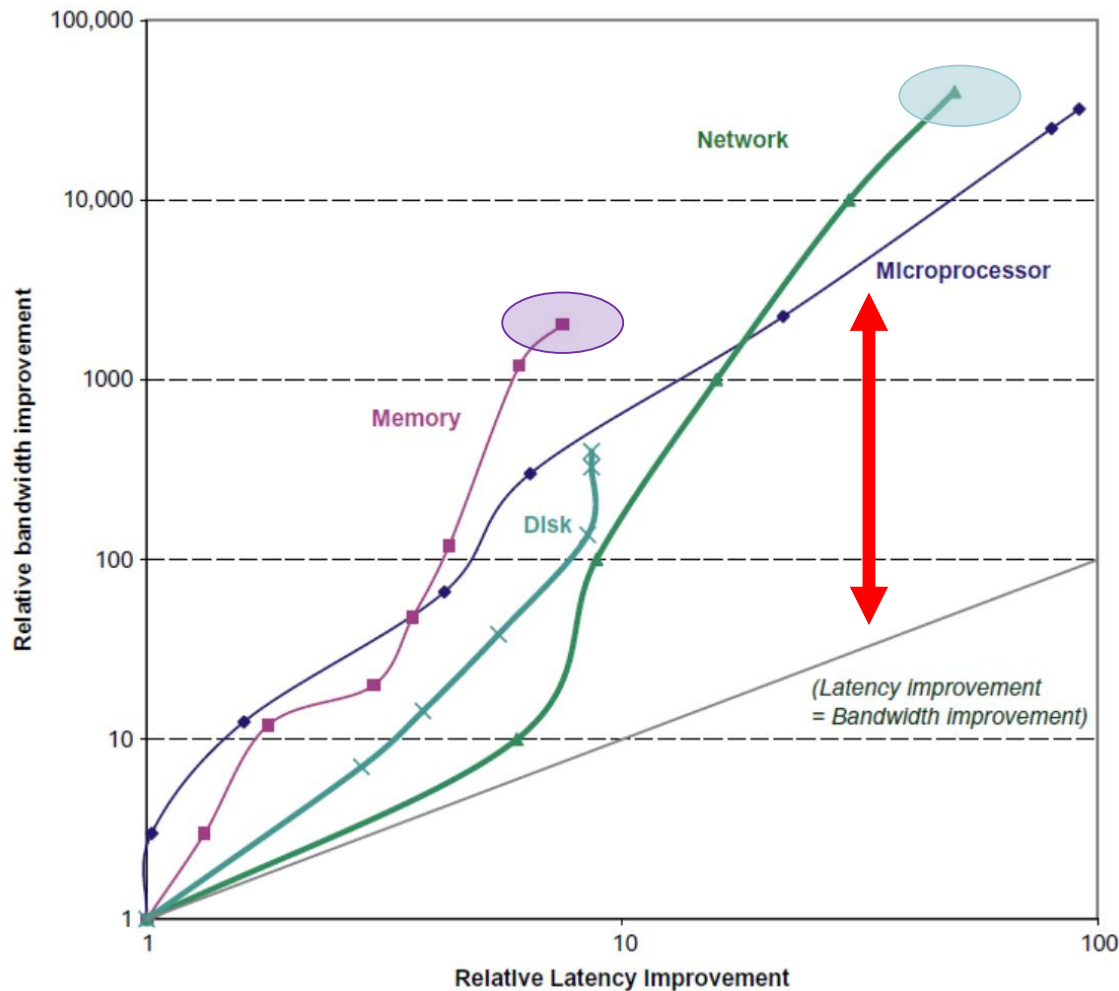
- Bandwidth or throughput
 - Total work done in a given time
 - 32,000-40,000X improvement for processors
 - 300-1200X improvement for memory and disks
- Latency or response time
 - Time between start and completion of an event
 - 50-90X improvement for processors
 - 6-8X improvement for memory and disks

Bandwidth and Latency



Log-log plot of bandwidth and latency milestones

Bandwidth and Latency



2019 update,
no significant
change in trends:

Faster Networks

Memory latency
is a bit better

BW-Latency
improvement
ratio: ~100x

Log-log plot of bandwidth and latency milestones

Transistors and Wires

- Feature size
 - Minimum size of transistor or wire in x or y dimension
 - 10 microns @1971, 32nm @2011, 5nm now (...)
 - “Transistor performance scales linearly”
 - Wire delay does not improve with feature size!
 - Not even true for xtors any more... ☹
 - Integration density scales quadratically
 - First order approximation, reality is more complex
 - Linear performance and quadratic density growth present a challenge and opportunity, creating the need for computer architect!
- 3D-stacking: the z dimension!

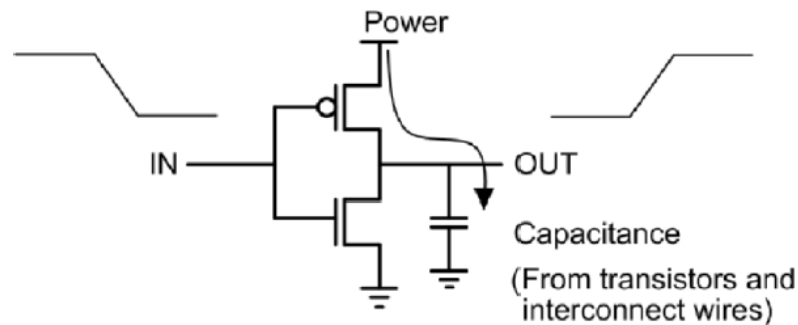
Power and Energy

- Problem: Get power in, get power out
- Thermal Design Power (TDP)
 - Characterizes sustained power consumption
 - Used as target for power supply and cooling system
 - Lower than peak power (1.5X higher), higher than average power consumption
- Clock rate can be reduced dynamically to limit power consumption
- Clock gating: turn clock off for units that are idle
- Energy per task is often a better measurement

Dynamic + Static power

- The total power for an SoC design consists of dynamic power and static power.
- Dynamic power is the power consumed when the device is active—that is, when signals are changing values.
- Static power is the power consumed when the device is powered up but no signals are changing value. In CMOS devices, static power consumption is due to leakage.

Dynamic Power



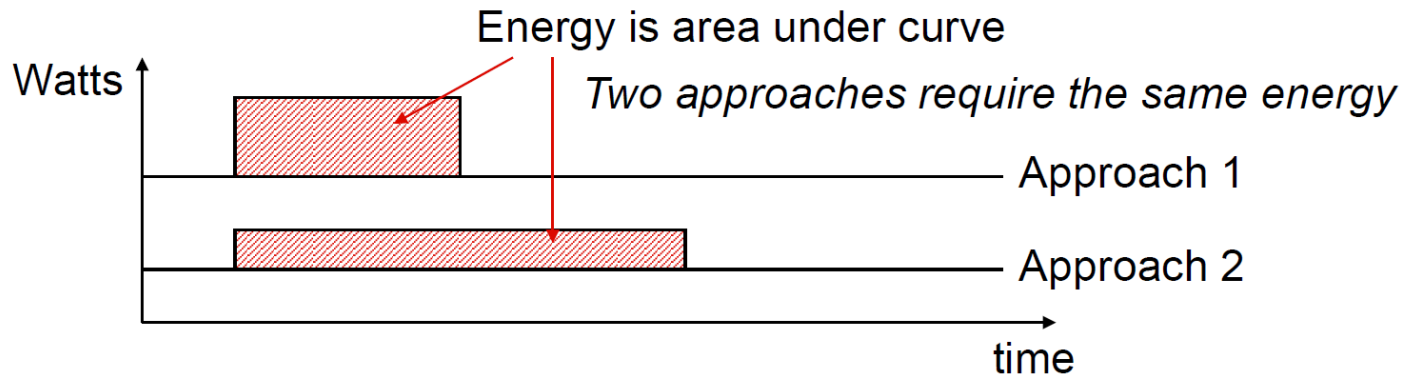
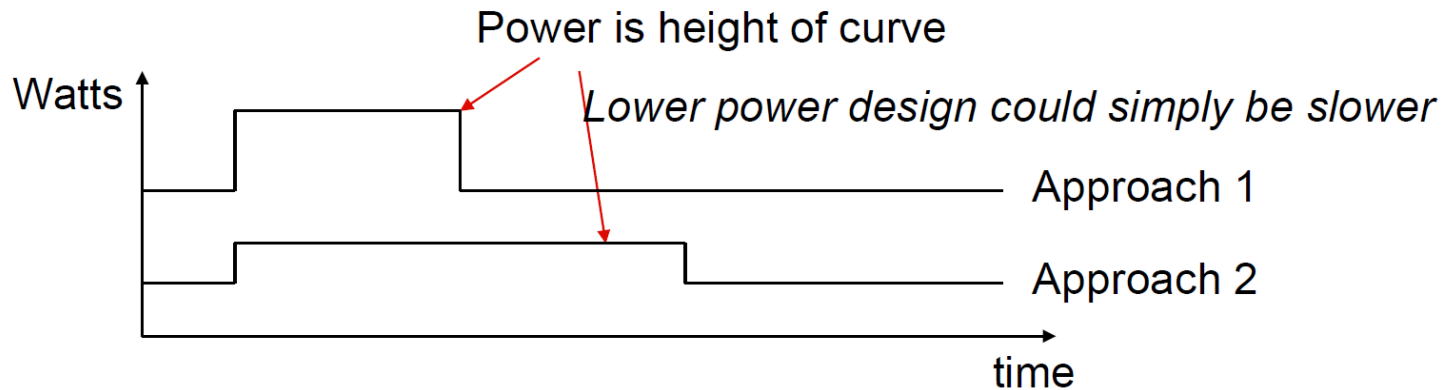
Dynamic Power

$$P_{dyn} = \alpha \cdot C_{eff} \cdot V_{dd}^2 \cdot F$$

Dynamic Energy and Power

- Dynamic energy
 - Transistor switch from $0 \rightarrow 1$ or $1 \rightarrow 0$
 - $\alpha \times \text{Capacitive load} \times \text{Voltage}^2$
 - α is the *activity factor*
 - For $\alpha = 1/2$ we get $1/2 \times \text{Capacitive load} \times \text{Voltage}^2$
 - Typical assumption for activity factor is $1/2$
- Dynamic power
 - $1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$
 - Assumes activity factor is $1/2$
- Reducing clock rate reduces power, not energy

Power vs Energy



Power

- $\text{Power} = \alpha * CFV^2$
alpha: percent time switched
C: capacitance
F: frequency
V: voltage

Capacitance is related to area.

So, as the size of the transistors shrunk, and the voltage was reduced, circuits could operate at higher frequencies at the same power

Dennard Scaling (1974)

Why haven't clock speeds increased, even though transistors have continued to shrink?

Dennard (1974) observed that voltage and current should be proportional to the linear dimensions of a transistor.

Thus, as transistors shrank, so did necessary voltage and current; power is proportional to the area of the transistor.

Source: William Gropp, cs598, www.cs.illinois.edu/~wgropp

End of Dennard Scaling

Dennard scaling ignored the “**leakage current**” and “**threshold voltage**”, which establish a baseline of power per transistor.

- As transistors get smaller,
 - Leakage current increases => Power density increases
 - Leakage & threshold voltage do not scale well with size
- These created a “Power Wall” that has limited practical processor frequency to around 4 GHz since 2006

Source: William Gropp, cs598, www.cs.illinois.edu/~wgropp

Power Wall

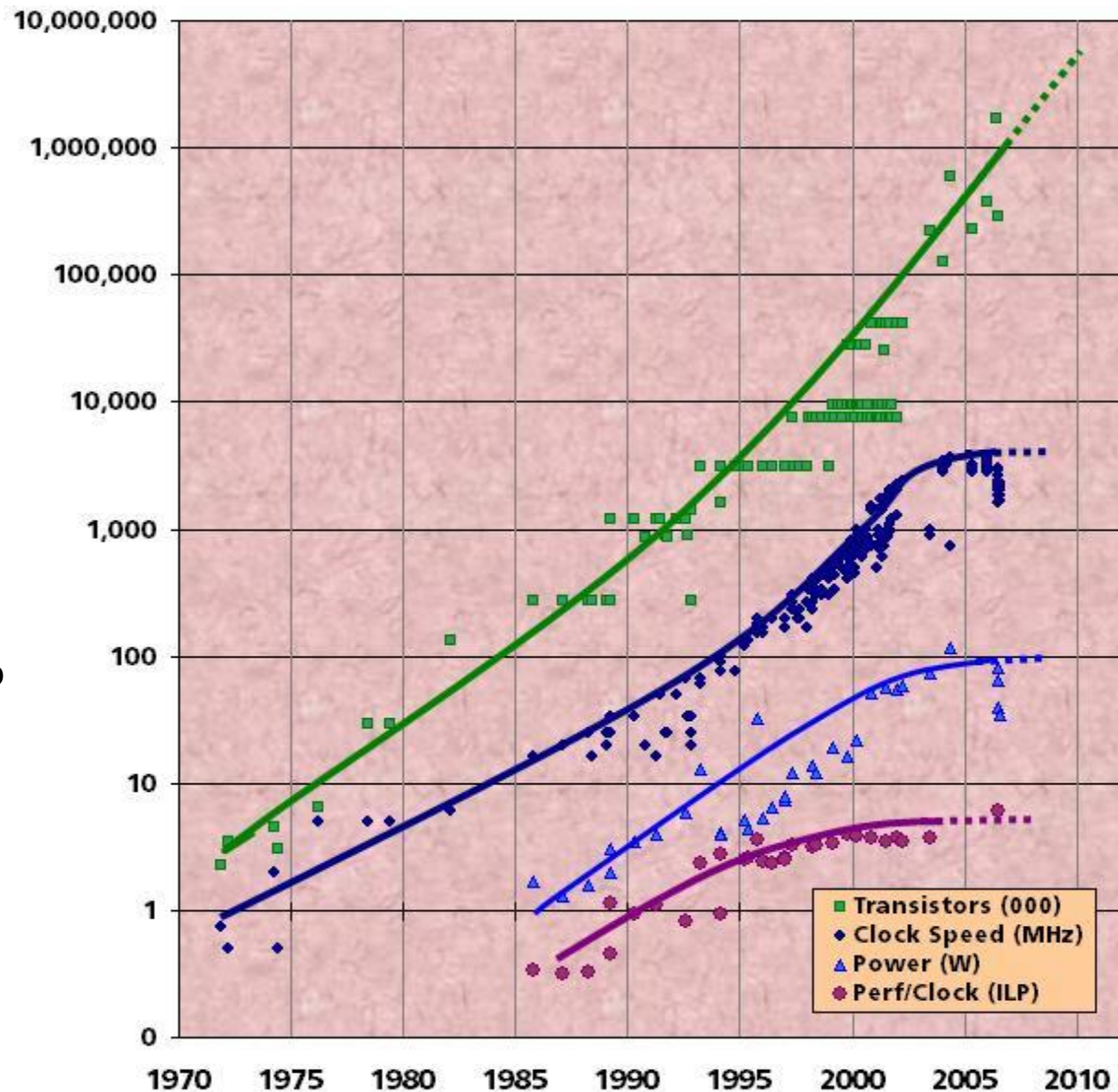
Ο «γνήσιος» νόμος του Moore συνεχίζει να ισχύει!

Chip density is continuing increase
~2x every 2 years

- **Clock speed is not**
- **Number of processor cores doubles instead**

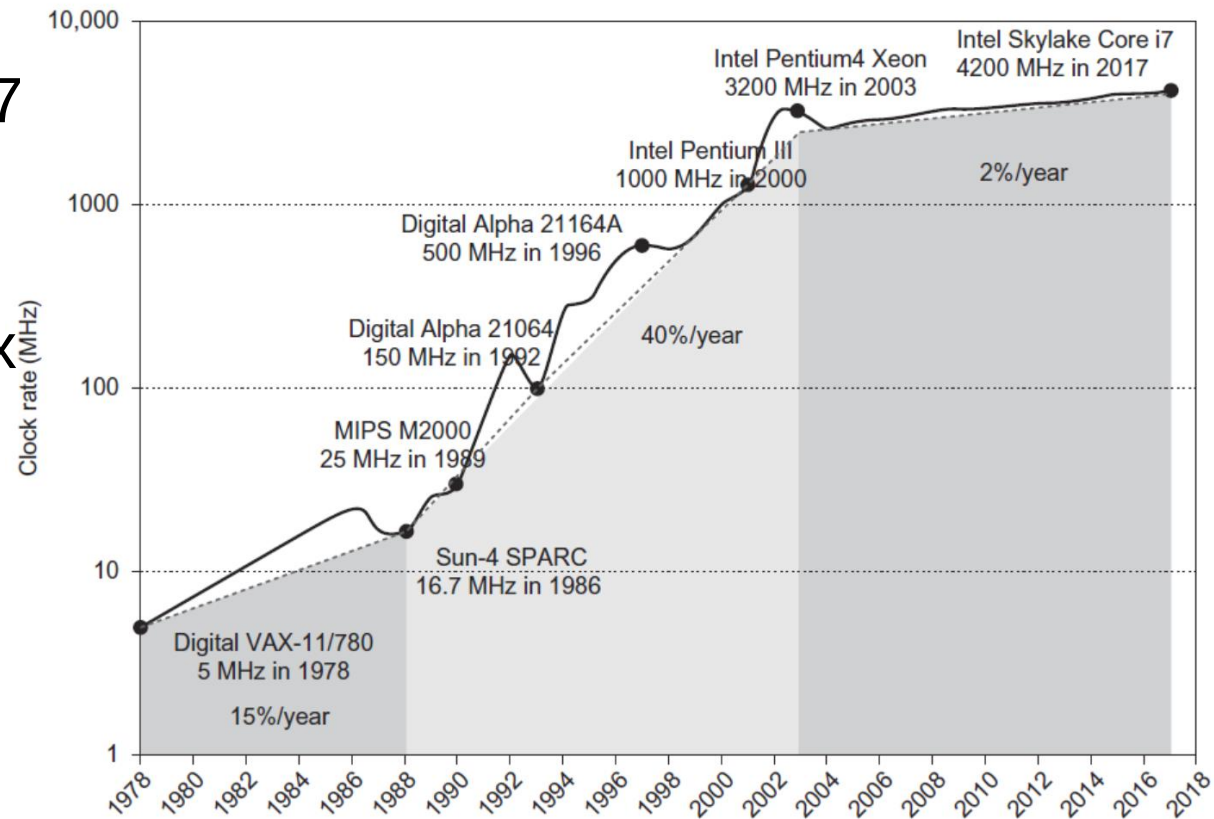
There is little or no hidden parallelism (ILP) to be found
Parallelism must be exposed to and managed by software

Source: Intel, Microsoft (Sutter) and Stanford (Olukotun, Hammond)



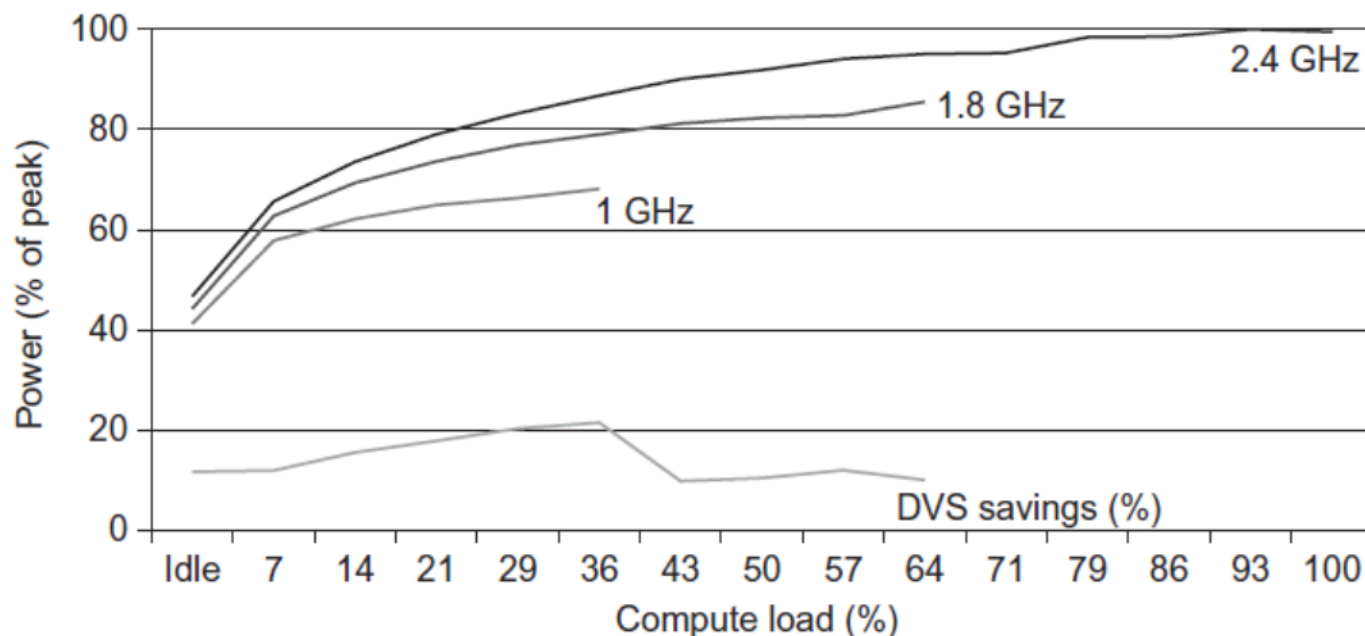
Power

- Intel 80386 consumed ~ 2 W
- 3.3 GHz Intel Core i7 consumes 130 W
- Heat must be dissipated from 1.5×1.5 cm chip
- This is the limit of what can be cooled by air
- Liquid cooling?
- Liquid immersion?
- Cool chips are faster, hot chips are slower!



Reducing Power

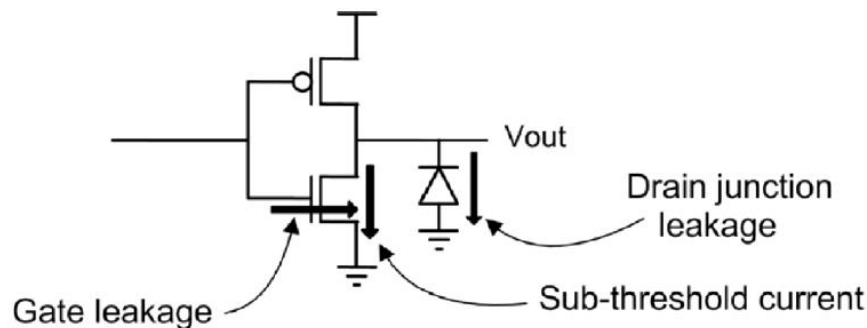
- Techniques for reducing power:
 - Dynamic Voltage Scaling (DVS)
 - Dynamic Frequency Scaling (DFS)
 - Dynamic Voltage-Frequency Scaling (DVFS)



- Low power state for DRAM, disks (sleep modes)
- Overclocking, then turn off cores (race-to-halt)

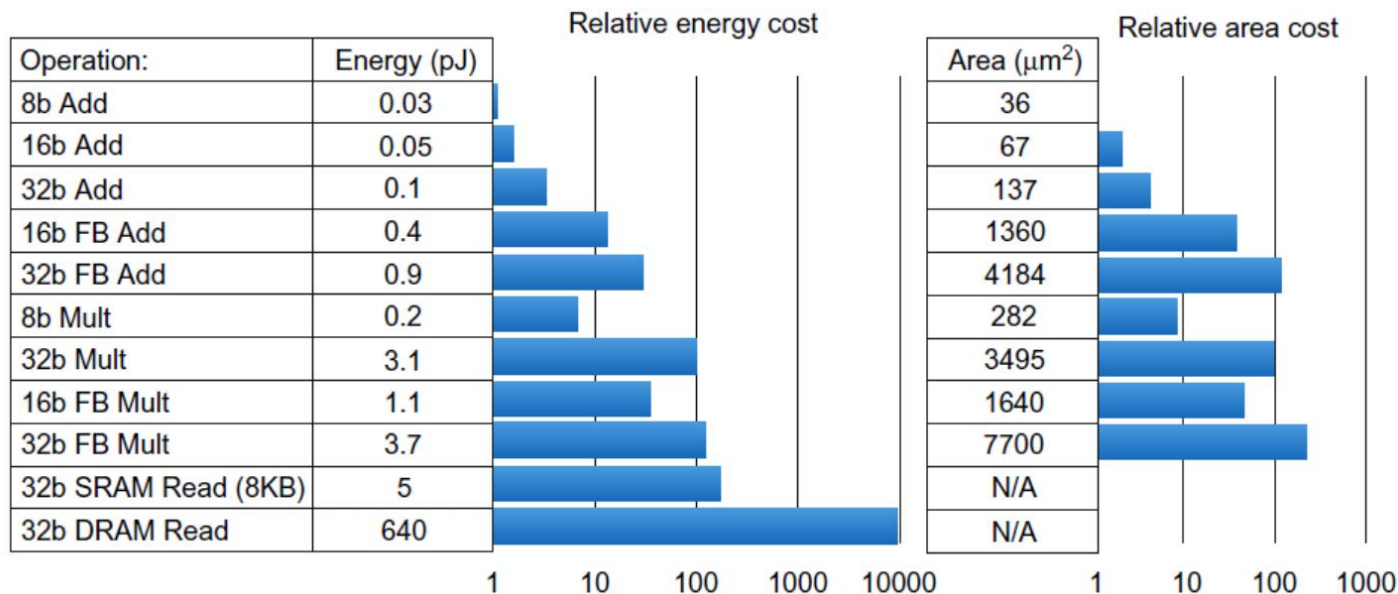
Static Power

- Static power consumption
 - $\text{Current}_{\text{static}} \times \text{Voltage}$
 - Why? Leakage in transistors, capacitors...
 - Scales with number of transistors
 - To reduce: power gating



Power

- The new primary evaluation for design innovation
 - Tasks per joule
 - Performance per watt



- Why use both energy & power metrics?

Trends in Cost

- Cost driven down by learning curve
 - Yield
- DRAM: price closely tracks cost
- Microprocessors: price depends on volume
 - 10% reduction for each doubling of volume

Integrated Circuit Cost

- Integrated circuit

$$\text{Cost of integrated circuit} = \frac{\text{Cost of die} + \text{Cost of testing die} + \text{Cost of packaging and final test}}{\text{Final test yield}}$$

$$\text{Cost of die} = \frac{\text{Cost of wafer}}{\text{Dies per wafer} \times \text{Die yield}}$$

$$\text{Dies per wafer} = \frac{\pi \times (\text{Wafer diameter}/2)^2}{\text{Die area}} - \frac{\pi \times \text{Wafer diameter}}{\sqrt{2} \times \text{Die area}}$$

- Bose-Einstein formula (!)

$$\text{Die yield} = \text{Wafer yield} \times 1 / (1 + \text{Defects per unit area} \times \text{Die area})^N$$

- Defects per unit area = 0.016-0.057 defects per square cm (2010)
- N = process-complexity factor = 11.5-15.5 (40 nm, 2010)
- *The manufacturing process dictates the wafer cost, wafer yield and defects per unit area*
- *The architect's design affects the die area, which in turn affects the defects and cost per die*

Dependability

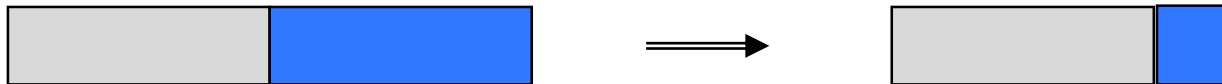
- Systems alternate between two states of service with respect to Service Level Agreements/ Objectives (SLA/SLO):
 - Service accomplishment, where service is delivered as specified by SLA
 - Service interruption, where the delivered service is different from the SLA
- Module reliability:
 - Mean time to failure (MTTF)
 - Mean time to repair (MTTR)
 - Mean time between failures (MTBF) = $MTTF + MTTR$
 - Availability = $MTTF / MTBF$

Measuring Performance

- Typical performance metrics:
 - Response time
 - Throughput
- Speedup of X relative to Y
 - $\text{Execution time}_Y / \text{Execution time}_X$
- Execution time
 - Wall clock time: includes all system overheads
 - CPU time: only computation time
- Benchmarks
 - Kernels (e.g. matrix multiply)
 - Toy programs (e.g. sorting)
 - Synthetic benchmarks (e.g. Dhrystone)
 - Benchmark suites (e.g. SPEC06fp, TPC-C)

Principles of Computer Design

- Take Advantage of Parallelism
 - e.g. multiple processors, disks, memory banks, pipelining, multiple functional units
- Principle of Locality
 - Reuse of data and instructions
- Focus on the Common Case!!! Amdahl's Law:



$$\text{Execution time}_{\text{new}} = \text{Execution time}_{\text{old}} \times \left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)$$

$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

Amdahl's Law

$$\text{ExTime}_{\text{new}} = \text{ExTime}_{\text{old}} \left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)$$

$$\text{Speedup}_{\text{overall}} = \frac{\text{ExTime}_{\text{old}}}{\text{ExTime}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

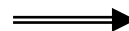
Best you could ever hope to get:

$$F=0.1 \Rightarrow S \sim 1.1$$

$$F=0.5 \Rightarrow S = 2$$

$$F=0.9 \Rightarrow S = 10$$

$$\text{Speedup}_{\text{maximum}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}})}$$



Principles of Computer Design

■ The Processor Performance Equation

CPU time = CPU clock cycles for a program $\times$ Clock cycle time

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

CPU time = Instruction count $\times$ Cycles per instruction $\times$ Clock cycle time

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

Principles of Computer Design

- Different instruction types having different CPIs

$$\text{CPU clock cycles} = \sum_{i=1}^n \text{IC}_i \times \text{CPI}_i$$

$$\text{CPU time} = \left(\sum_{i=1}^n \text{IC}_i \times \text{CPI}_i \right) \times \text{Clock cycle time}$$

Fallacies and Pitfalls

- All exponential laws must come to an end
 - Dennard scaling (constant power density)
 - Stopped by threshold voltage
 - Disk capacity
 - 30-100% per year to 5% per year
 - Moore's Law
 - Most visible with DRAM capacity
 - ITRS disbanded
 - Only four foundries left producing state-of-the-art logic chips
 - 3 nm might be the limit

Fallacies and Pitfalls

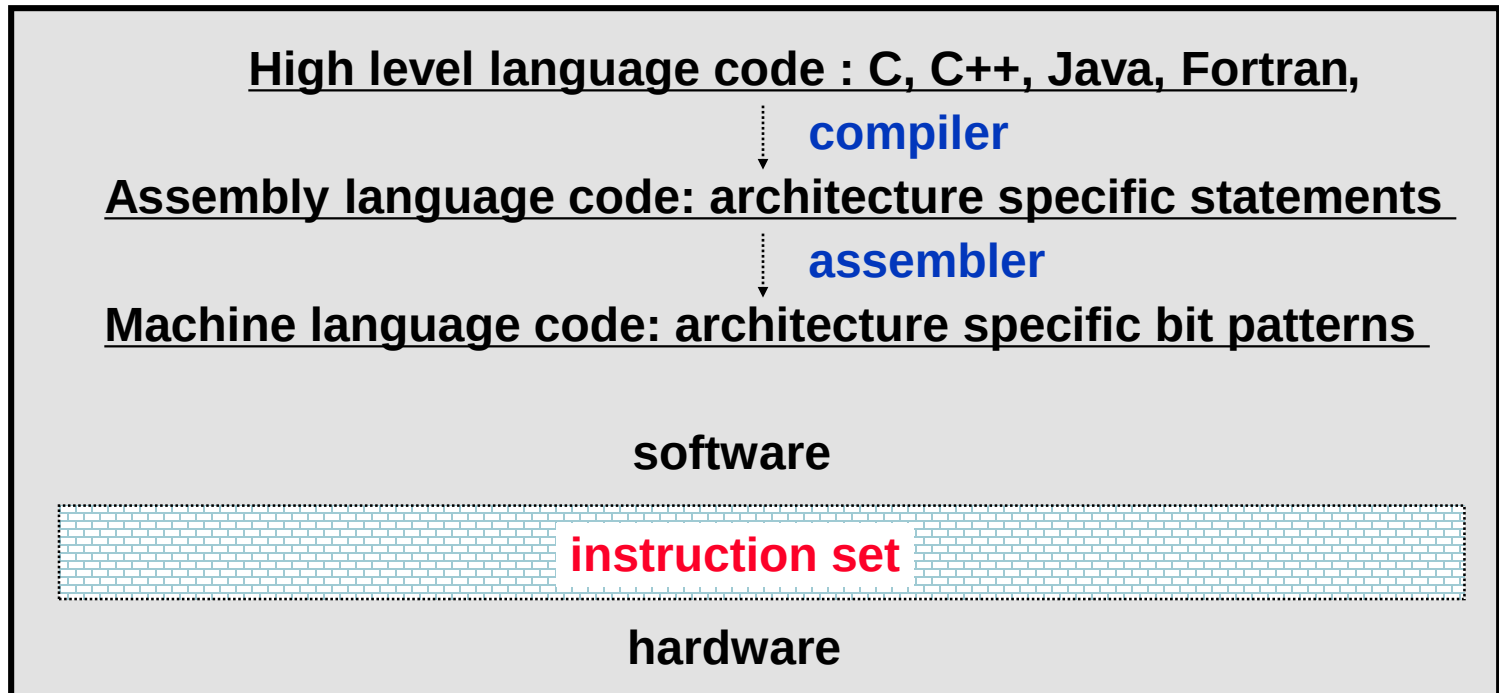
- Microprocessors are a silver bullet
 - Performance is now a programmer's burden
- Falling prey to Amdahl's Law
- A single point of failure
- Hardware enhancements that increase performance also improve energy efficiency, or are at worst energy neutral
- Benchmarks remain valid indefinitely
 - Compiler optimizations target benchmarks!

Fallacies and Pitfalls

- The rated mean time to failure of disks is 1,200,000 hours or almost 140 years, so disks practically never fail
 - MTTF value from manufacturers assume regular replacement
- Failure is random (e.g. for disks of the same batch)
- Peak performance tracks observed performance
- Fault detection can lower availability
 - Not all operations are needed for correct execution

Instruction Set Architecture (ISA)

- Serves as an **interface** between software and hardware.
- Provides a mechanism by which the software **tells the hardware what should be done.**



Instruction Set Design Issues

- Instruction set design issues include:
 - Where are operands stored?
 - registers, memory, stack, accumulator
 - How many explicit operands are there?
 - 0, 1, 2, or 3
 - How is the operand location specified?
 - register, immediate, indirect, . . .
 - What type & size of operands are supported?
 - byte, int, float, double, string, vector. . .
 - What operations are supported?
 - add, sub, mul, move, compare . . .

Classifying ISAs

Accumulator (before 1960, e.g. 68HC11):

1-address	add A	$\text{acc} \leftarrow \text{acc} + \text{mem}[A]$
-----------	-------	--

Stack (1960s to 1970s):

0-address	add	$\text{tos} \leftarrow \text{tos} + \text{next}$
-----------	-----	--

Memory-Memory (1970s to 1980s):

2-address	add A, B	$\text{mem}[A] \leftarrow \text{mem}[A] + \text{mem}[B]$
3-address	add A, B, C	$\text{mem}[A] \leftarrow \text{mem}[B] + \text{mem}[C]$

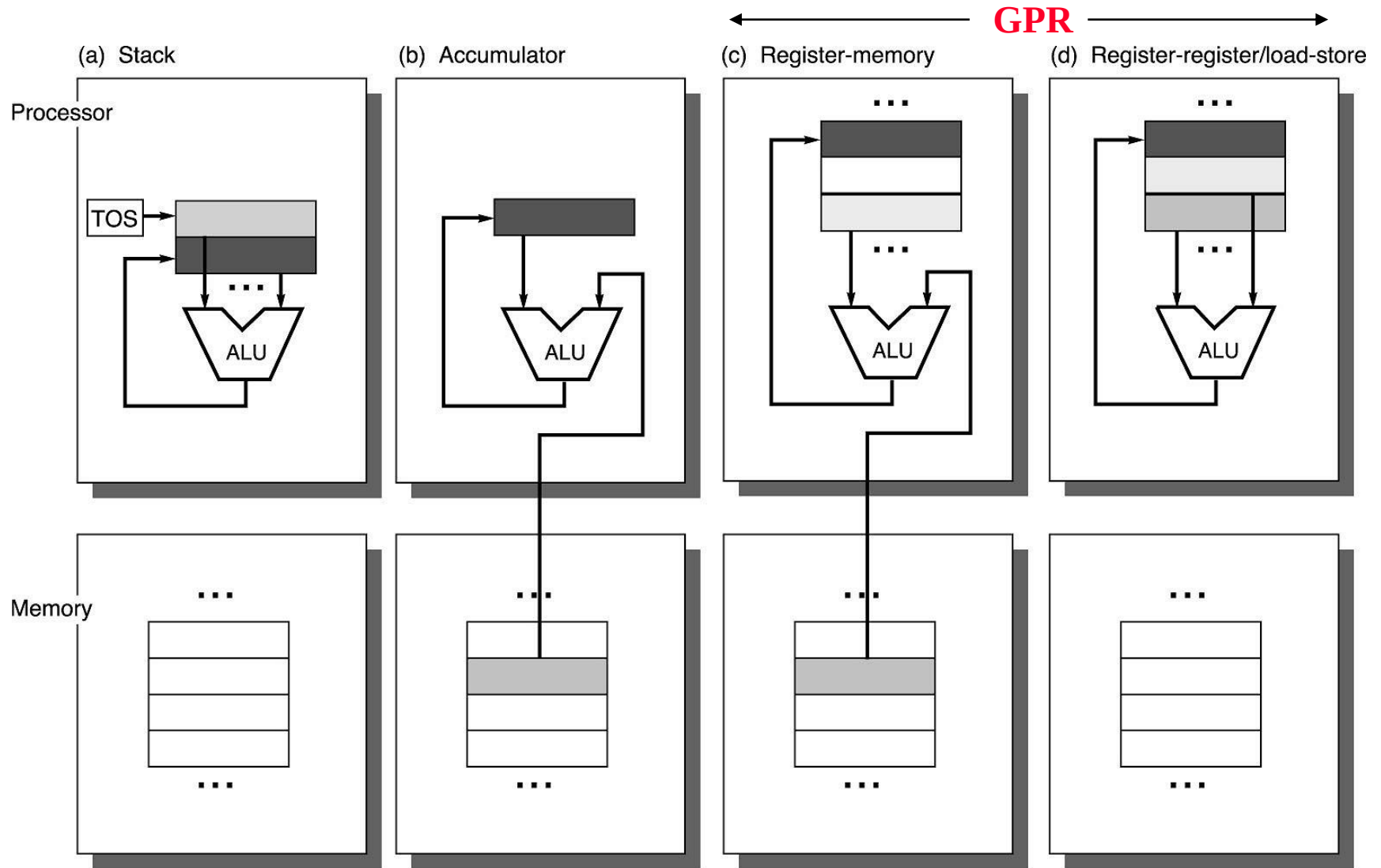
Register-Memory (1970s to present, e.g. 80x86):

2-address	add R1, A	$R1 \leftarrow R1 + \text{mem}[A]$
	load R1, A	$R1 \leftarrow \text{mem}[A]$

Register-Register (Load/Store/RISC) ('60s to present, e.g. MIPS):

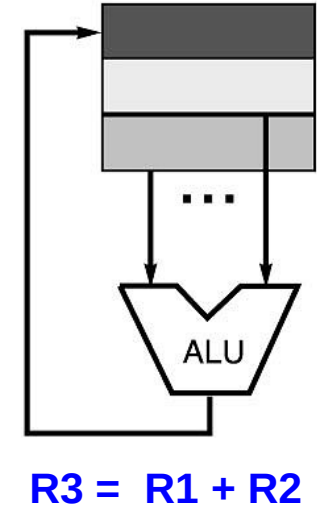
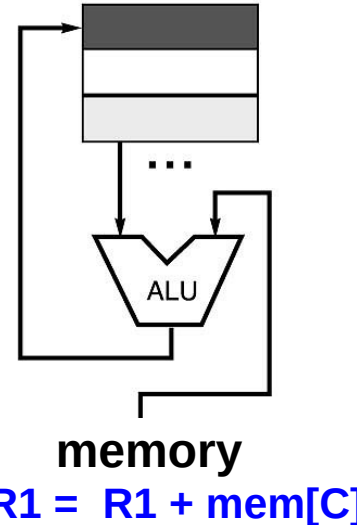
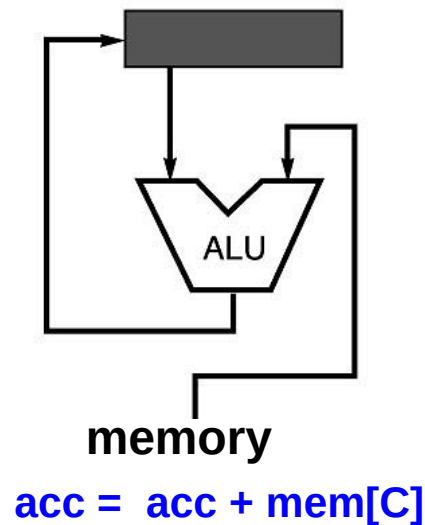
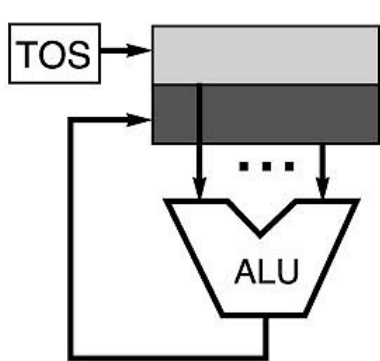
3-address	add R1, R2, R3	$R1 \leftarrow R2 + R3$
	load R1, R2	$R1 \leftarrow \text{mem}[R2]$
	store R1, R2	$\text{mem}[R1] \leftarrow R2$

Operand Locations in Four ISA Classes



Code Sequence $C = A + B$ for 4 ISAs

Stack	Accumulator	Register (register-memory)	Register (load- store)
Push A Push B Add Pop C	Load A Add B Store C	Load R1, A Add R1, B Store C, R1	Load R1, A Load R2, B Add R3, R1, R2 Store C, R3



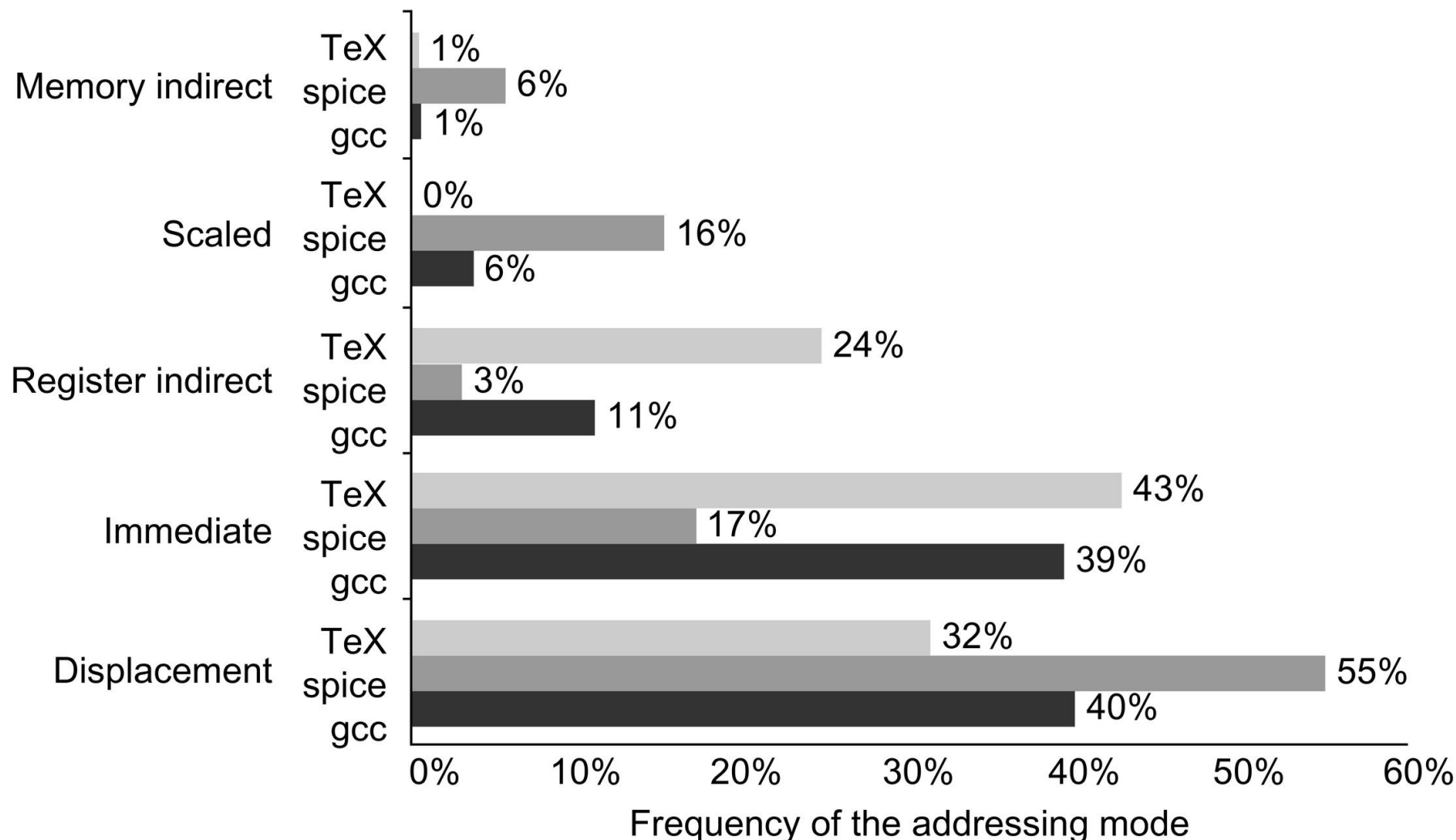
Types of Addressing Modes (VAX)

Addressing Mode	Example	Action
1. Register direct	Add R4, R3	$R4 \leftarrow R4 + R3$
2. Immediate	Add R4, #3	$R4 \leftarrow R4 + 3$
3. Displacement	Add R4, 100(R1)	$R4 \leftarrow R4 + M[100 + R1]$
4. Register indirect	Add R4, (R1)	$R4 \leftarrow R4 + M[R1]$
5. Indexed	Add R4, (R1 + R2)	$R4 \leftarrow R4 + M[R1 + R2]$
6. Direct	Add R4, (1000)	$R4 \leftarrow R4 + M[1000]$
7. Memory Indirect	Add R4, @(R3)	$R4 \leftarrow R4 + M[M[R3]]$
8. Autoincrement	Add R4, (R2)+	$R4 \leftarrow R4 + M[R2]$ $R2 \leftarrow R2 + d$
9. Autodecrement	Add R4, (R2)-	$R4 \leftarrow R4 + M[R2]$ $R2 \leftarrow R2 - d$
10. Scaled	Add R4, 100(R2)[R3]	$R4 \leftarrow R4 +$ $M[100 + R2 + R3*d]$

- [Clark and Emer]: modes 1-4 => **93%** of all operands on the VAX!!!

Types of Addressing Modes (VAX)

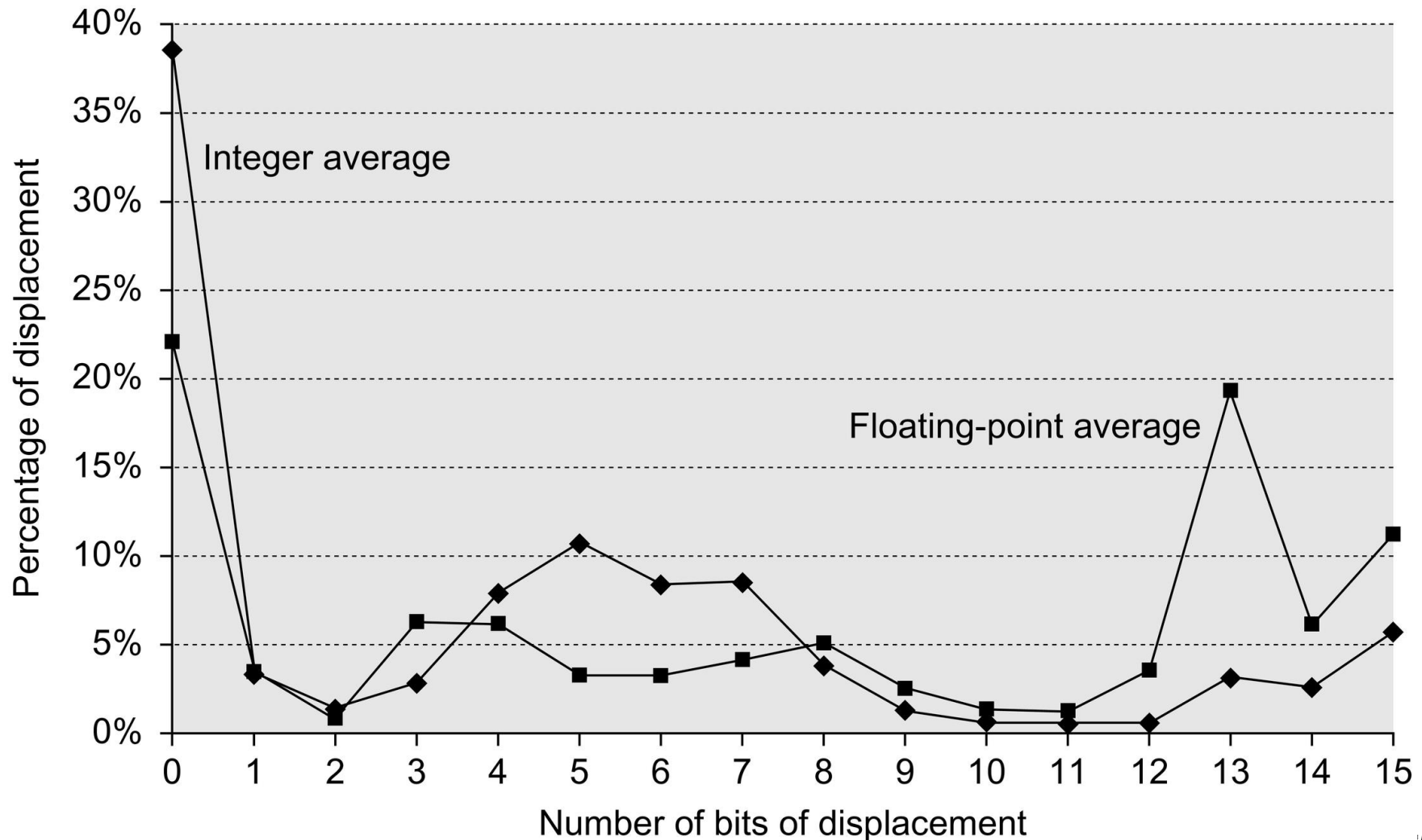
[Clark and Emer]: modes 1-4 => **93%** of all operands on the VAX!!!



Address Offset bits (Alpha)

SPECINT200 & SPECFP2000 optimized on Alpha

Sign bit is not included



Immediate values

On a VAX that supported
32 bit immediate
values:

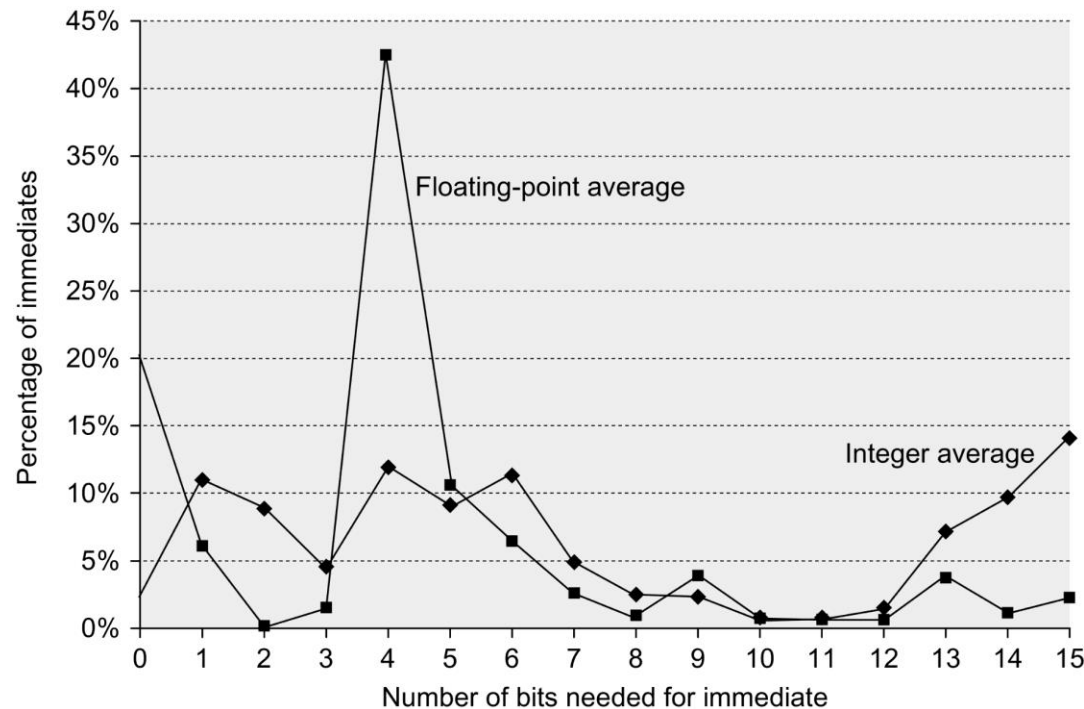
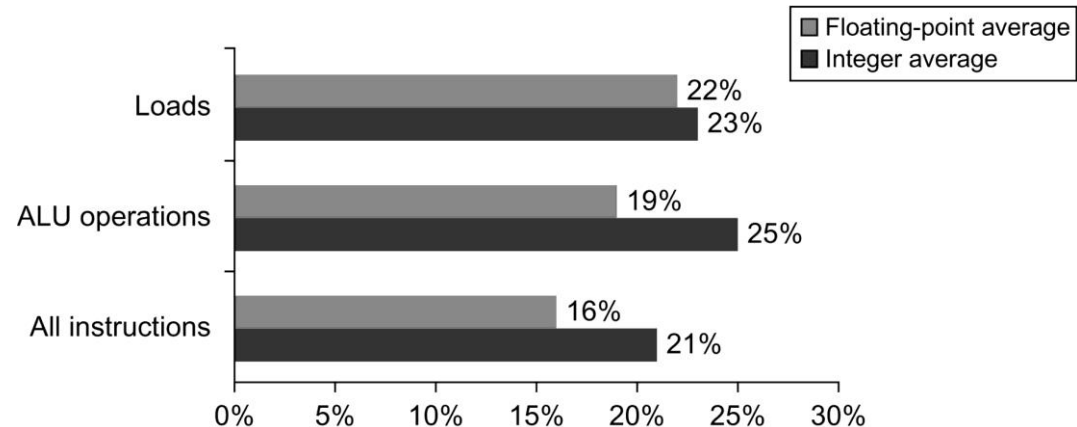
~20% of instructions use
immediate values

20-25% of values need
>16 bits

16 bits suffice for ~80%

8 bits suffice for ~50%

The plots are on an Alpha
for SPECINT and
SPECFP 2000

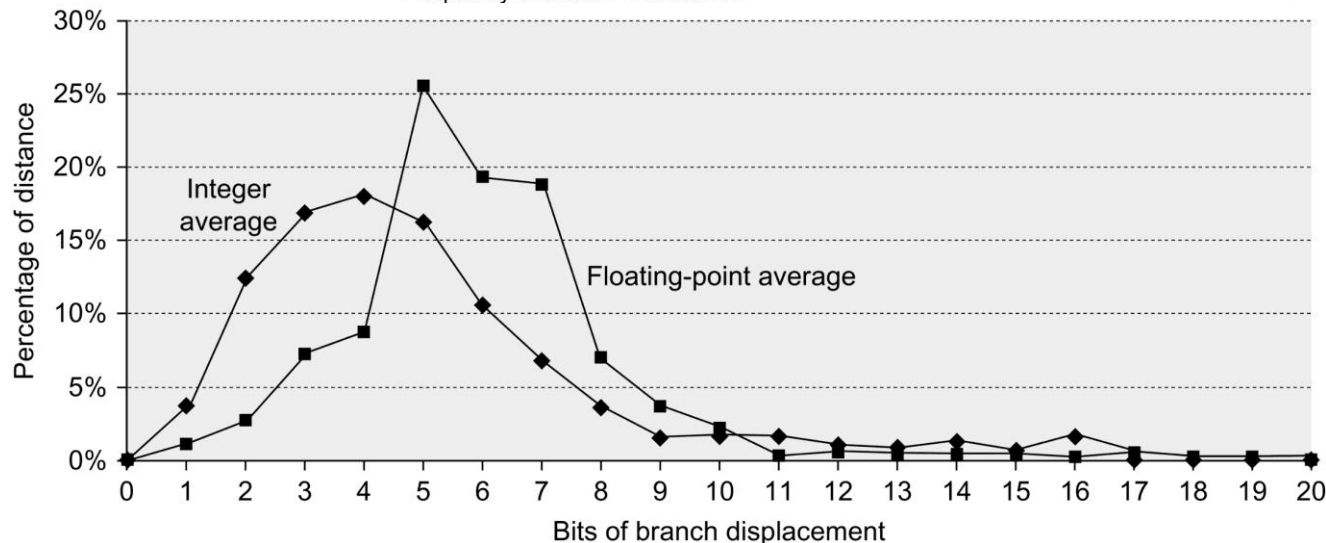
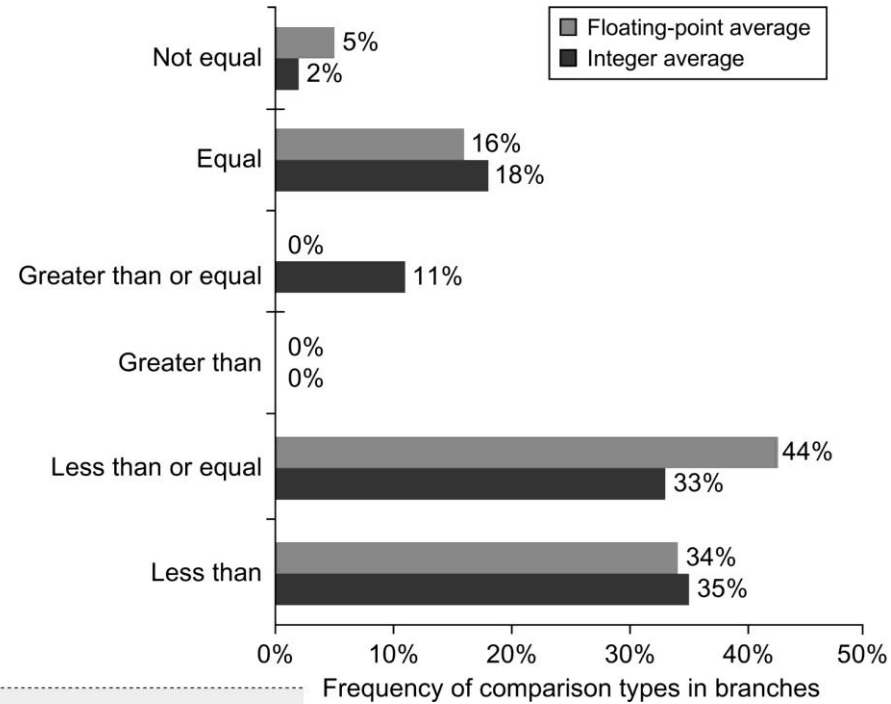
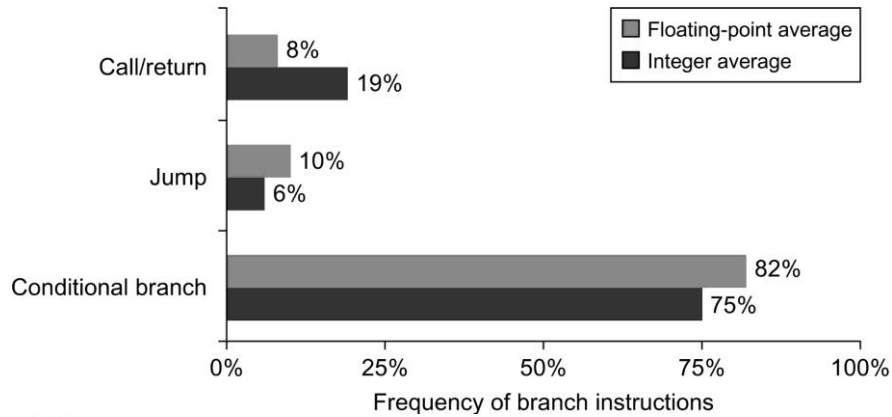


Branches

Most are conditional

Branch offset bits ≤ 16 (10-11 are OK)

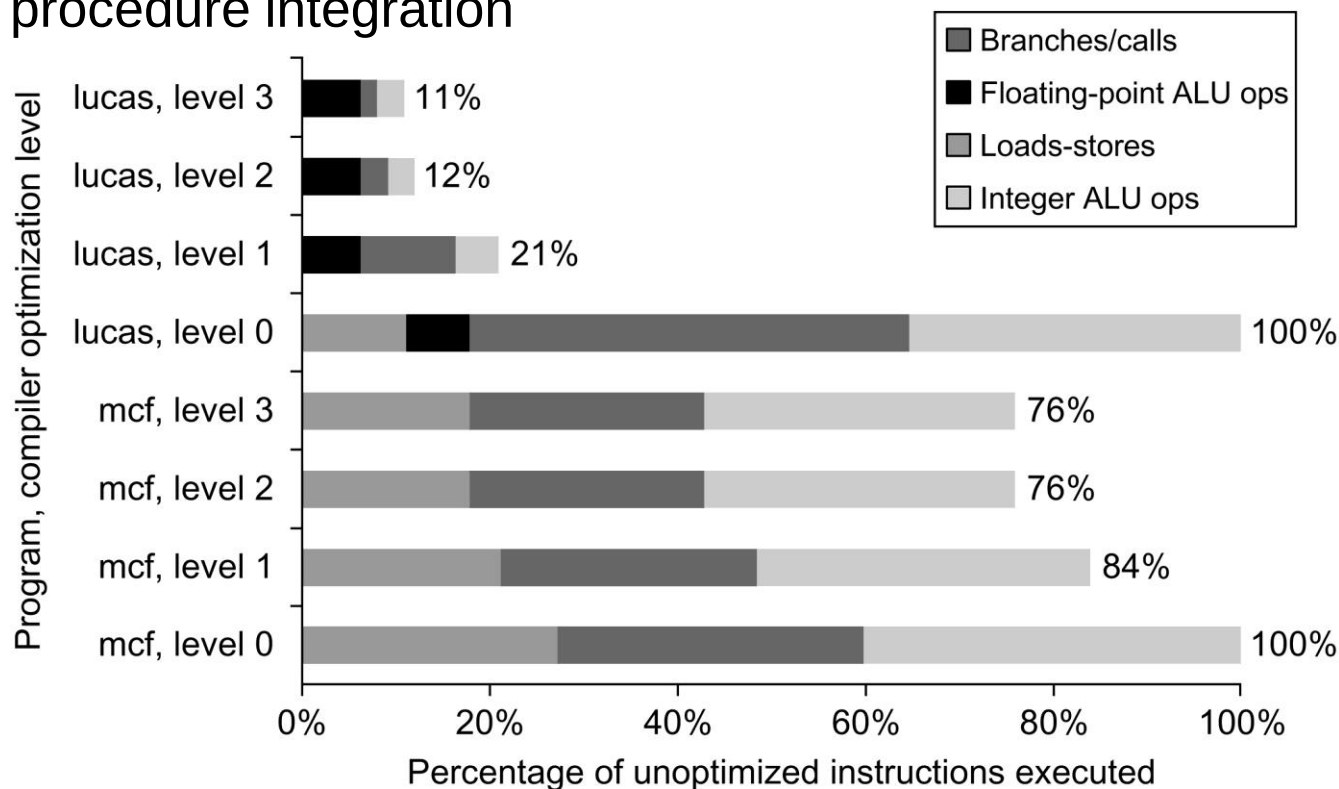
Eq/Ne tests 20%, $\{<, \leq\}$ ~70%



Compilers

Experiments performed on Alpha compilers

- O0 => no optimizations
- O1: +local optimizations, code scheduling, and local register allocation
- O2: +global optimizations, loop transformations (software pipelining), and global register allocation
- O3: + procedure integration

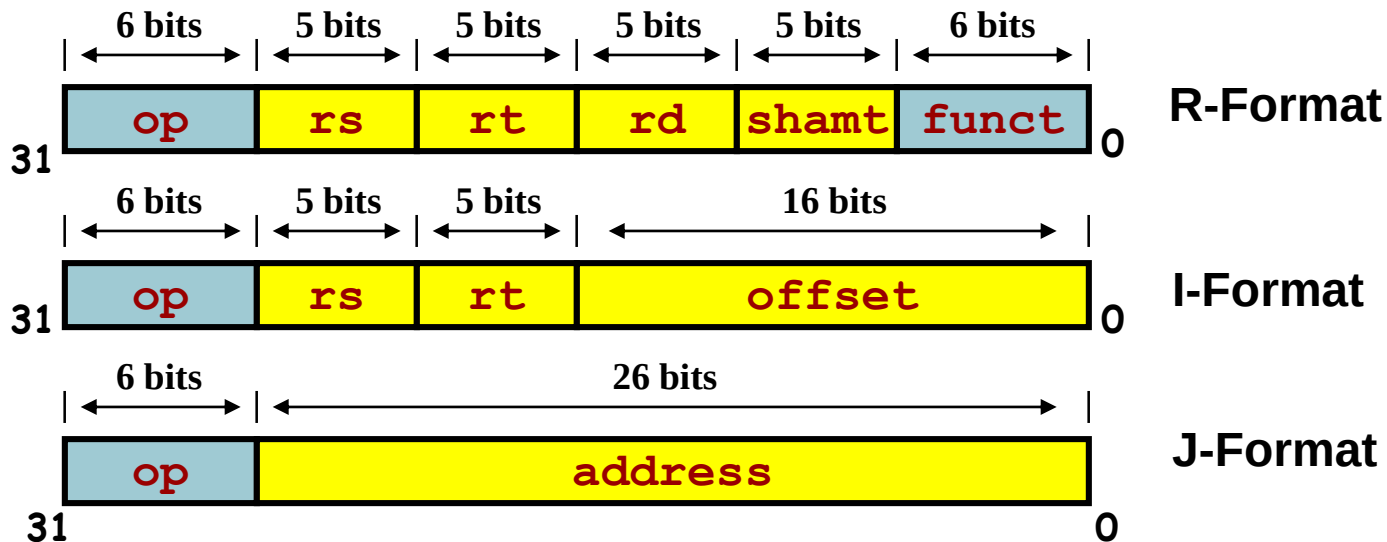


Types of Operations

- Arithmetic and Logic: AND, ADD
- Data Transfer: MOVE, LOAD, STORE
- Control: BRANCH, JUMP, CALL
- System: OS CALL, VM
- Floating Point: ADDF, MULF, DIVF
- Decimal: ADDD, CONVERT
- String: MOVE, COMPARE
- Graphics: (DE)COMPRESS

MIPS Instructions

- All instructions exactly 32 bits wide
- Different formats for different purposes
- Similarities in formats ease implementation



MIPS Instruction Types

- **Arithmetic & Logical** - manipulate data in registers

add \$s1, \$s2, \$s3 $\$s1 = \$s2 + \$s3$
or \$s3, \$s4, \$s5 $\$s3 = \$s4 \text{ OR } \$s5$

- **Data Transfer** - move register data to/from memory → load & store

lw \$s1, 100(\$s2) $\$s1 = \text{Memory}[\$s2 + 100]$
sw \$s1, 100(\$s2) $\text{Memory}[\$s2 + 100] = \$s1$

- **Branch** - alter program flow

beq \$s1, \$s2, 25 if ($\$s1 == \$s2$) $\text{PC} = \text{PC} + 4 + 4 * 25$
else $\text{PC} = \text{PC} + 4$

MIPS Arithmetic & Logical Instructions

- Instruction usage (assembly)

add dest, src1, src2	dest = src1 + src2
sub dest, src1, src2	dest = src1 - src2
and dest, src1, src2	dest = src1 AND src2

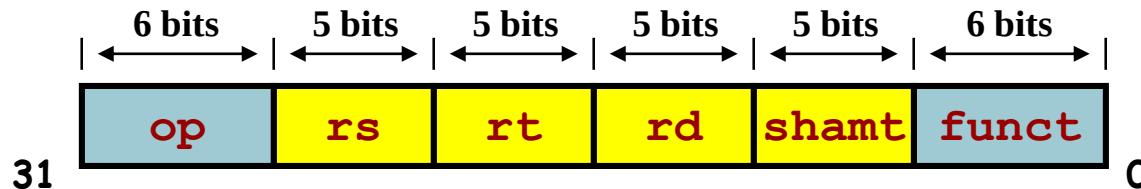
- Instruction characteristics

- Always 3 operands: destination + 2 sources
- Operand order is fixed
- Operands are always general purpose registers

- Design Principles:

- Design Principle 1: **Simplicity favors regularity**
- Design Principle 2: **Smaller is faster***

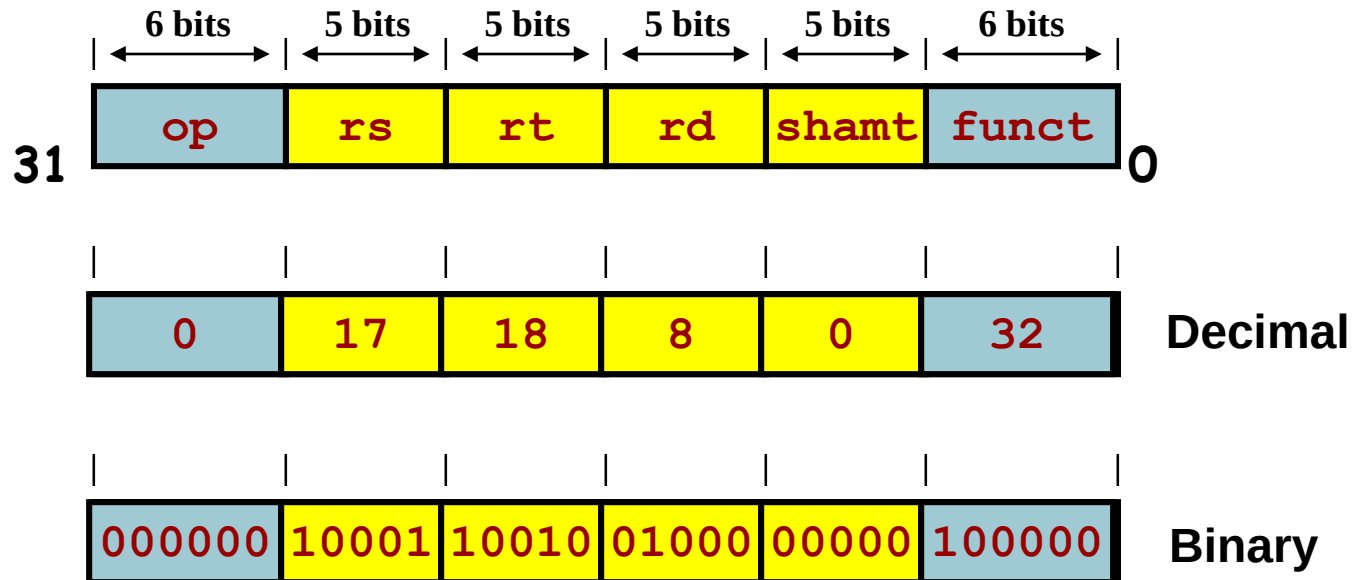
Arithmetic & Logical Instructions



- Used for arithmetic, logical, shift instructions
 - **op**: Basic operation of the instruction (*opcode*)
 - **rs**: first register source operand
 - **rt**: second register source operand
 - **rd**: register destination operand
 - **shamt**: shift amount (more about this later)
 - **funct**: function - specific type of operation
- Also called “**R-Format**” or “**R-Type**” Instructions

Arithmetic & Logical Instructions

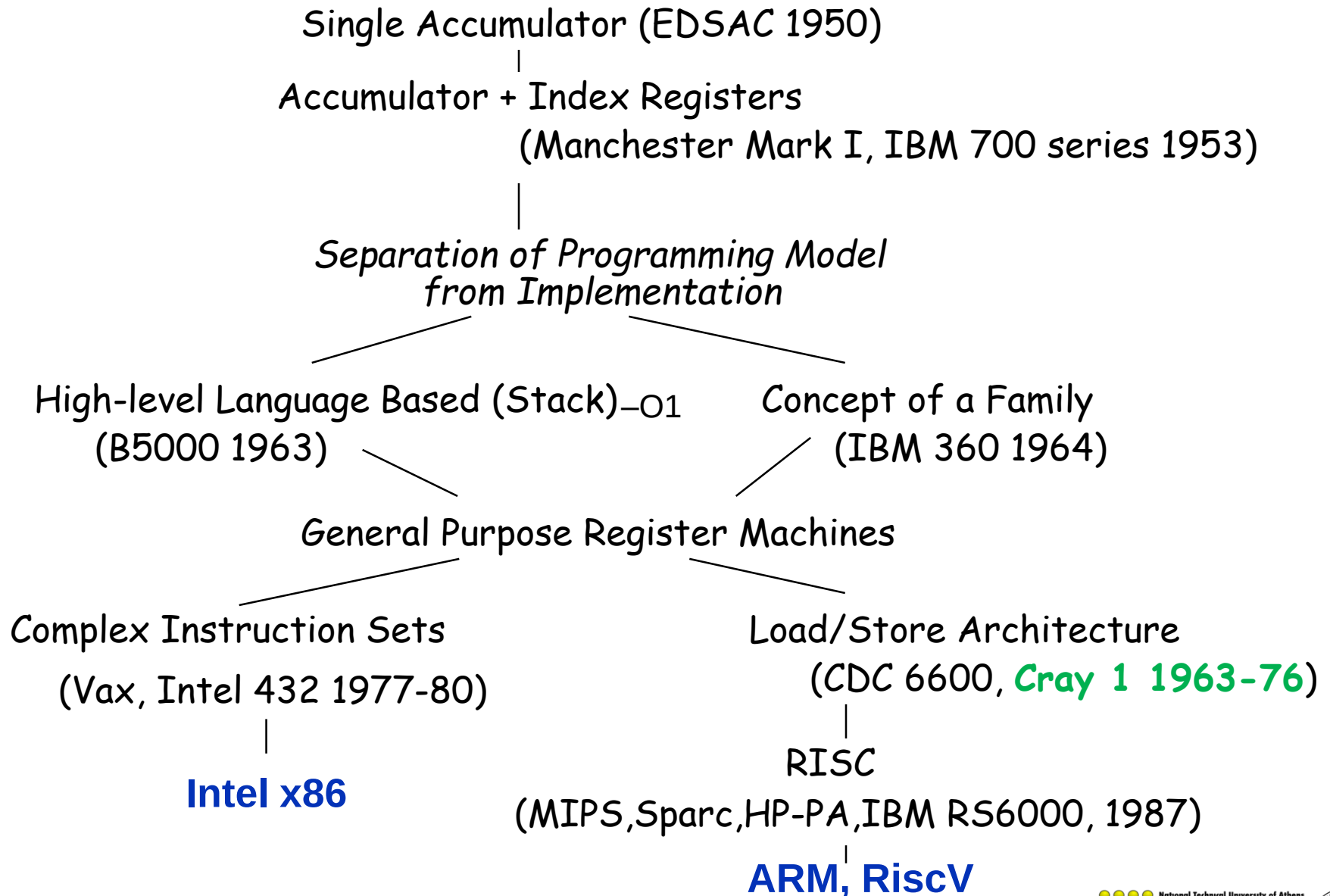
- Machine language for
`add $8, $17, $18`
- See reference card for `op`, `funct` values



MIPS Data Transfer Instructions

- Transfer data between registers and memory
- Instruction format (assembly)
 - lw \$dest, offset(\$addr) load word
 - sw \$src, offset(\$addr) store word
- Uses:
 - Accessing a variable in main memory
 - Accessing an array element

Evolution of Instruction Sets



Current RISC Instruction Sets

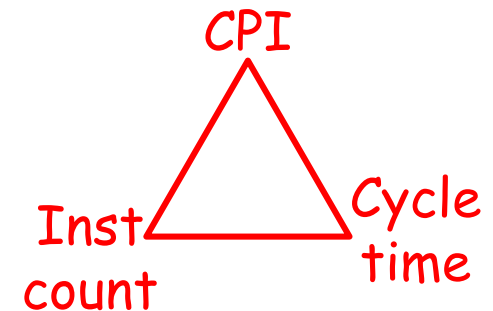
Converged features

	ARMv8	MIPS64 R6	Power v3.0	RV64G	SPARCV9
Original date (base ISA)	1986	1986	1990	2016	1987
Date of this ISA	2011	2014	2013	2016	2008
Instruction size (bits)	32	32	32	32	32
Address space (size, model)	64 bits (flat)	64 bits, flat	64 bits, flat	64 bits, flat	64 bits, flat
Data alignment	Aligned preferred	Aligned preferred	Unaligned	Aligned preferred	Aligned
Data addressing modes	8 (including scaled, pre/post increment)	1 (+1 for FP only)	4	1	2
Integer registers (number, model, size)	31 GPR x 64, plus stack pointer	31 GPR x 64 bits	31 GPR x 64 bits	31 GPR x 64 bits	31 GPR x 64 bits
Separate floating-point registers	32x32 or 32x64 bits	32 x 32 or 32 x 64 bits	32 x 32 or 32 x 64 bits	32 x 32 or 32 x 64 bits	32 x 32 or 32 x 64 bits
Floating-point format	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double

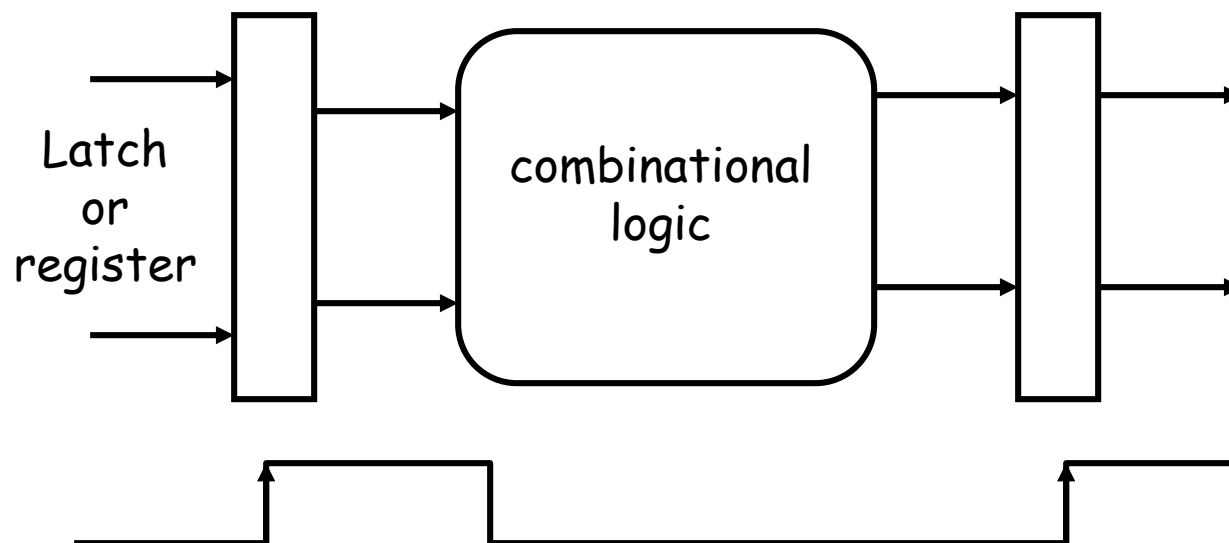
Components of Performance

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

	Instr. count	CPI	Clock rate
Program	X		
Compiler	X	X	
Instruction Set Architecture (ISA)	X	X	X
Οργάνωση		X	X
Τεχνολογία			X



What's a Clock Cycle?



- Old days: 10 FO4 levels of gates (fan-out-of-four)
- Today: determined by numerous time-of-flight issues + gate delays
 - clock propagation, wire lengths, drivers
 - Still ~10-20...

Integrated Approach

What really matters is the functioning of the complete system, i.e. hardware, runtime system, compiler, and operating system

In networking, this is called the “End to End argument”

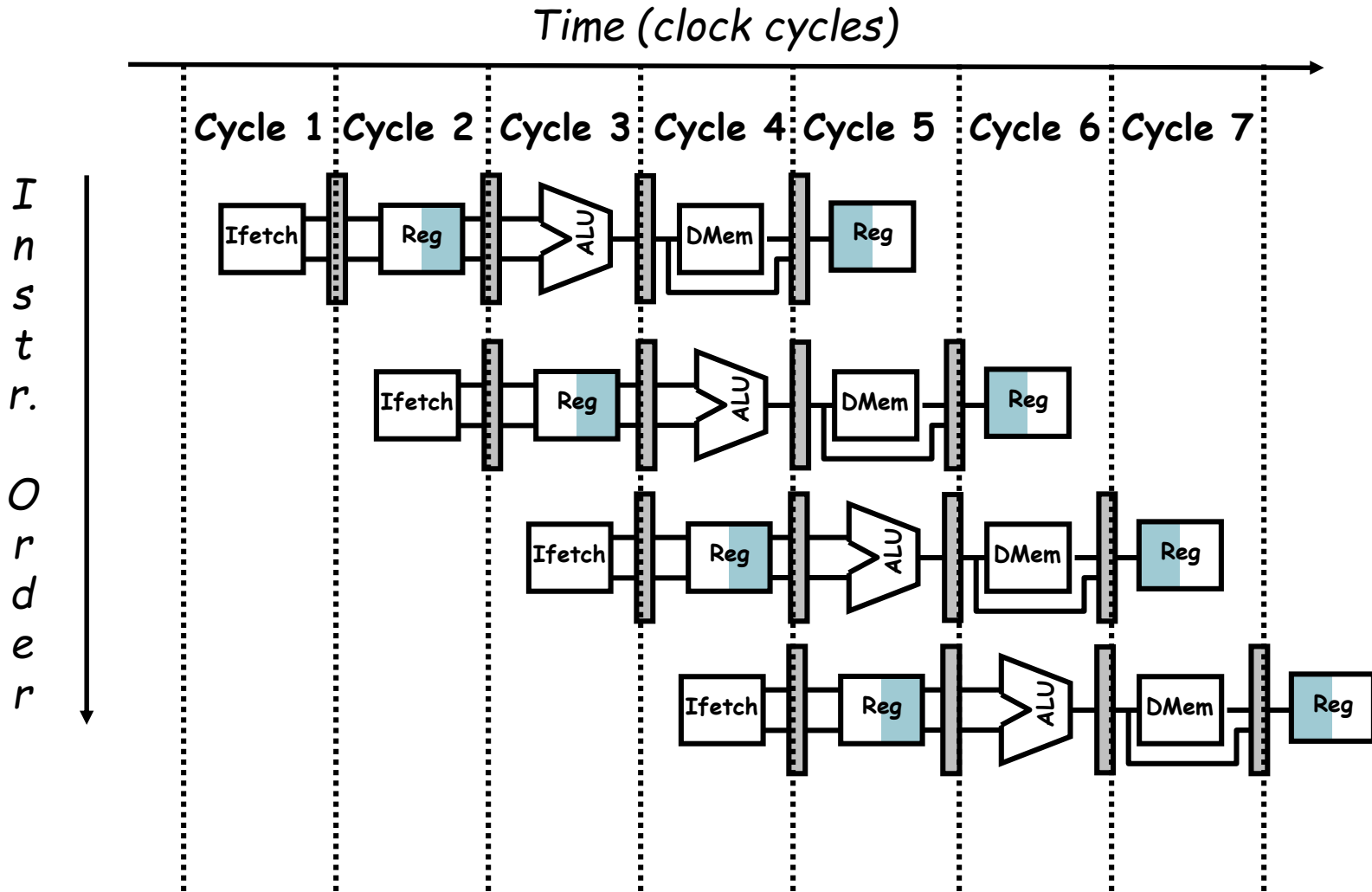
- Computer architecture is not just about transistors, individual instructions, or particular implementations
- Original RISC projects replaced complex instructions with a compiler + simple instructions

How to achieve better performance?

- ISA is a contract!
- But: underneath the ISA *illusion*....
 - Do more things at once (i.e. in parallel)
 - Do the things that you do faster

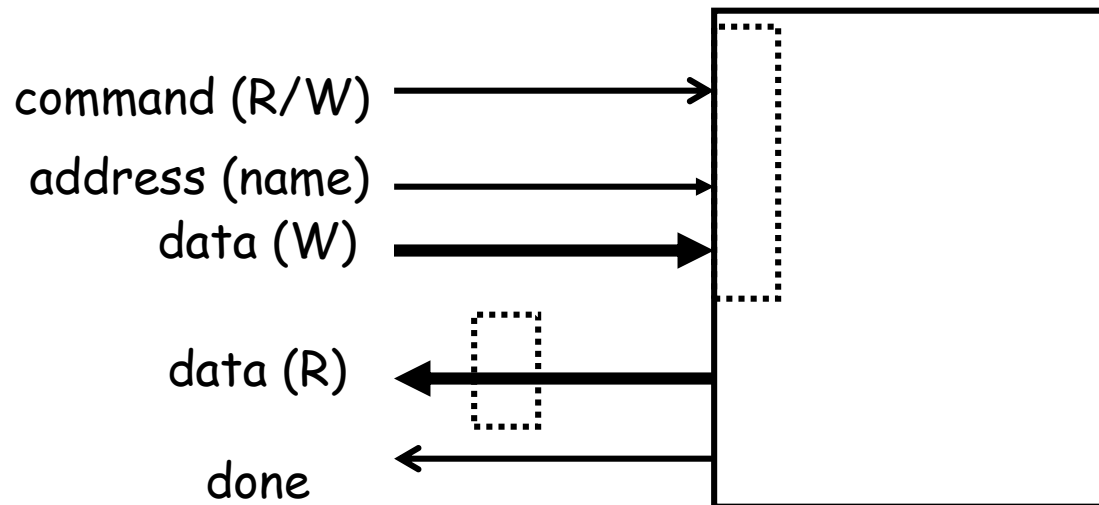
This is what computer architecture is all about!

Example Pipelining

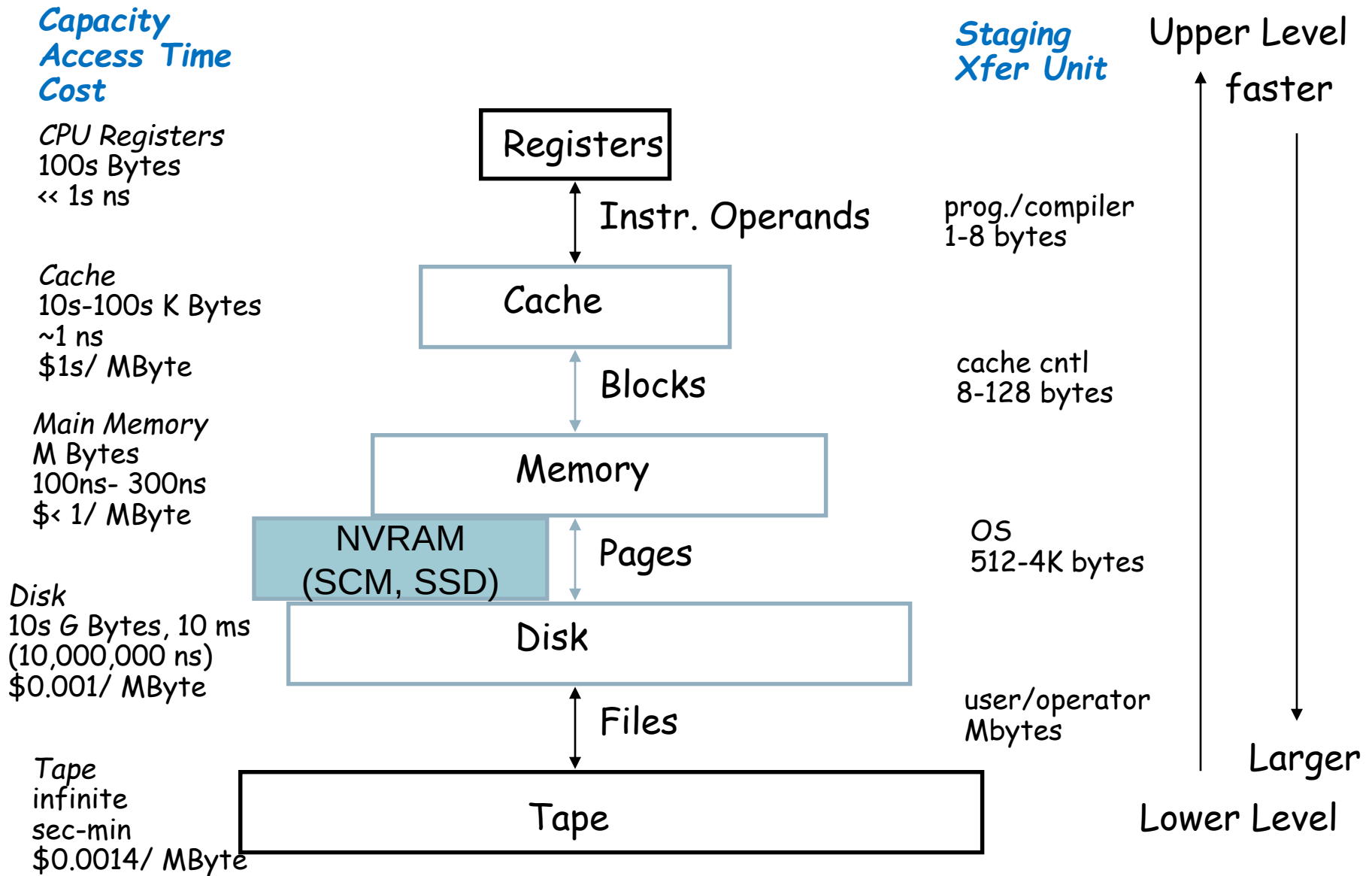


The Memory Abstraction

- Association of <name, value> pairs
 - typically named as byte addresses
 - often values aligned on multiples of size
- Sequence of Reads and Writes
- Write binds a value to an address
- Read of addr returns most recently written value bound to that address



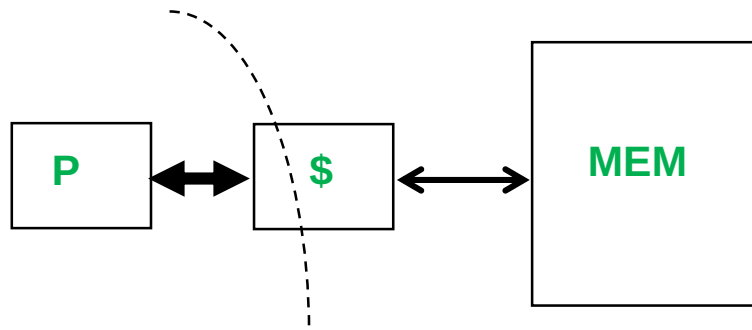
Levels of the Memory Hierarchy



circa 1995 numbers

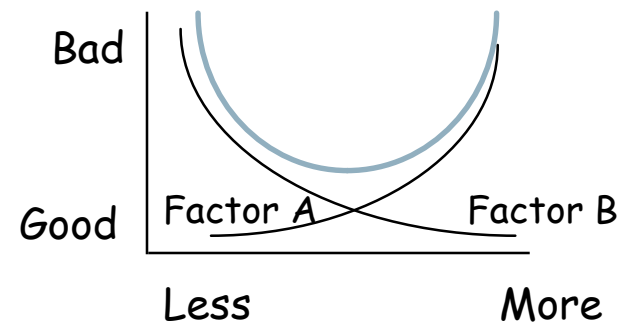
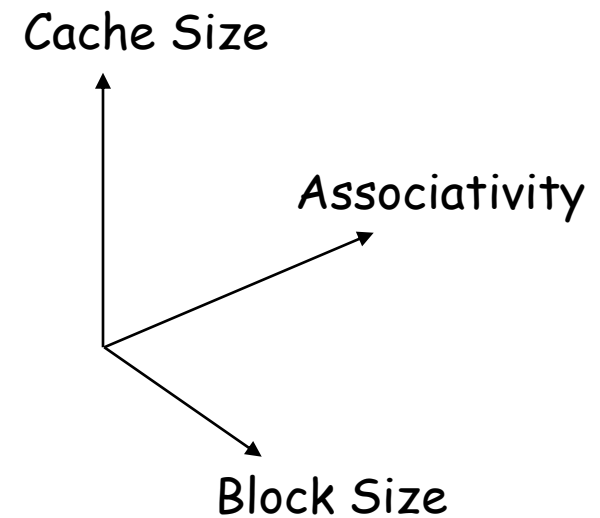
The Principle of Locality

- The Principle of Locality:
 - Program access a relatively small portion of the address space at any instant of time
- Two Different Types of Locality:
 - Temporal Locality (Locality in Time): If an item is referenced, it will tend to be referenced again soon (e.g., loops, reuse)
 - Spatial Locality (Locality in Space): If an item is referenced, items whose addresses are close by tend to be referenced soon (e.g., straightline code, array access)
- Last 30 years, HW relied on locality for speed



The Cache Design Space

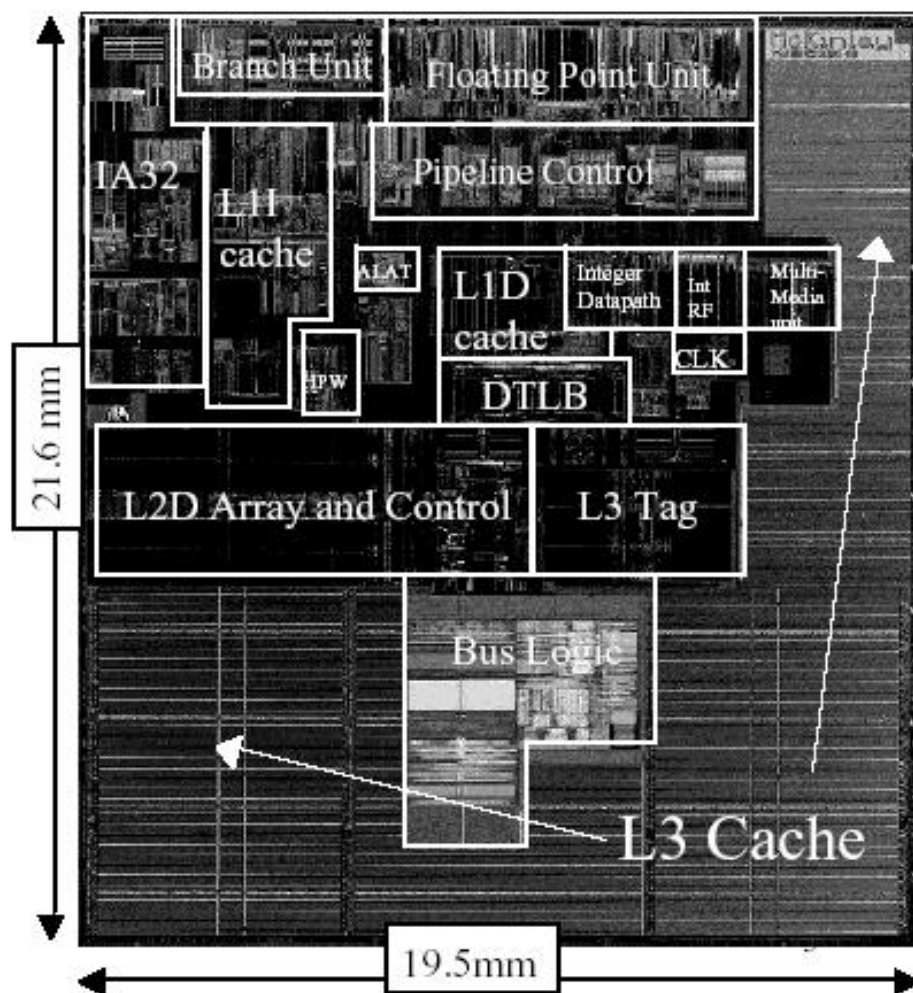
- Several interacting dimensions
 - cache size
 - block size
 - associativity
 - replacement policy
 - write-through vs write-back
- The optimal choice is a compromise
 - depends on access characteristics
 - workload
 - use (I-cache, D-cache, TLB)
 - depends on technology / cost
- Simplicity often wins



Is it all about memory system design?

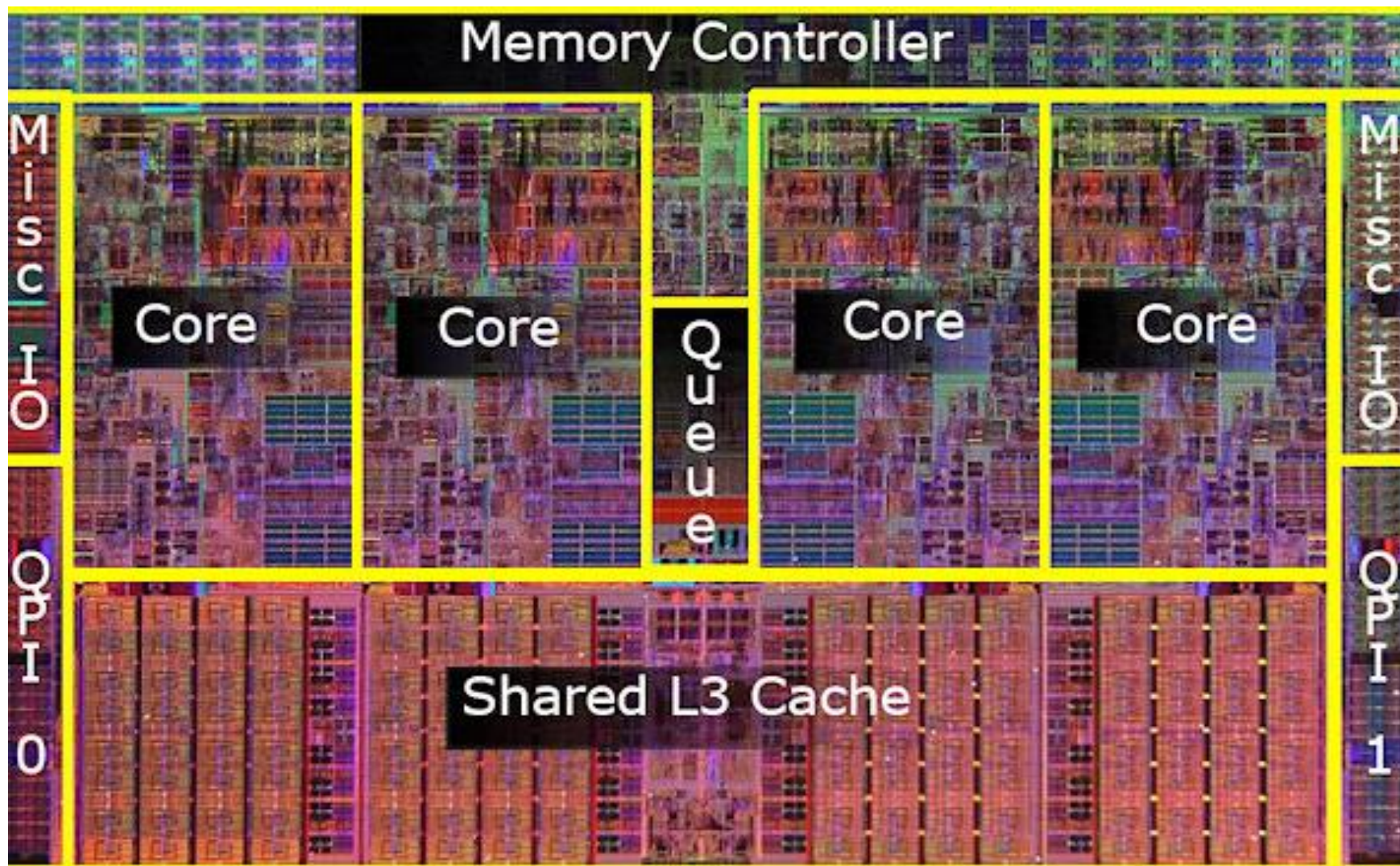
McKinley Floorplan

- 0.18 μm , AI process
- 200MHz system clock
- 1GHz core clock
- Core clocking:
 - 260 mm²
 - 1 primary driver
 - 5 repeaters
 - 33 delay SLCBs
 - 18k gated buffers
 - 157k clocked latches



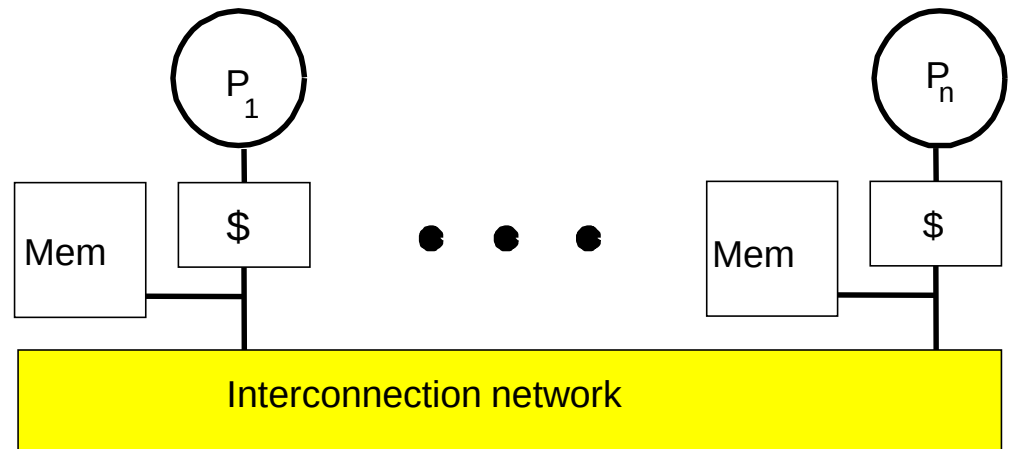
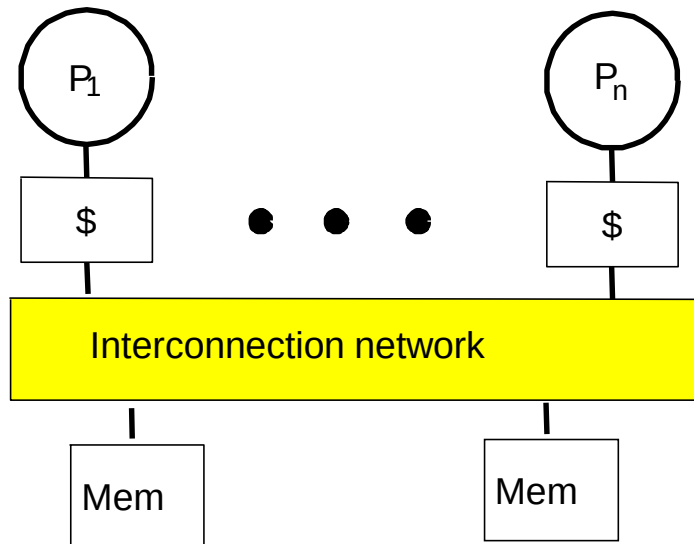
Is it all about memory system design?

Nahalem, ~2008



Memory Abstraction and Parallelism

- Maintaining the *illusion* of sequential access to memory
- What happens when multiple processors access the same memory at once?
 - Do they see a consistent picture?

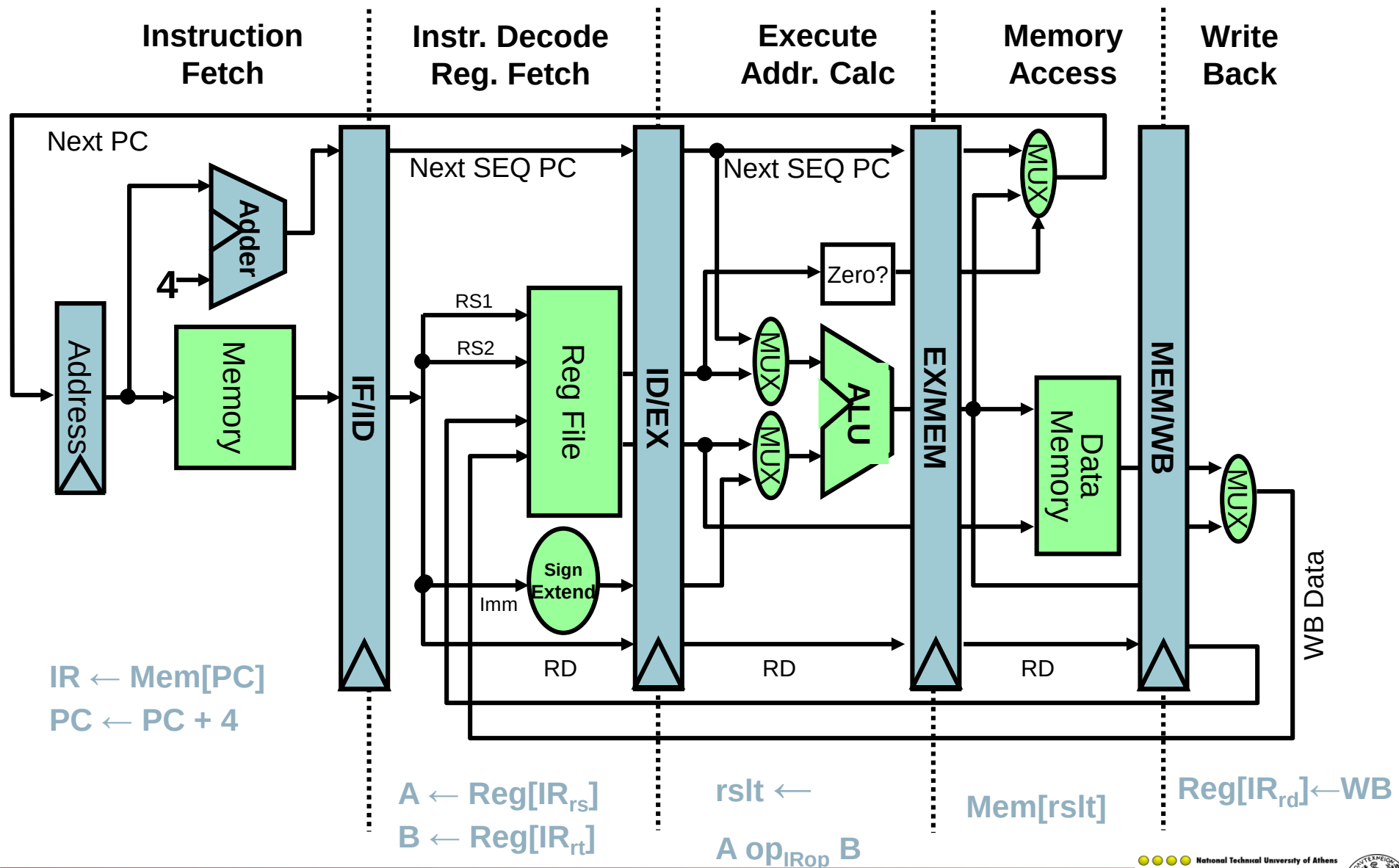


- Processing and processors embedded in the memory?
- Does/should the SW know about hierarchy? NUMA!

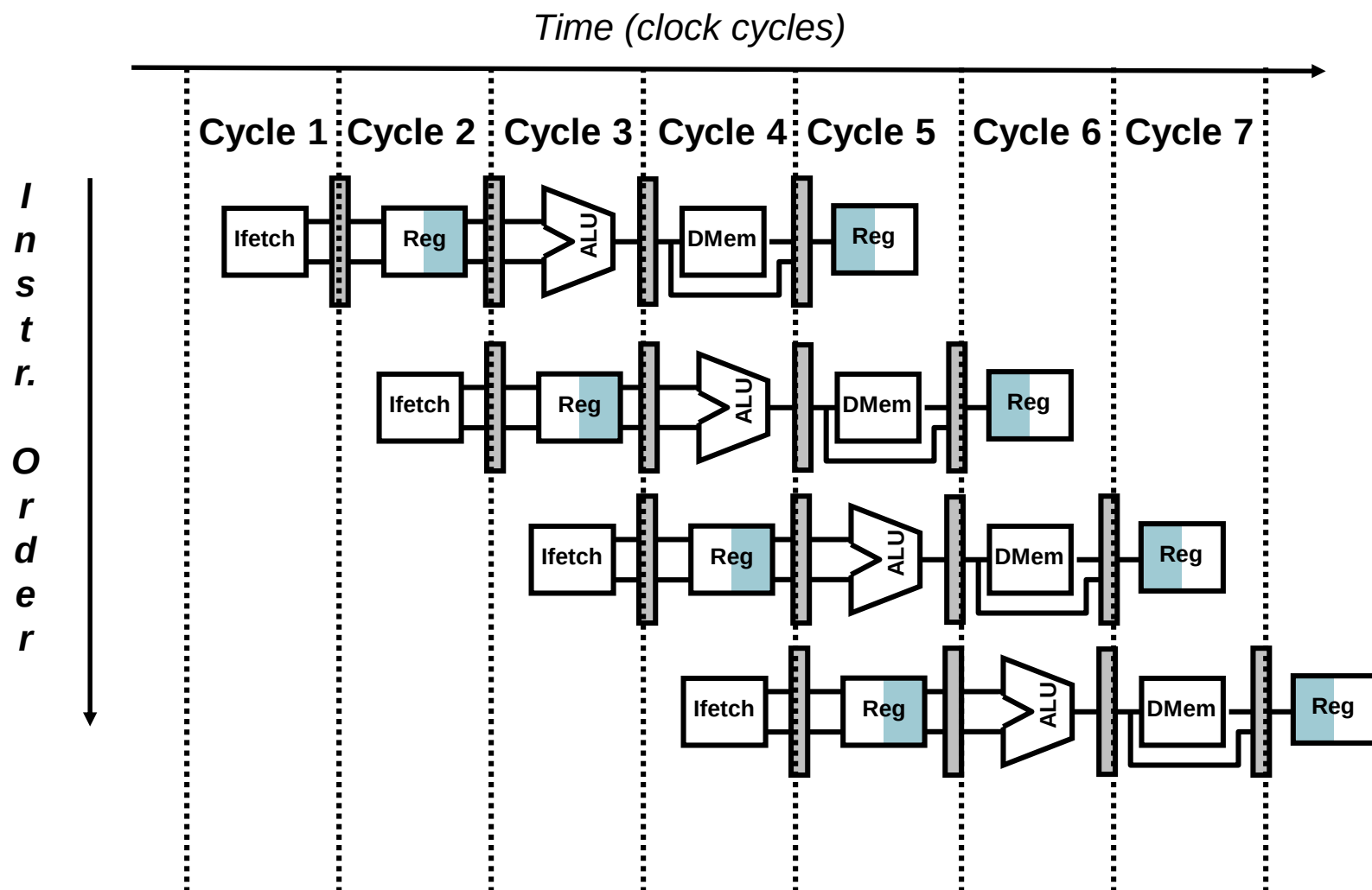
Basic pipelining principles

- The following slides are repeated from the Compute Architecture course
- Review the material, you are supposed to know this already!

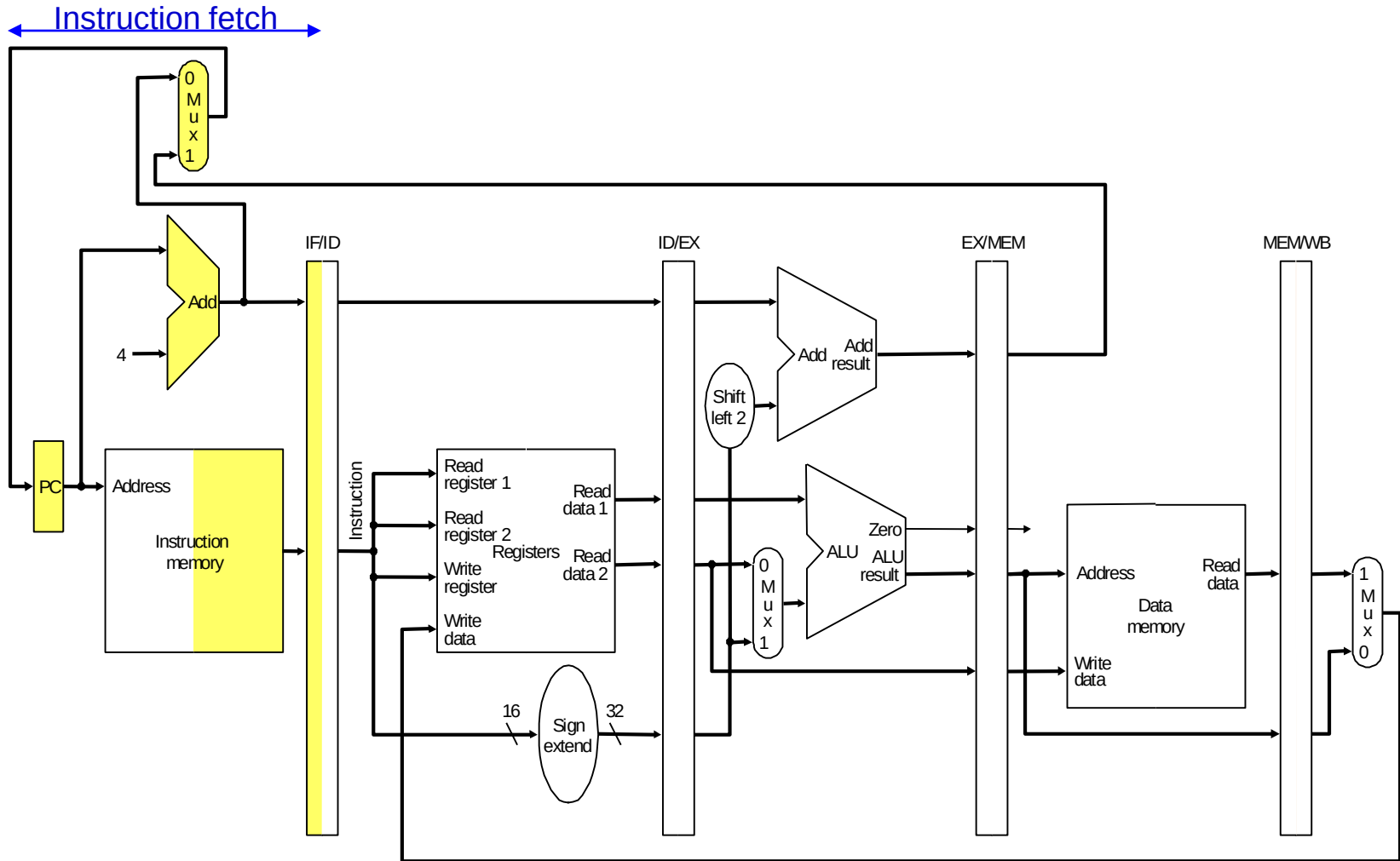
5-Stage Pipelined Datapath



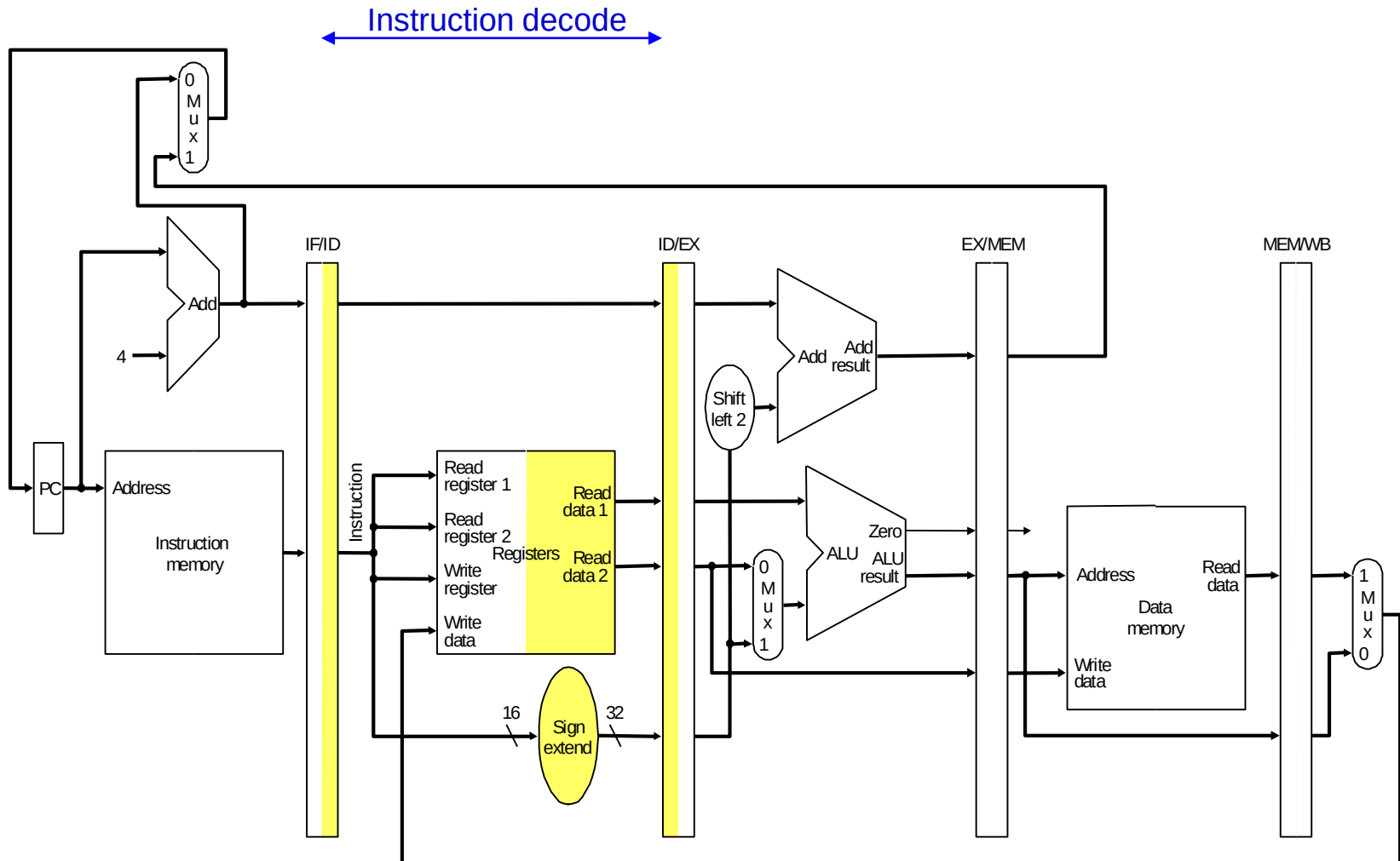
Timing diagram



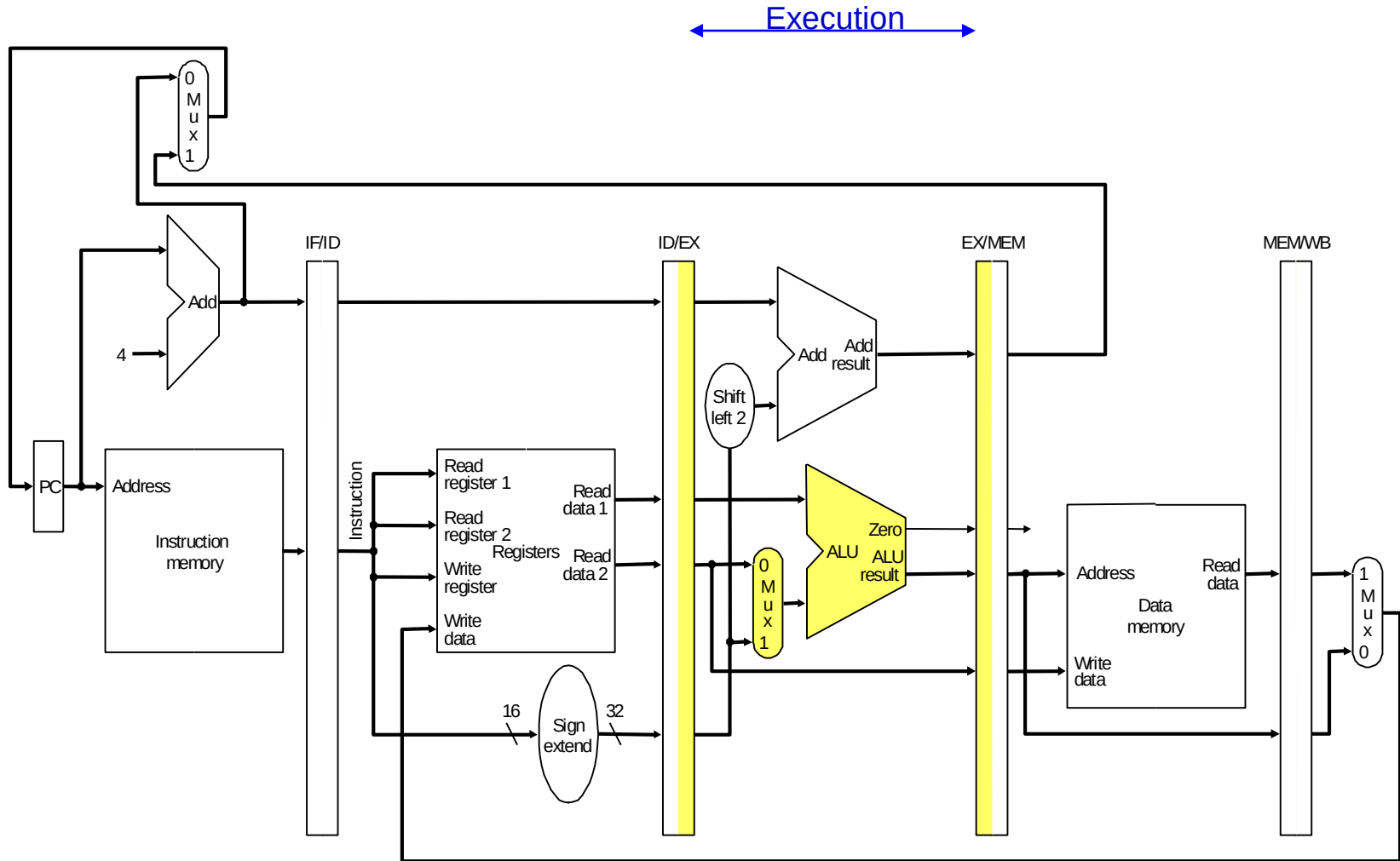
Example w/ a lw instruction: Instruction Fetch (IF)



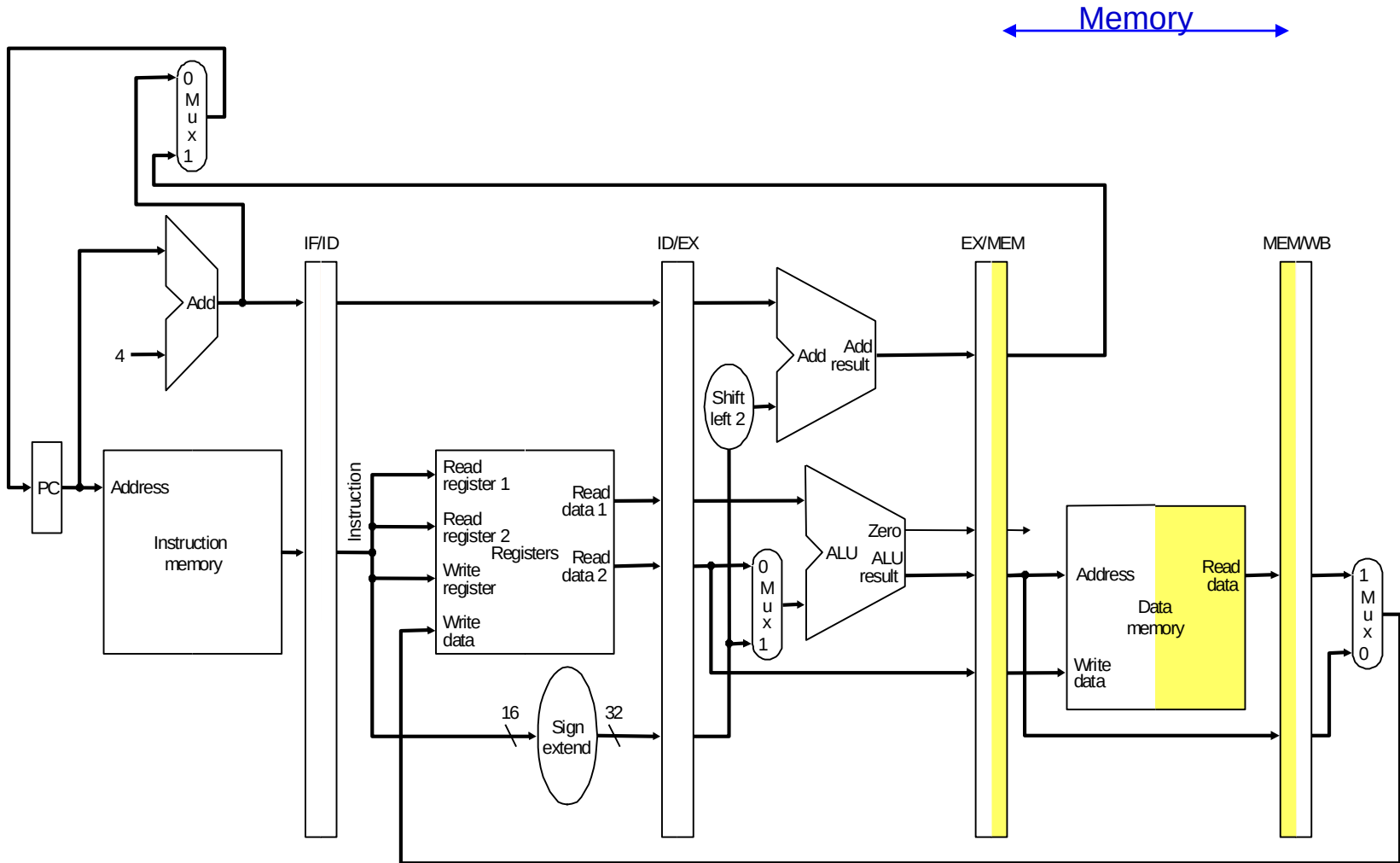
Example w/ a lw instruction: Instruction Decode (ID)



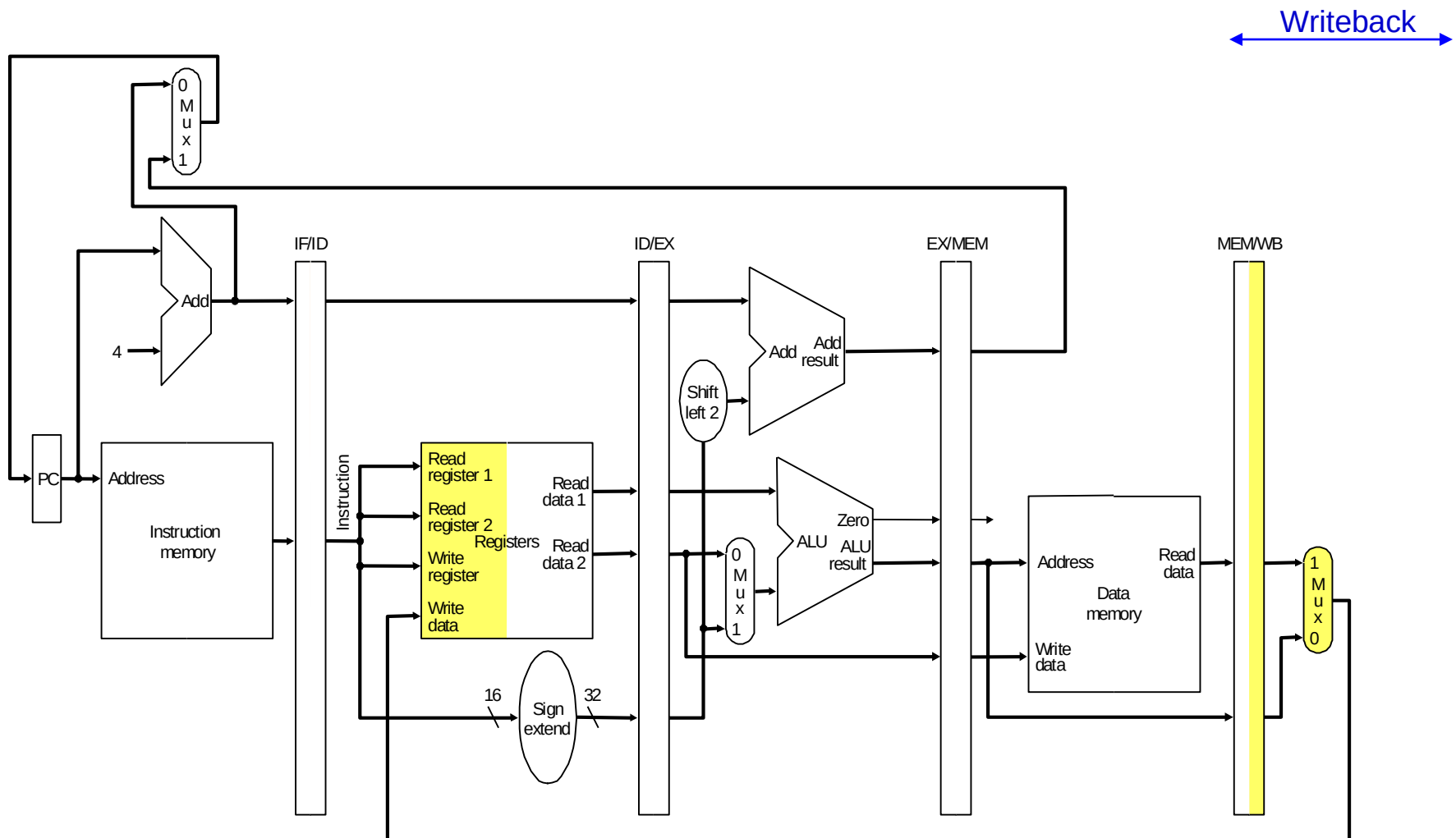
Example w/ a lw instruction: Execution (EX)



Example w/ a lw instruction: Memory (MEM)



Example w/ a lw instruction: Writeback (WB)



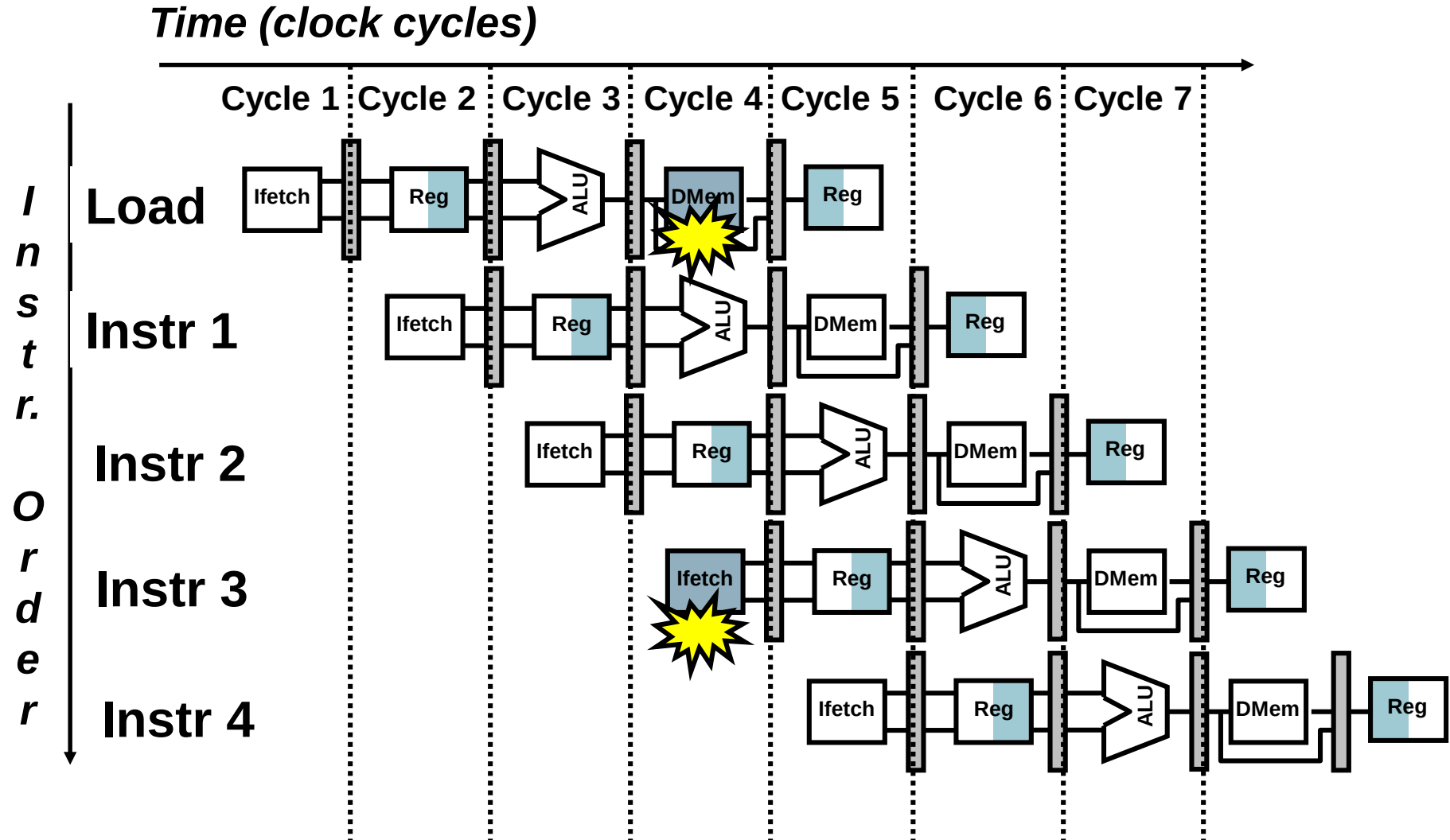
Pipeline Hazards

- Οι **κίνδυνοι (hazards)** αποτρέπουν την επόμενη εντολή από το να εκτελεστεί στον κύκλο που πρέπει
- **Δομικοί κίνδυνοι (structural)**: όταν το υλικό δεν μπορεί να υποστηρίξει ταυτόχρονη εκτέλεση συγκεκριμένων εντολών
- **Κίνδυνοι δεδομένων (data)**: όταν μια εντολή χρειάζεται το αποτέλεσμα μιας προηγούμενης, η οποία βρίσκεται ακόμη στο pipeline
- **Κίνδυνοι ελέγχου (control)**: όταν εισάγεται καθυστέρηση μεταξύ του φορτώματος εντολών και της λήψης αποφάσεων σχετικά με την αλλαγή της ροής του προγράμματος (branches, jumps)

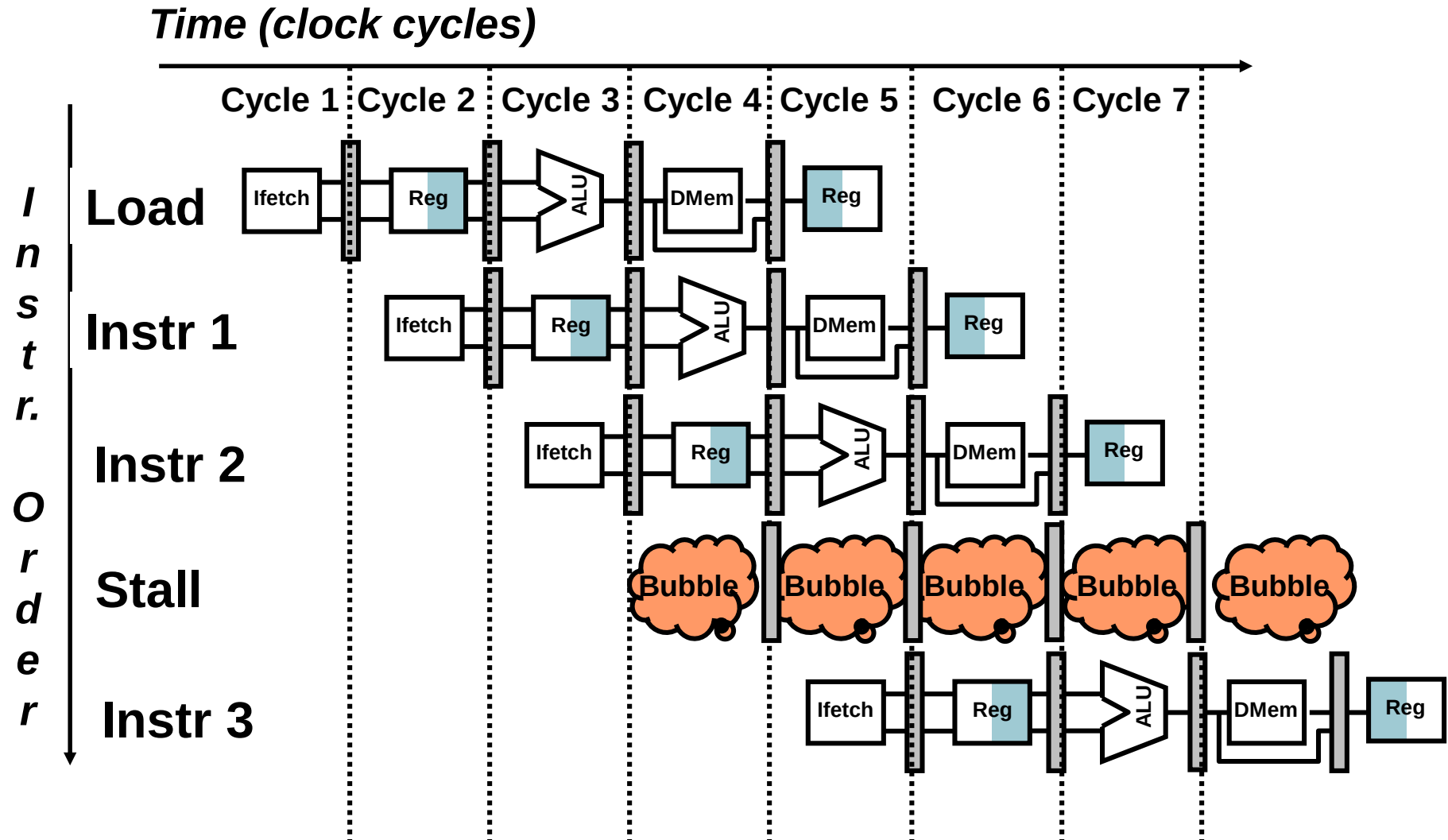
Pipeline Hazards

- **hazards** prevent the next instruction to execute on the cycle it could otherwise
- Structural hazards: The HW implementation cannot support the concurrent execution of certain instruction combination
- Data hazards: when an instruction needs the results of a previous instruction that is still in the pipeline
- Control hazards: when the execution and/or operation of an instruction is conditional on a previous control flow instruction (i.e. branch, jump)

Structural hazard: only one memory port

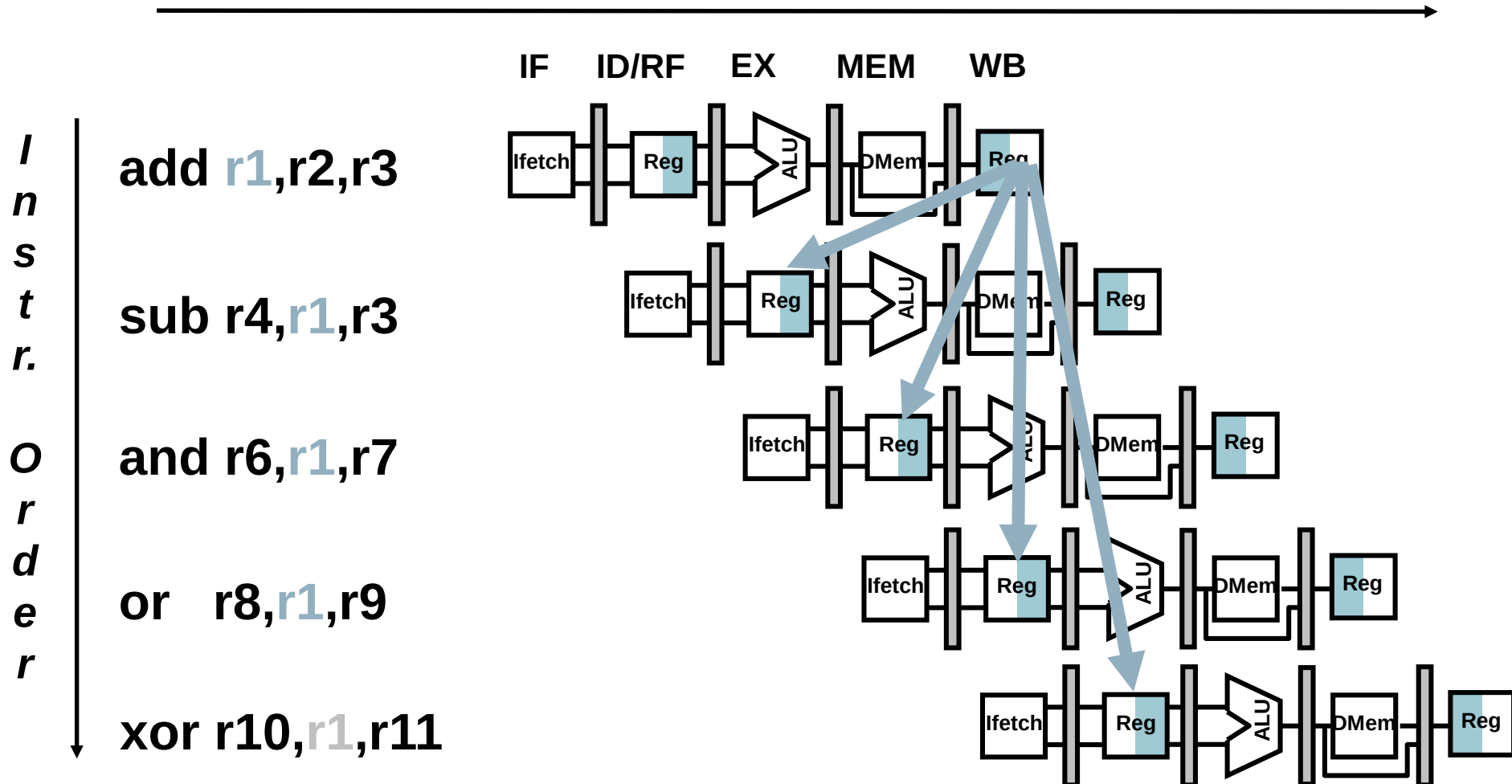


Structural hazard: solving w/ stall



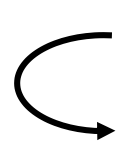
Example: Data hazard on r1

Time (clock cycles)



Data Dependencies and Hazards

- Η J είναι **data dependent** από την I:
Η J προσπαθεί να διαβάσει τον source operand πριν τον γράψει η I

 I: add **r1**,r2,r3
J: sub r4,**r1**,r3

- ή, η J είναι data dependent από την K, η οποία είναι data dependent από την I (αλυσίδα εξαρτήσεων)
- **Πραγματικές εξαρτήσεις (True Dependencies)**
- Προκαλούν **Read After Write (RAW) hazards** στο pipeline

Data Dependencies and Hazards

- J is **data dependent** on I:
hazard occurs when J tries to read its source operand before I writes its result

 I: add **r1**,r2,r3
 J: sub r4,**r1**,r3



- or, J is data dependent from K, which is data dependent on I (dependence chain)
- **True Dependencies**
- Cause **Read After Write (RAW) hazards** in the pipeline

Data Dependencies and Hazards

- Οι εξαρτήσεις είναι ιδιότητα των **προγραμμάτων**
- Η παρουσία μιας εξάρτησης υποδηλώνει την **πιθανότητα** εμφάνισης hazard, αλλά το αν θα συμβεί πραγματικά το hazard, και το πόση καθυστέρηση θα εισάγει, είναι ιδιότητα του **pipeline**
- Η σημασία των εξαρτήσεων δεδομένων:
 - 1) υποδηλώνουν την πιθανότητα για hazards
 - 2) καθορίζουν τη σειρά σύμφωνα με την οποία πρέπει να υπολογιστούν τα δεδομένα
 - 3) θέτουν ένα άνω όριο στο ποσό του παραλληλισμού που μπορούμε να εκμεταλλευτούμε

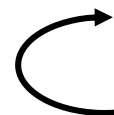
Data Dependencies and Hazards

- Dependencies are a of **program** property
- The presence of dependence indicate the **possibility** of a hazard, but whether a hazard appears, and how long it will last, is a **pipeline** property
- Data dependence significance:
 - 1) They indicate the possibility of hazards
 - 2) Determine the sequence under which data values are computed
 - 3) Impose an upper limit to the amount off parallelism we can exploit

Name Dependences (1): Anti-dependences

Name dependence, όταν 2 εντολές χρησιμοποιούν τον ίδιο καταχωρητή ή θέση μνήμης ("**name**"), χωρίς όμως να υπάρχει πραγματική ροή δεδομένων μεταξύ τους

1. Anti-dependence: η J γράφει τον r1 πριν τον διαβάσει η I

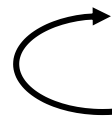
 I: sub r4,**r1**,r3
J: add **r1**,r2,r3
K: mul r6,r1,r7

- Προκαλεί **Write After Read (WAR) data hazards** στο pipeline
- Δε μπορεί να συμβεί στο κλασικό 5-stage pipeline διότι:
 - όλες οι εντολές χρειάζονται 5 κύκλους για να εκτελεστούν, και
 - οι αναγνώσεις συμβαίνουν πάντα στο στάδιο 2, και
 - οι εγγραφές στο στάδιο 5

Name Dependences (1): Anti-dependences

Name dependence: when 2 instructions use the same register or memory location ("**name**"), but without true information flow between the two

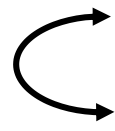
1. Anti-dependence: J writes to r1 before I read it

 I: sub r4,**r1**,r3
J: add **r1**,r2,r3
K: mul r6,r1,r7

- Causes a **Write After Read (WAR) data hazards** in the pipeline
- Cannot happen in the classic 5-stage pipeline because:
 - All instructions take 5 cycles to execute, and
 - All reads happen at the 2nd stage, and
 - All write happen at the 5th stage

Name Dependences (2): Output dependences

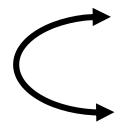
2. Output dependence: η J γράφει τον r1 πριν τον γράψει η I

 I: sub r1, r4, r3
J: add r1, r2, r3
K: mul r6, r1, r7

- Προκαλεί Write After Write (WAW) data hazards στο pipeline
- Δε μπορεί να συμβεί στο κλασικό 5-stage pipeline διότι:
 - όλες οι εντολές χρειάζονται 5 κύκλους για να εκτελεστούν, και
 - οι εγγραφές συμβαίνουν πάντα στο στάδιο 5
- Τόσο οι κίνδυνοι WAW όσο και οι WAR συναντώνται σε πιο περίπλοκα pipelines (π.χ. multi-cycle pipeline, out-of-order execution)

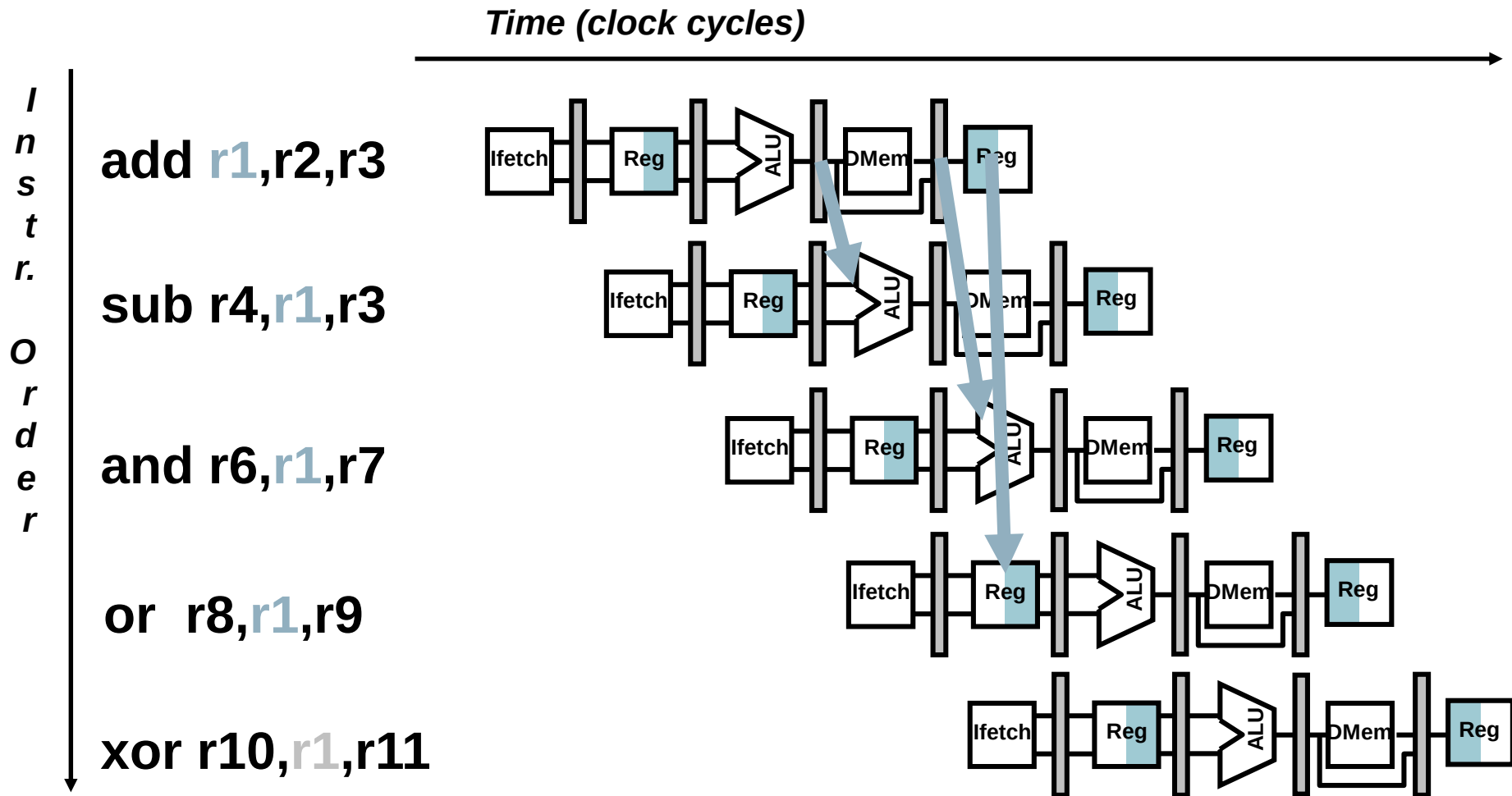
Name Dependences (2): Output dependences

2. Output dependence: J writes r1 before I writes it

 I: sub **r1**,r4,r3
J: add **r1**,r2,r3
K: mul r6,r1,r7

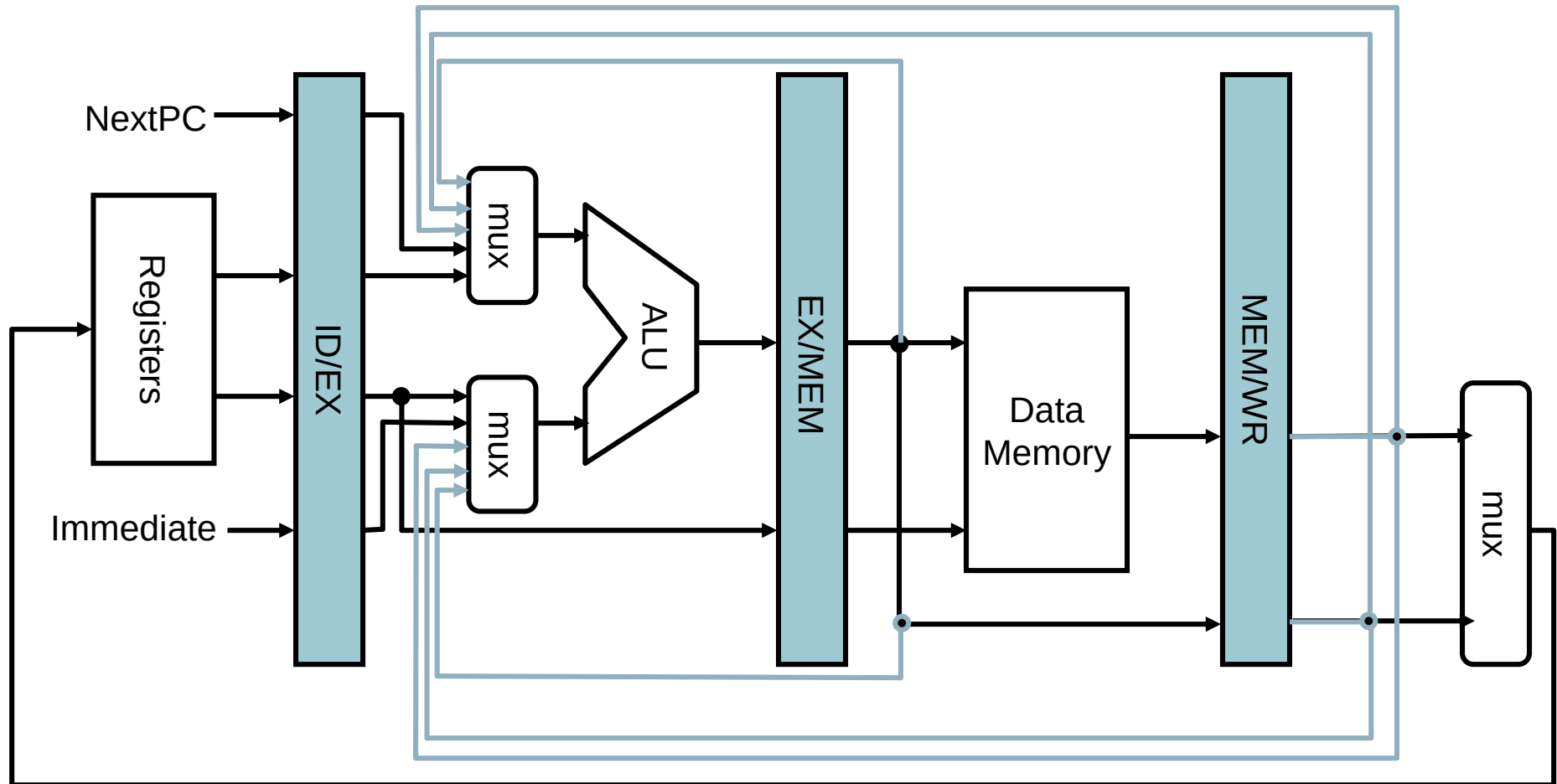
- Cause **Write After Write (WAW) data hazards** in the pipeline
- Cannot happen in the classic 5-stage pipeline because:
 - All reads happen at the 2nd stage, and
 - All write happen at the 5th stage
- Both WAW and WAR hazards appear in more complex pipeline structures (e.g. multi-cycle pipeline, out-of-order execution)

Forwarding



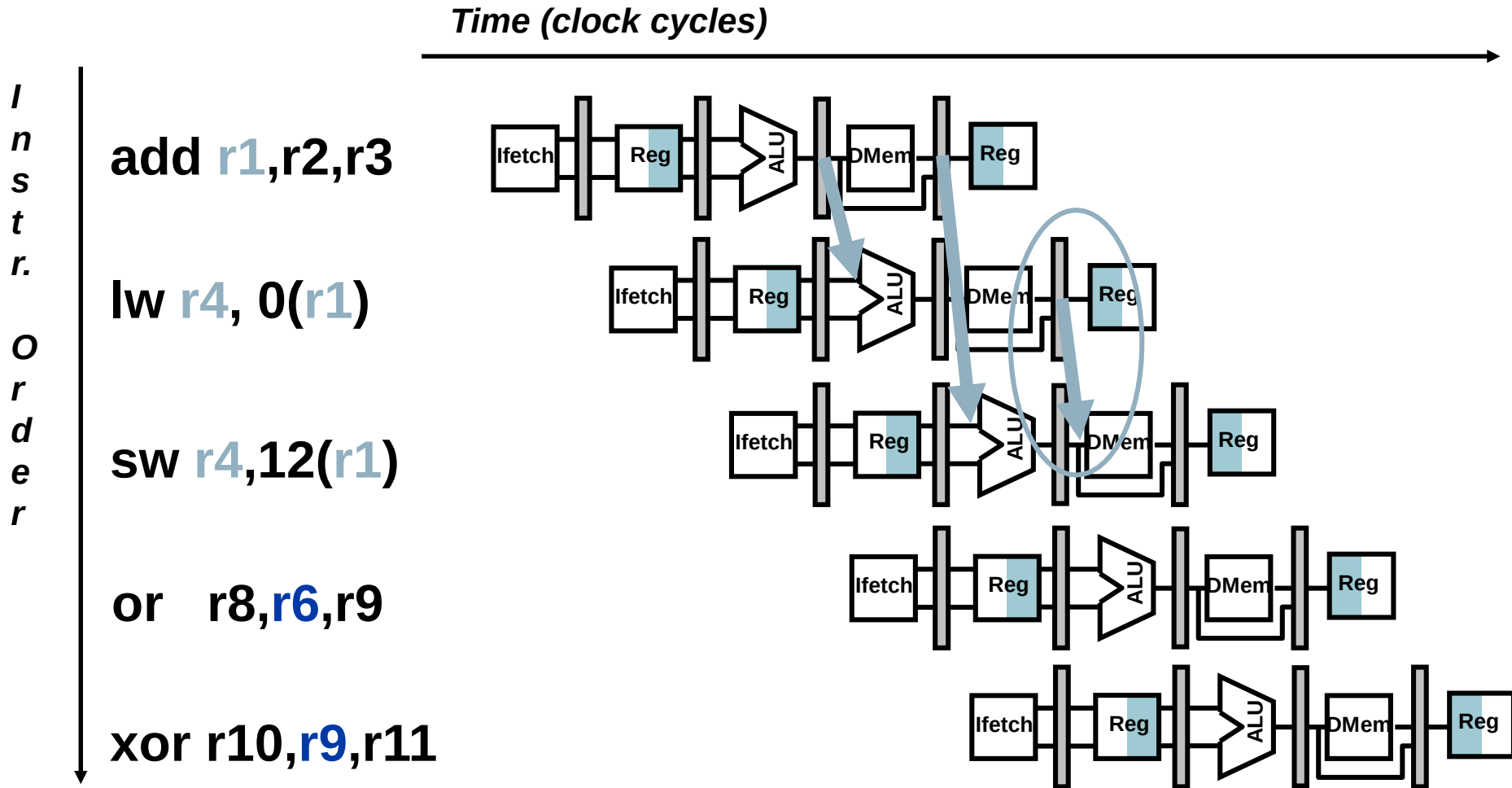
- μειώνονται τα RAW hazards

Hardware changes to support forwarding

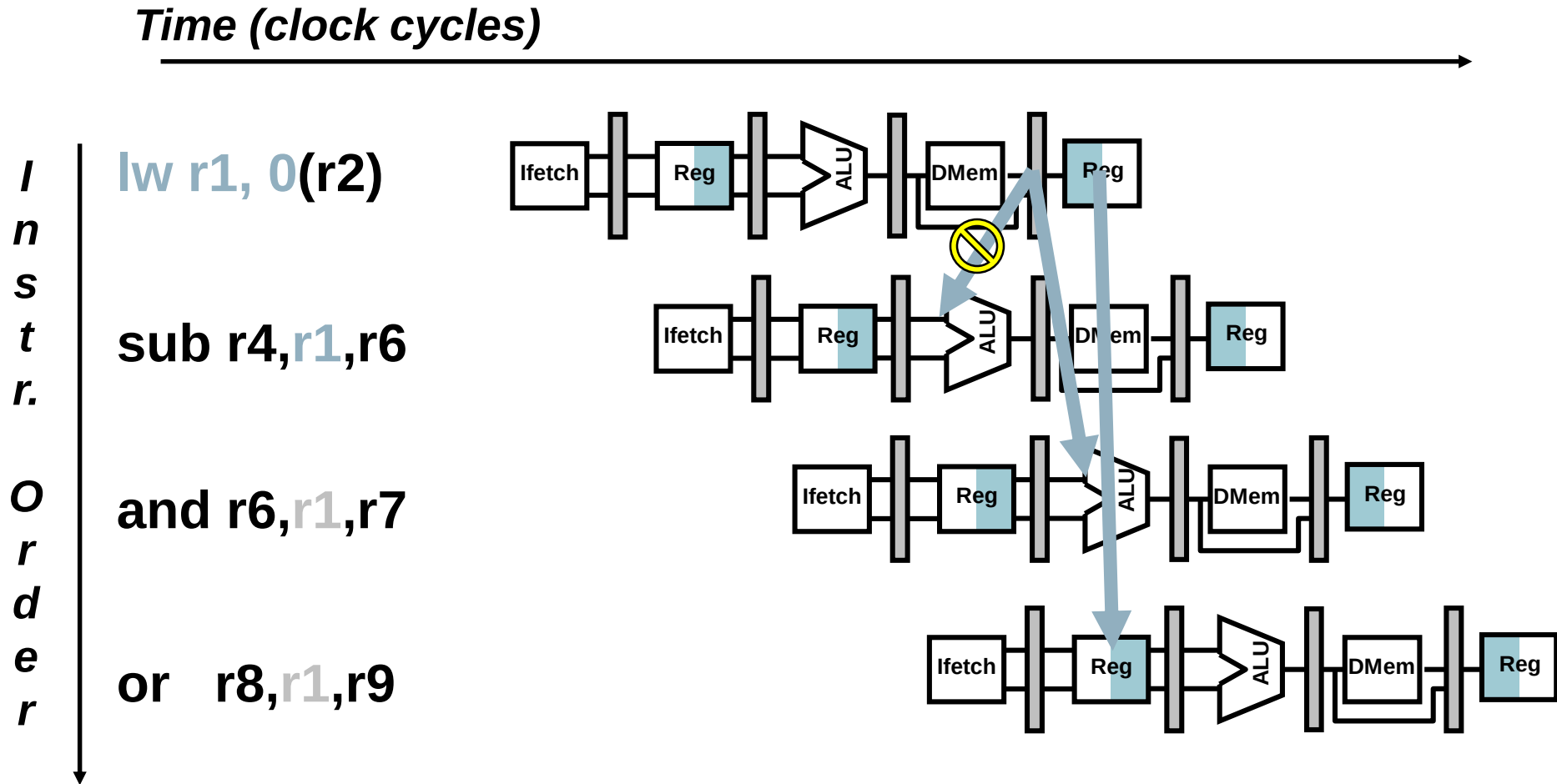


EX->EX, MEM->EX forwarding

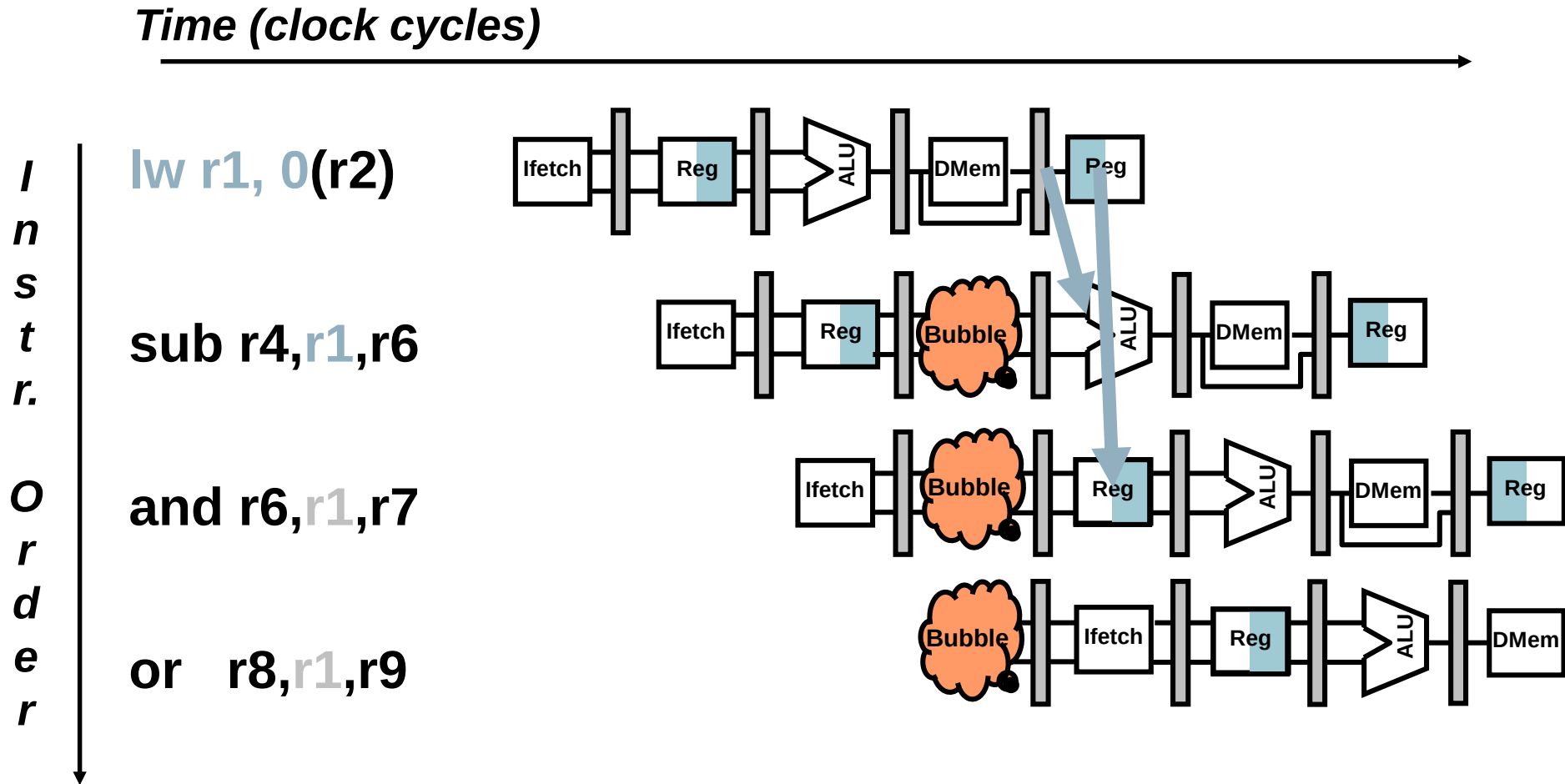
Forwarding MEM->MEM



Forwarding does not always work!



Forwarding does not always work!



Re-arranging instructions to avoid RAW hazards

How can we produce efficient assembly code for the following operations?

$a = b + c;$

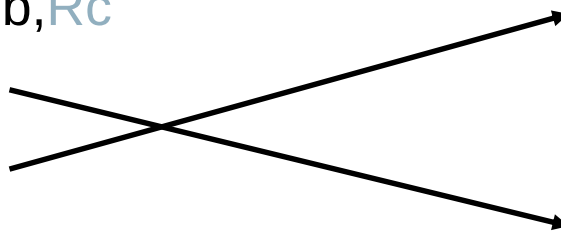
$d = e - f;$

Slow code:

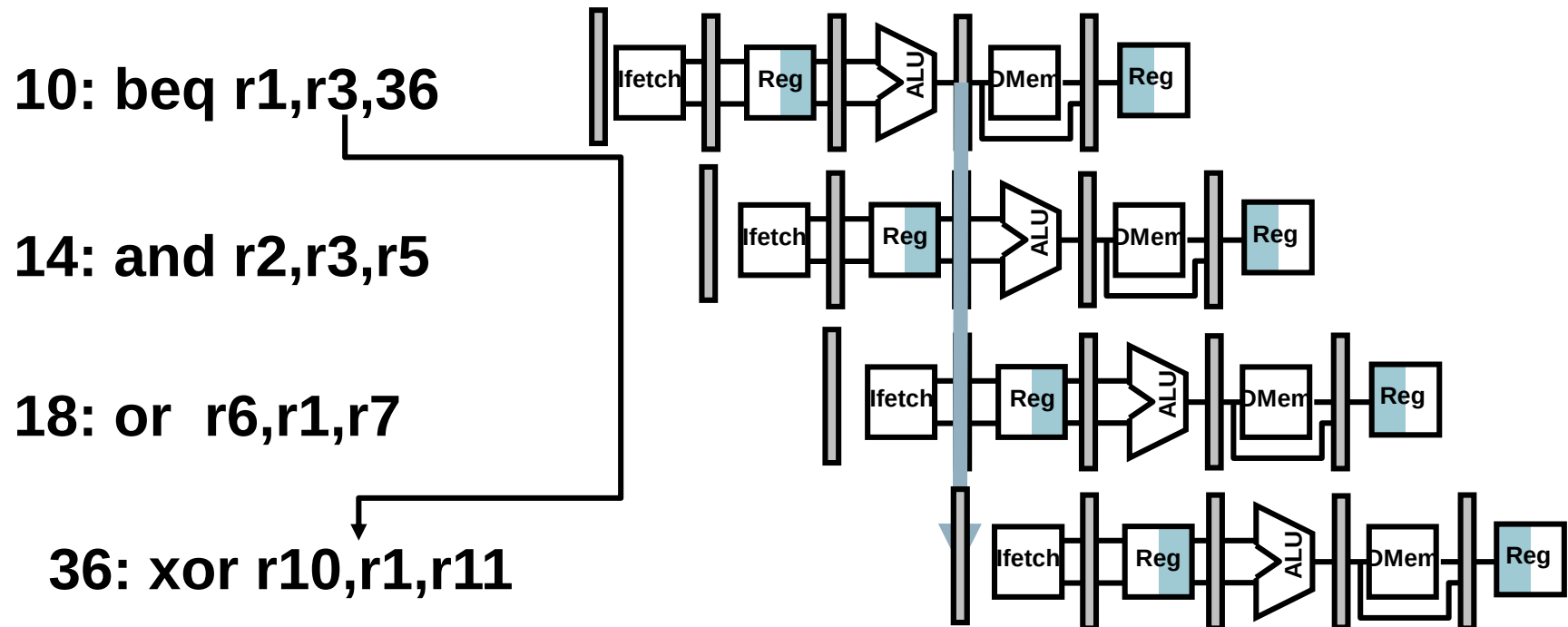
```
LW    Rb,b
LW    Rc,c
ADD   Ra,Rb,Rc
SW    a,Ra
LW    Re,e
LW    Rf,f
SUB   Rd,Re,Rf
SW    d,Rd
```

Fast code:

```
LW    Rb,b
LW    Rc,c
LW    Re,e
ADD   Ra,Rb,Rc
LW    Rf,f
SW    a,Ra
SUB   Rd,Re,Rf
SW    d,Rd
```



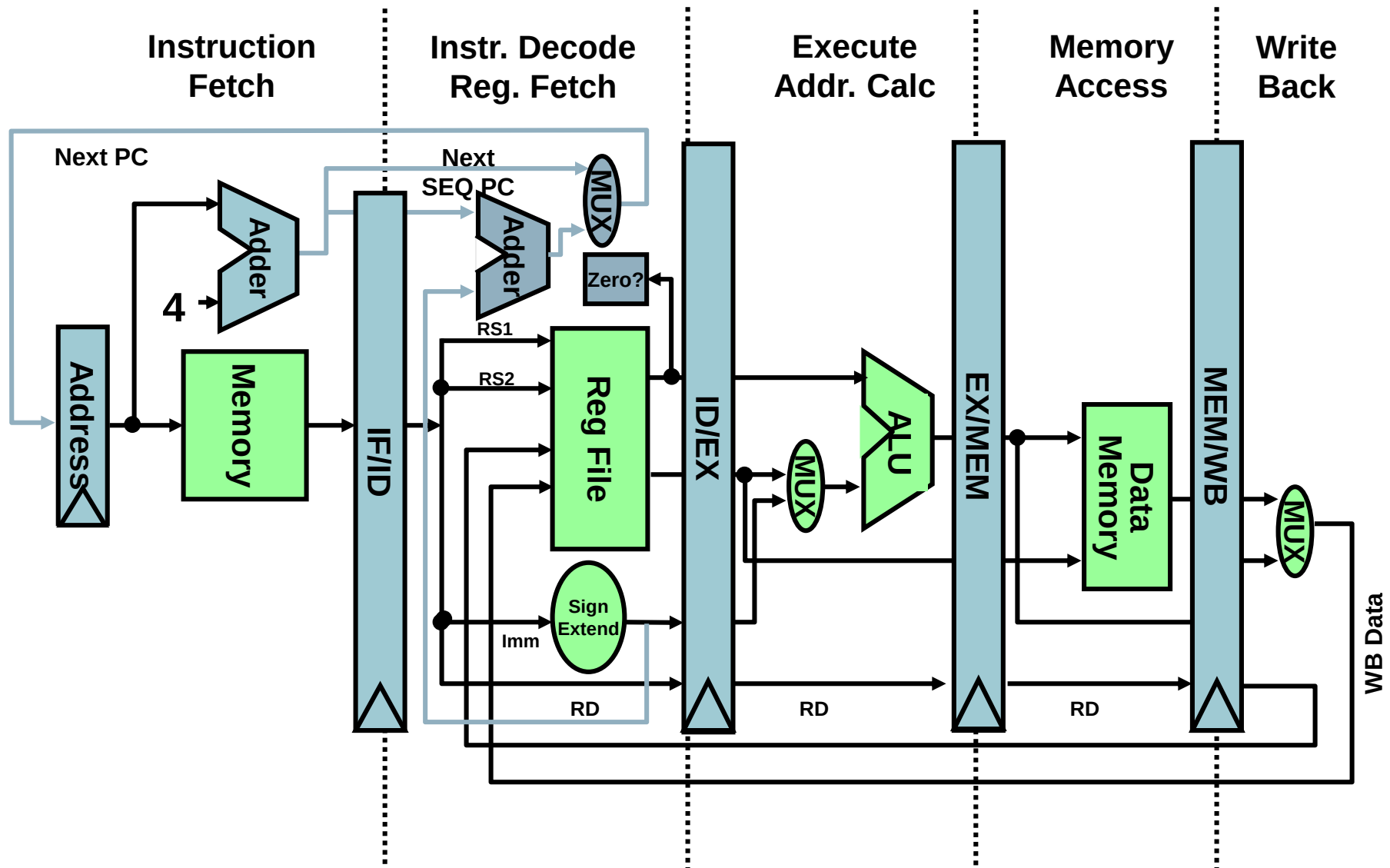
Control hazards on branches: 2 stage stalls



Branch stalls impact

- If $CPI = 1$, branches frequency is 30%, and branch stalls last 2 cycles $\Rightarrow$ overall $CPI = 1.6$!
- One solution: determine the branch earlier, AND compute the branch target address earlier
 - Move register equality check to the ID/RF stage
 - Add an adder at the ID/RF to compute the target PC address
 - 1 cycle branch penalty instead of 2

Pipeline changes



Dealing w/ control hazards: 4 alternatives

#1: Πάγωμα του pipeline μέχρι ο στόχος του branch να γίνει γνωστός

#2: Πρόβλεψη “Not Taken” για κάθε branch

- συνεχίζουμε να φορτώνουμε τις επόμενες εντολές, σα να ήταν η εντολή branch μια «κανονική» εντολή
- απορρίπτουμε από το pipeline αυτές τις εντολές, αν τελικά το branch είναι “Taken”
- το $PC+4$ είναι ήδη υπολογισμένο, το χρησιμοποιούμε για να πάρουμε την επόμενη εντολή

#3: Πρόβλεψη “Taken” για κάθε branch

- φορτώνουμε εντολές αρχίζοντας από τη διεύθυνση-στόχο του branch
- Στον MIPS η διεύθυνση-στόχος δε γίνεται γνωστή πριν το αποτέλεσμα του branch
 - κανένα επιπλέον πλεονέκτημα στον MIPS (1 cycle branch penalty)
 - θα είχε νόημα σε άλλα pipelines, όπου η διεύθυνση-στόχος γίνεται γνωστή πριν το αποτέλεσμα του branch

Dealing w/ control hazards: 4 alternatives

#1: stall the pipeline until the result is available

#2: predict “Not Taken” for all branches

- Continue to fetch subsequent instructions as if the branch was a regular instruction
- Drop these instructions if the branch is “Taken”
- PC+4 is already computed for every instruction, use it to fetch the next instruction

#3: Predict “Taken” for all branches

- Start fetching instructions from the branch target
- In MIPS the target address is not known until the branch is resolved
 - No benefit in the 5-stage MIPS pipeline (1 cycle branch penalty)
 - Can be useful in other pipelines, where the target address calculation is possible before the branch condition evaluation

Dealing w/ control hazards: 4 alternatives

#4: Delayed Branches

branch instruction

sequential successor₁

sequential successor₂

.....

sequential successor_n

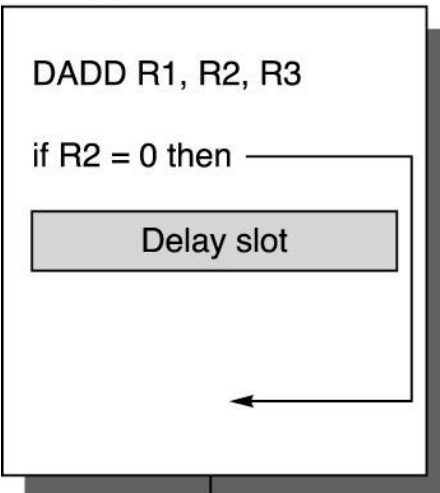
branch target if taken

Branch delay of length n:
Instructions are executed
whether the branch is taken
or not

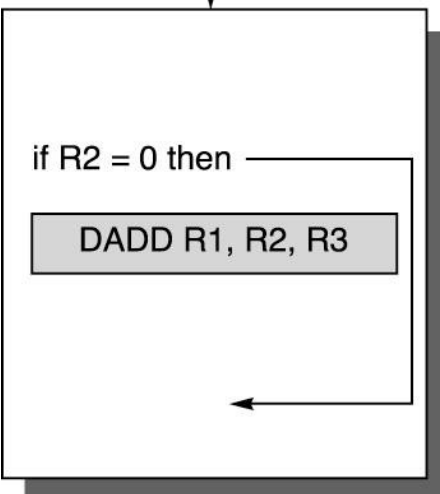
- One delay slot: allows the determination of the condition and the target address calculation in a 5-stage pipeline without stalls

“scheduling” a branch delay slot

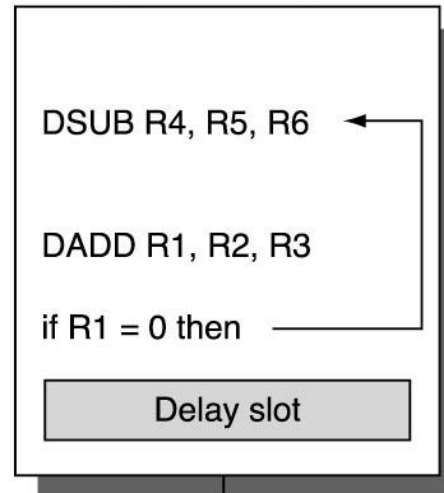
(a) From before



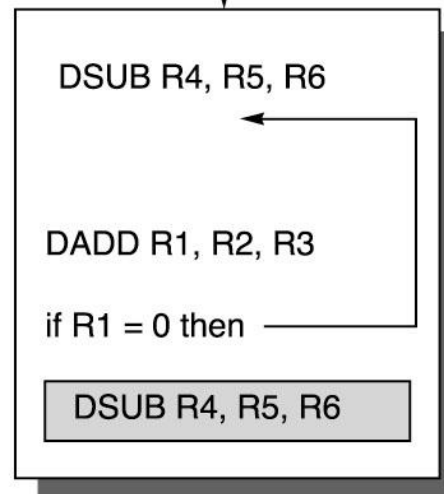
becomes



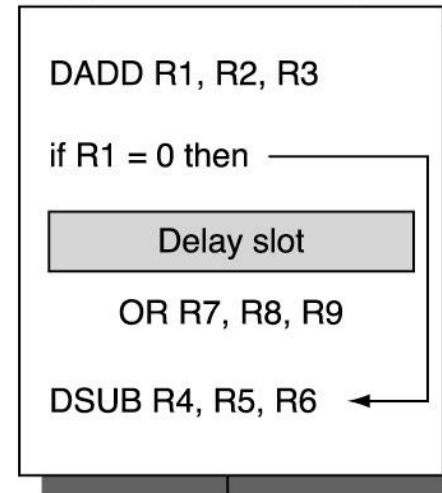
(b) From target



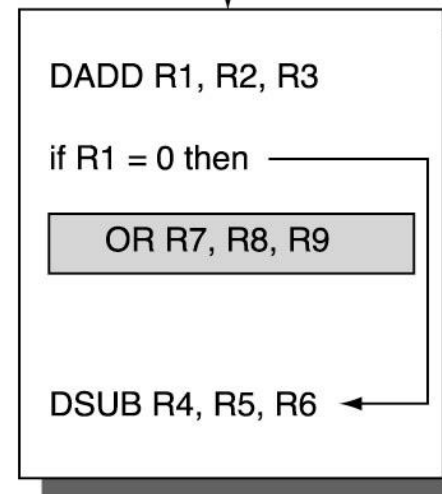
becomes



(c) From fall-through



becomes



Best choice?

(a) ✓

(b) ~

(c) OK

Why?

“scheduling” a branch delay slot

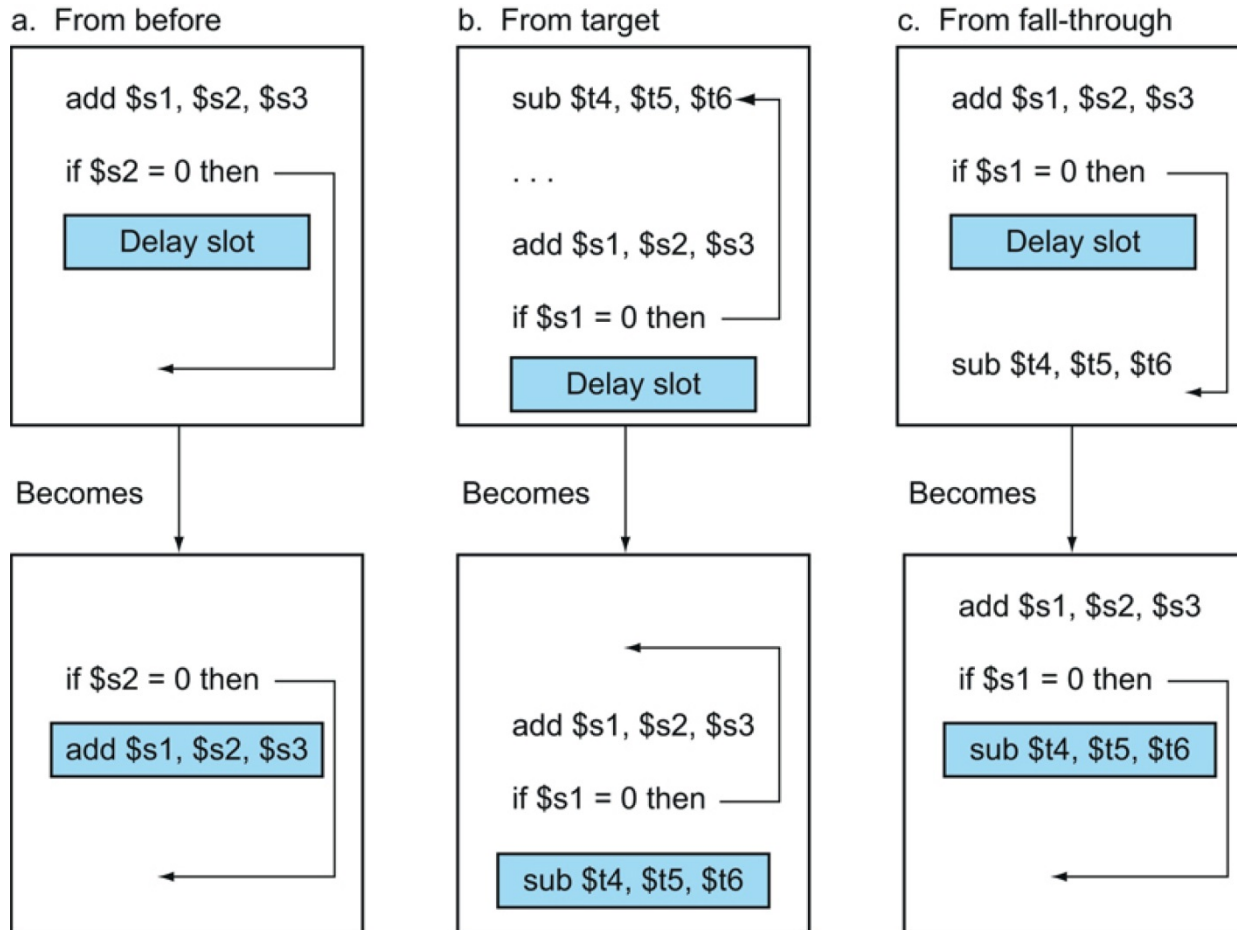


FIGURE 4.64 Scheduling the branch delay slot. The top box in each pair shows the code before scheduling; the bottom box shows the scheduled code. In (a), the delay slot is scheduled with an independent instruction from before the branch. This is the best choice. Strategies (b) and (c) are used when (a) is not possible. In the code sequences for (b) and (c), the use of \$s1 in the branch condition prevents the add instruction (whose destination is \$s1) from being moved into the branch delay slot. In (b) the branch delay slot is scheduled from the target of the branch; usually the target instruction will need to be copied because it can be reached by another path. Strategy (b) is preferred when the branch is taken with high probability, such as a loop branch. Finally, the branch may be scheduled from the not-taken fall-through as in (c). To make this optimization legal for (b) or (c), it must be OK to execute the sub instruction when the branch goes in the unexpected direction. By “OK” we mean that the work is wasted, but the program will still execute correctly. This is the case, for example, if \$t4 were an unused temporary register when the branch goes in the unexpected direction.

Copyright © 2021 Elsevier Inc. All rights reserved.

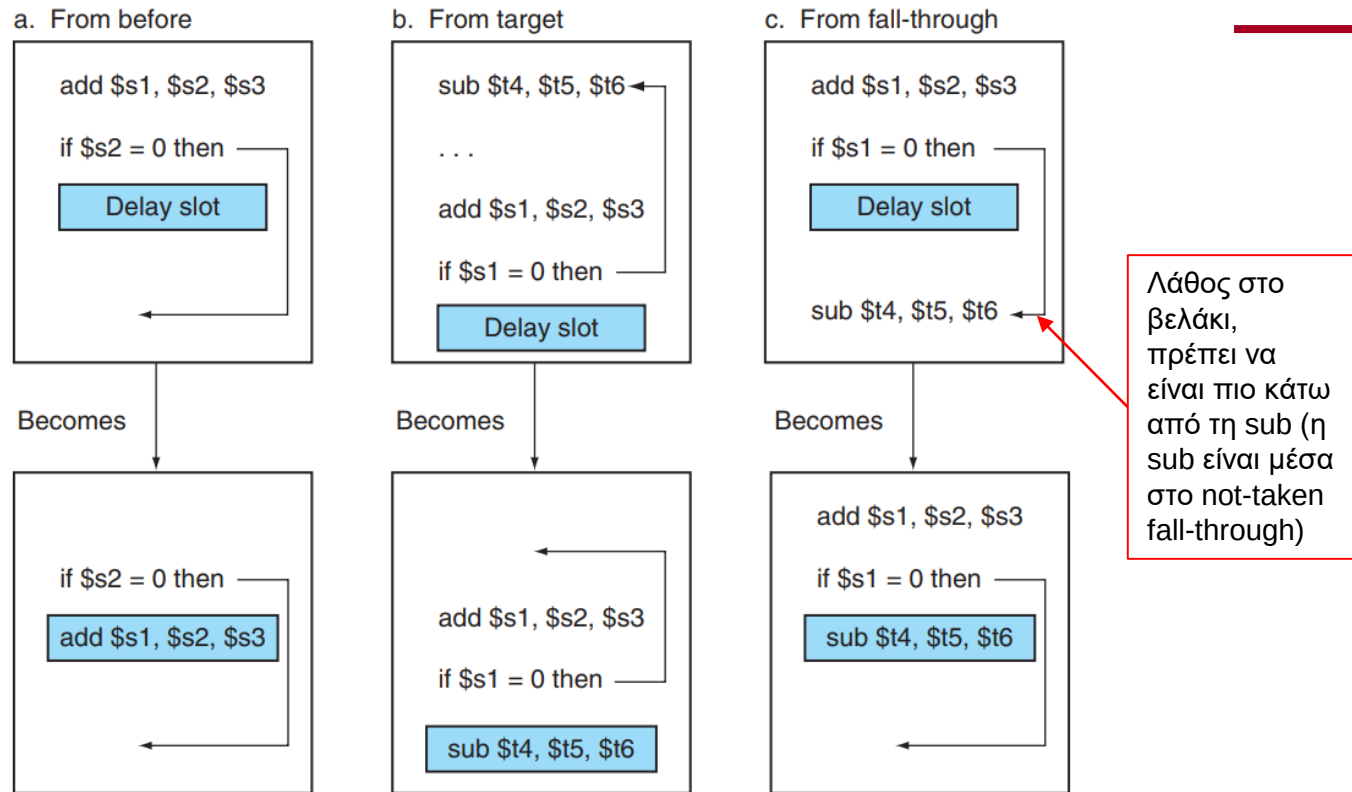


FIGURE 4.64 Scheduling the branch delay slot. The top box in each pair shows the code before scheduling; the bottom box shows the scheduled code. In (a), the delay slot is scheduled with an independent instruction from before the branch. This is the best choice. Strategies (b) and (c) are used when (a) is not possible. In the code sequences for (b) and (c), the use of \$s1 in the branch condition prevents the add instruction (whose destination is \$s1) from being moved into the branch delay slot. In (b) the branch delay slot is scheduled from the target of the branch; usually the target instruction will need to be copied because it can be reached by another path. Strategy (b) is preferred when the branch is taken with high probability, such as a loop branch. Finally, the branch may be scheduled from the not-taken fall-through as in (c). To make this optimization legal for (b) or (c), it must be OK to execute the sub instruction when the branch goes in the unexpected direction. By “OK” we mean that the work is wasted, but the program will still execute correctly. This is the case, for example, if \$t4 were an unused temporary register when the branch goes in the unexpected direction. Copyright © 2009 Elsevier, Inc. All rights reserved.

Scalar architecture limitations

- Max throughput: 1 instruction/clock cycle ($IPC \leq 1$)
- All types of instructions are forced in the common pipeline
- Delays due to stall of a particular instruction affect all subsequent instructions that come in-order as specified in the program

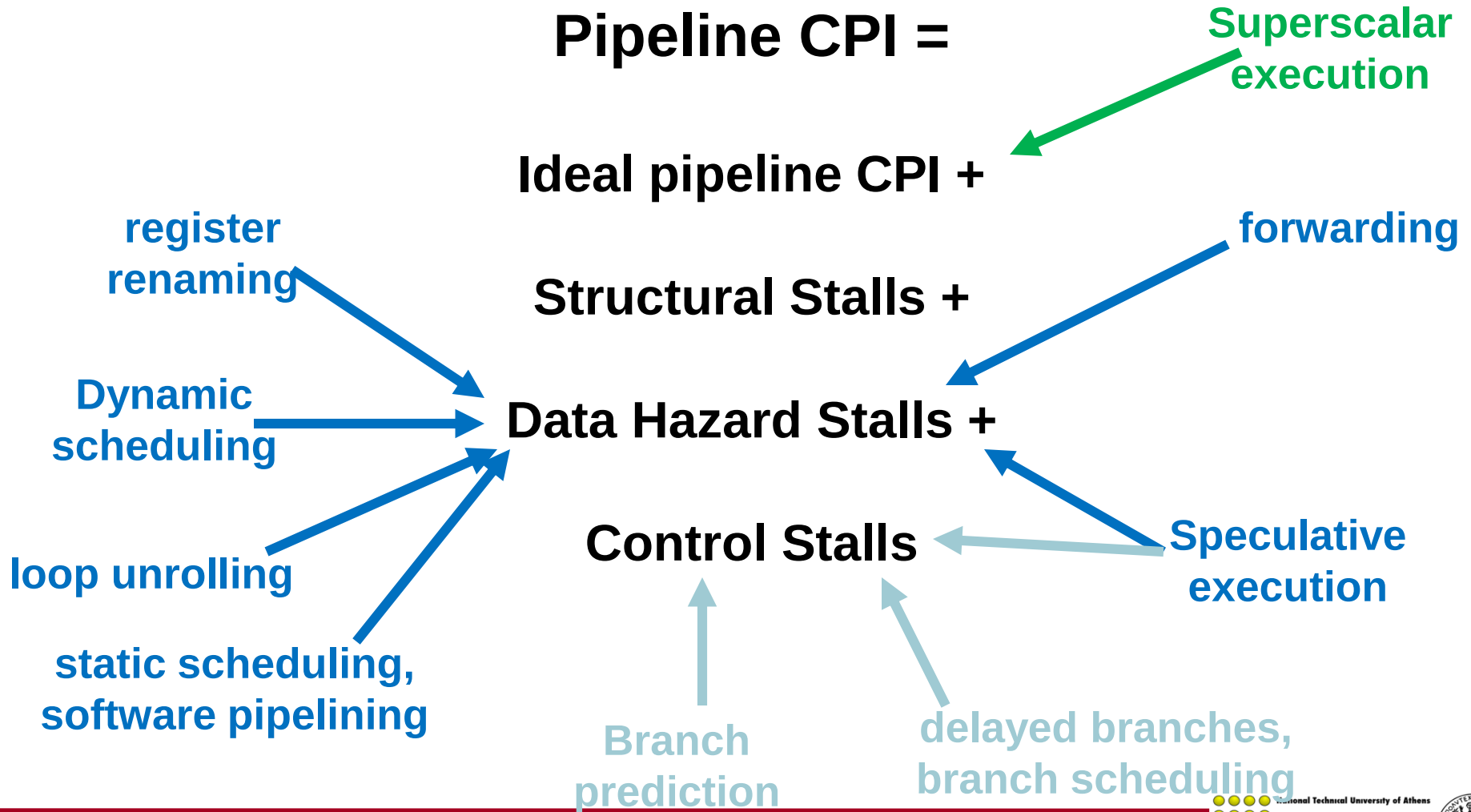
Can I overcome these limitations?

- Execute multiple instructions per clock cycle(parallel execution)
 - **superscalar architectures**
- Incorporate different datapaths and data flows each with the same of different types of functional units
 - deal w/ **multicycle operations**
- out-of-order instruction execution is possible!
 - **dynamic architectures**

$$\begin{aligned} \text{Pipeline CPI} = & \\ & \text{Ideal pipeline CPI} + \\ & \text{Structural Stalls} + \\ & \text{Data Hazard Stalls} + \\ & \text{Control Stalls} \end{aligned}$$

Measure of the performance we can obtain with a particular pipeline

Alternatively...

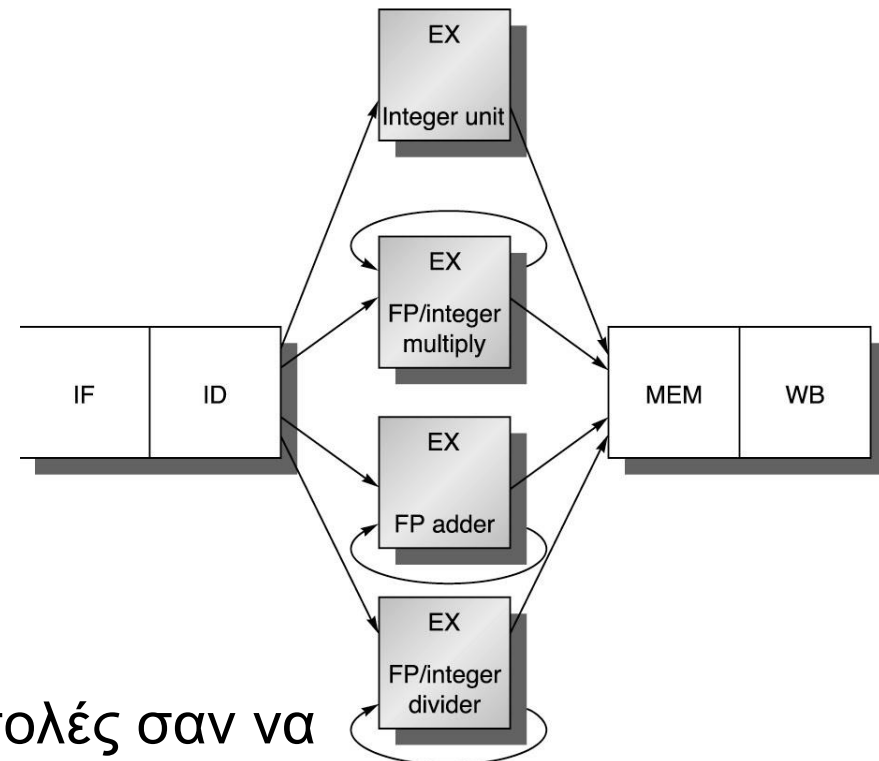


Multi-cycle instructions

- Στο κλασικό 5-stage pipeline όλες οι λειτουργίες χρειάζονται 1 κύκλο
- Το να έχουμε όλες τις εντολές να τελειώνουν σε έναν κύκλο σημαίνει:
 - μείωση της συχνότητας για να προσαρμοστεί το pipeline στη διάρκεια της πιο χρονοβόρας λειτουργίας, ή
 - χρησιμοποίηση εξαιρετικά πολύπλοκων κυκλωμάτων για την υλοποίηση της πιο χρονοβόρας λειτουργίας σε 1 κύκλο
- Ρεαλιστική αντιμετώπιση:
 - επέκταση του pipeline για να υποστηρίζει λειτουργίες διαφορετικής διάρκειας
- Παραδείγματα από πραγματικούς επεξεργαστές:
 - FP add, Int/FP mult: ~2-6 κύκλους
 - Int/FP div, sqrt: ~20-50 κύκλους
 - Προσπέλαση στη μνήμη: ~2-200 κύκλους...

Multi-cycle execution: επέκταση του 5-stage pipeline με non-pipelined FP units

- 4 διαφορετικές ALUs:
 - Integer
 - FP/Integer multiply
 - FP adder
 - FP/Integer divide



- φανταζόμαστε το EX για τις FP εντολές σαν να επαναλαμβάνεται για πολλούς συνεχόμενους κύκλους
- κάθε τέτοια μονάδα είναι non-pipelined: δε μπορεί να σταλεί ("issue") προς εκτέλεση σε μια μονάδα μια εντολή, αν κάποια προηγούμενη χρησιμοποιεί ακόμα τη μονάδα αυτή (structural hazard)
- η εντολή που stall-άρει καθυστερεί και όλες τις επόμενες

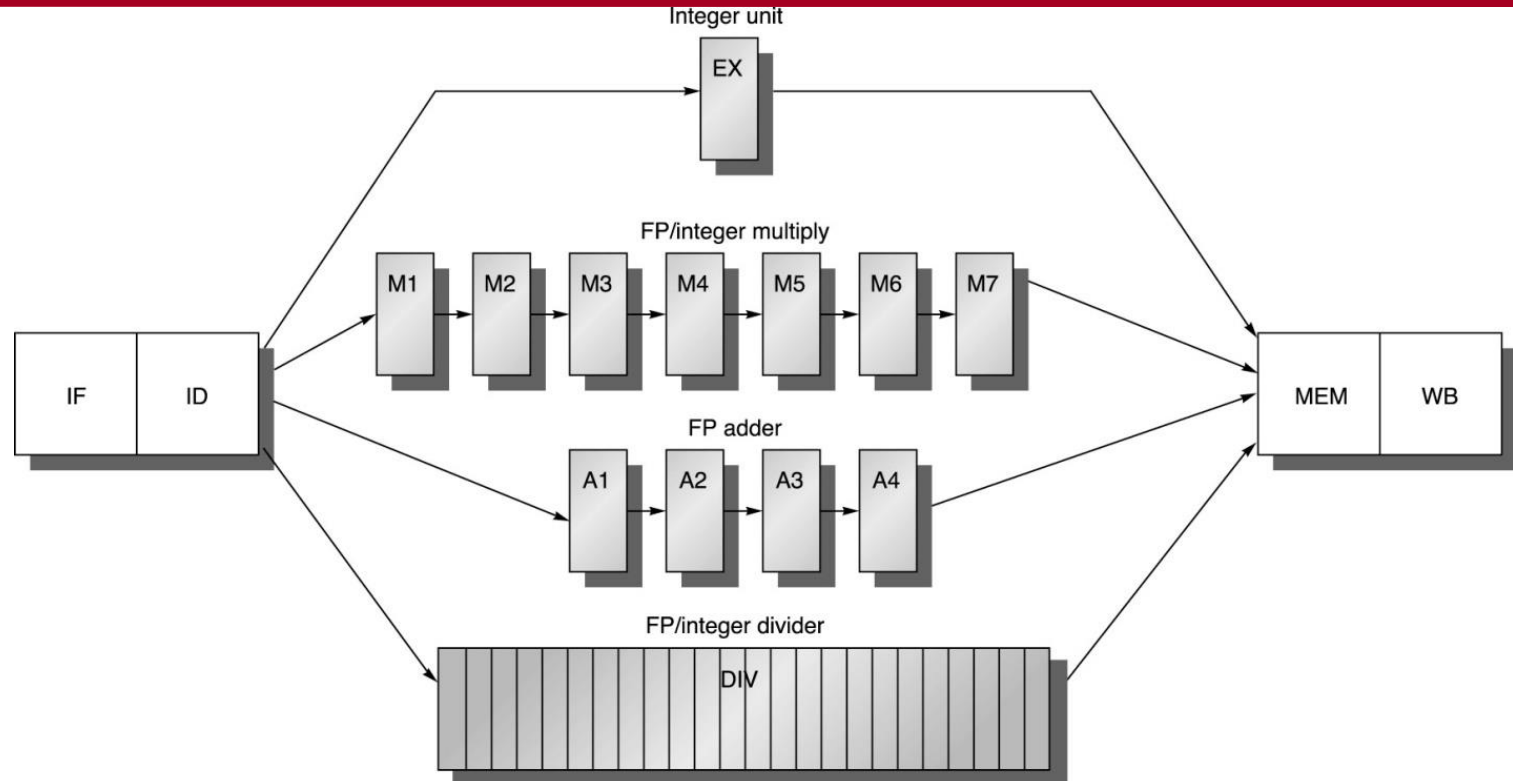
Μετρικές περιγραφής ενός multi-cycle pipeline

- **Initiation interval**: #κύκλων που μεσολαβούν ανάμεσα στην αποστολή στις μονάδες εκτέλεσης δύο εντολών ίδιου τύπου (“*structural hazard*”)
 - ενδεικτικό του throughput μιας μονάδας εκτέλεσης
- **Latency**: #κύκλων που μεσολαβούν μεταξύ της εντολής που παράγει ένα αποτέλεσμα και της εντολής που θα το καταναλώσει (“*RAW hazard*”)

Unit	Latency	Initiation interval
Integer ALU	0	1
Data memory	1	1
FP add	3	1
FP multiply	6	1
FP divide	24	25

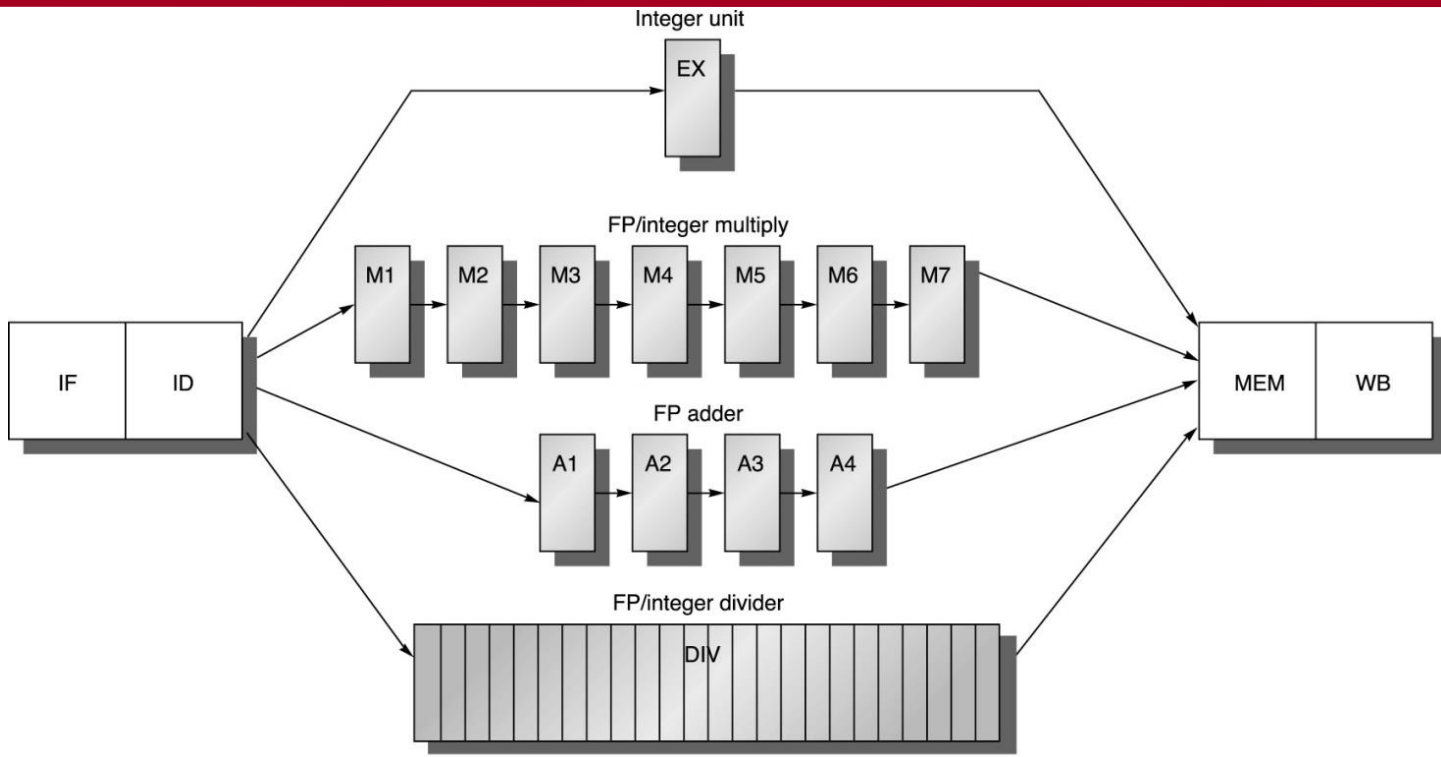
- Οι περισσότερες εντολές καταναλώνουν τους τελεστές τους στην αρχή του EX
 - Latency = #σταδίων μετά το EX όπου μια εντολή παράγει το αποτέλεσμά της

Multi-cycle execution: επέκταση του 5-stage pipeline με pipelined FP units



- επιτρέπει να βρίσκονται εν εκτελέσει μέχρι 4 FP-adds, 7 FP-muls, 1 FP-divide (non-pipelined)
- τα επιμέρους στάδια είναι ανεξάρτητα και χωρίζονται με ενδιάμεσους καταχωρητές
- διάσπαση μιας λειτουργίας σε πολλά επιμέρους στάδια:
 - ↑ συχνότητα ρολογιού
 - ↑ latency λειτουργιών + ↑ συχνότητα RAW hazards + ↑ stalls

Παράδειγμα



Clock Cycles 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18

MUL.D	IF	ID	M1	M2	M3	M4	M5	M6	M7	M	WB						
ADD.D		IF	ID	A1	A2	A3	A4	M	WB								
L.D			IF	ID	EX	M	WB										
S.D				IF	ID	EX	M	WB									

Hazards

- Νέα ζητήματα που προκύπτουν:
 - η μονάδα divide δεν υποστηρίζει pipelining → μπορεί να προκύψουν **structural hazards**
 - εντολές διαφορετικού latency → #εγγραφών στο register file σε έναν κύκλο μπορεί να είναι >1 (**structural hazards**)
 - οι εντολές μπορεί να φτάσουν στο WB εκτός σειράς προγράμματος → μπορούν να συμβούν **WAW hazards** (**WAR?**)
 - οι εντολές μπορεί να ολοκληρωθούν εκτός σειράς προγράμματος → πρόβλημα με τις εξαιρέσεις
 - μεγαλύτερο latency στις εντολές → συχνότερα τα stalls εξαιτίας **RAW hazards**

RAW hazards και αύξηση των stalls

Clock Cycles	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
L.D F4,0(R2)	IF	ID	EX	M	WB													
MUL.D F0,F4,F6		IF	ID	S	M1	M2	M3	M4	M5	M6	M7	M	WB					
ADD.D F2,F0,F8			IF	S	ID	S	S	S	S	S	S	A1	A2	A3	A4	M	WB	
S.D F2,0(R2)					IF	S	S	S	S	S	S	ID	EX	S	S	S	M	WB

- full bypassing/forwarding
- η S.D πρέπει να καθυστερήσει έναν κύκλο παραπάνω για να αποφύγουμε το conflict στο MEM της ADD.D

Structural Hazards

Clock Cycles 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

MUL.D F0,F4,F6	IF	ID	M1	M2	M3	M4	M5	M6	M7	M	WB				
...		IF	ID	EX	M	WB									
...			IF	ID	EX	M	WB								
ADD.D F2,F4,F6				IF	ID	A1	A2	A3	A4	M	WB				
...					IF	ID	EX	M	WB						
...						IF	ID	EX	M	WB					
L.D F2,0(R2)							IF	ID	EX	M	WB				

Με μόνο ένα write port στο register file → structural hazard

- multiple write ports (...όμως δεν είναι η συνήθης περίπτωση στα προγράμματα)
- interlocks:
 - ελέγχουμε στο ID το ενδεχόμενο η τρέχουσα εντολή να έχει conflict στο WB με μία προηγούμενη εντολή, και αν ισχύει αυτό την stall-άρουμε στο ID (προτού γίνει issue)
 - ελέγχουμε το ενδεχόμενο η εντολή να έχει conflict στο WB όταν αυτή πάει να μπει στο MEM ή στο WB, και αν ισχύει αυτό τη stall-άρουμε εκεί

WAW Hazards

Clock Cycles 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

...		IF	ID	EX	M	WB									
...			IF	ID	EX	M	WB								
ADD.D F2,F4,F6				IF	ID	A1	A2	A3	A4	M	WB				
inst					IF	ID	EX	M	WB						
L.D F2,0(R2)						IF	ID	EX	M	WB					

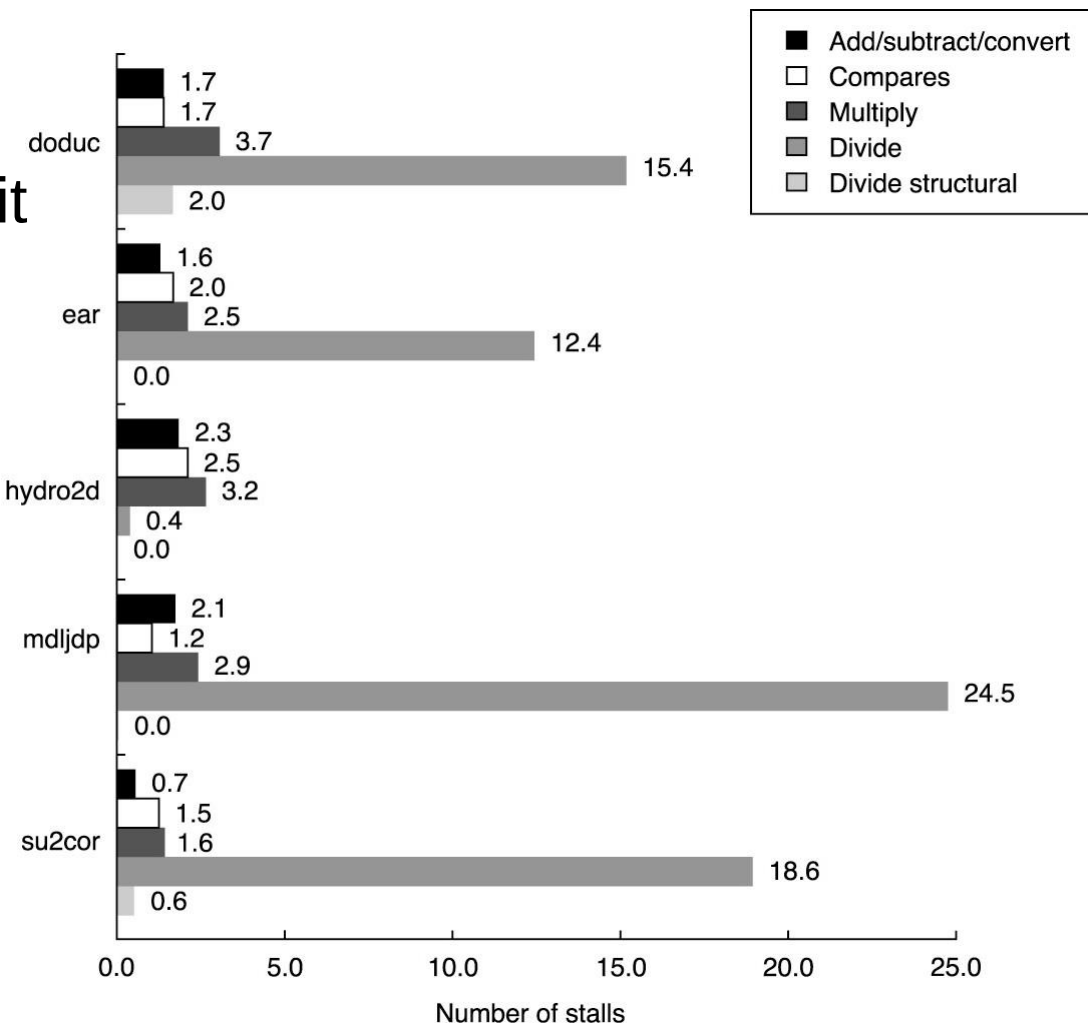
- Μπορεί να προκύψει μόνο αν η “inst” δε χρησιμοποιεί τον F2
 - ποιος ο λόγος να έχουμε δύο producers του ίδιου πράγματος χωρίς ενδιάμεσο consumer???
 - σπάνια περίπτωση, αφού ένας «λογικός» compiler θα έκανε eliminate τον πρώτο producer
 - μπορεί όμως να συμβεί!! π.χ. Loops, όταν η σειρά εκτέλεσης των εντολών δεν είναι η αναμενόμενη (branch delay slots, εντολές σε trap handlers, κ.λπ.), κ.α.
- Το hazard ανιχνεύεται στο ID, όταν η L.D είναι έτοιμη να γίνει issue
- Αντιμετώπιση:
 - καθυστερούμε το issue της L.D μέχρι η ADD.D να μπει στο MEM
 - απορρίπτουμε την εγγραφή της ADD.D → η L.D μπορεί να γίνει issue άμεσα
- Η δυσκολία δεν έγκειται τόσο στην αντιμετώπιση του hazard, όσο στο να βρούμε ότι η L.D μπορεί να ολοκληρωθεί πιο γρήγορα από την ADD.D!

Summary

- Τρεις επιπλέον έλεγχοι για hazards στο ID προτού μια εντολή γίνει issue:
 - **structural**:
 - εξασφάλισε ότι η (non-pipelined) μονάδα δεν είναι απασχολημένη
 - εξασφάλισε ότι το write port του register file θα είναι διαθέσιμο όταν θα ζητηθεί
 - **RAW**:
 - περίμενε μέχρι οι source registers της issuing εντολής να μην υπάρχουν πλέον σαν destinations στους ενδιάμεσους pipeline registers των pipelined μονάδων εκτέλεσης
 - π.χ. για τη μονάδα FP-add: αν η εντολή στο ID έχει σαν source τον F2, τότε για να μπορεί να γίνει issue, ο F2 δε θα πρέπει να συγκαταλέγεται στα destinations των ID/A1, A1/A2, A2/A3.
 - **WAW**:
 - έλεγξε αν κάποια εντολή στα στάδια A1,...,A4,D,M1,...,M7 έχει τον ίδιο destination register με αυτή στο ID, και αν ναι, stall-αρε την τελευταία στο ID
- Προώθηση:
 - έλεγξε αν το destination κάποιου από τους EX/MEM, A4/MEM, M7/MEM, D/MEM, MEM/WB ταυτίζεται με τον source register κάποιας FP εντολής, και αν ναι, ενεργοποίησε τον κατάλληλο πολυπλέκτη

Multi-cycle FP pipeline performance

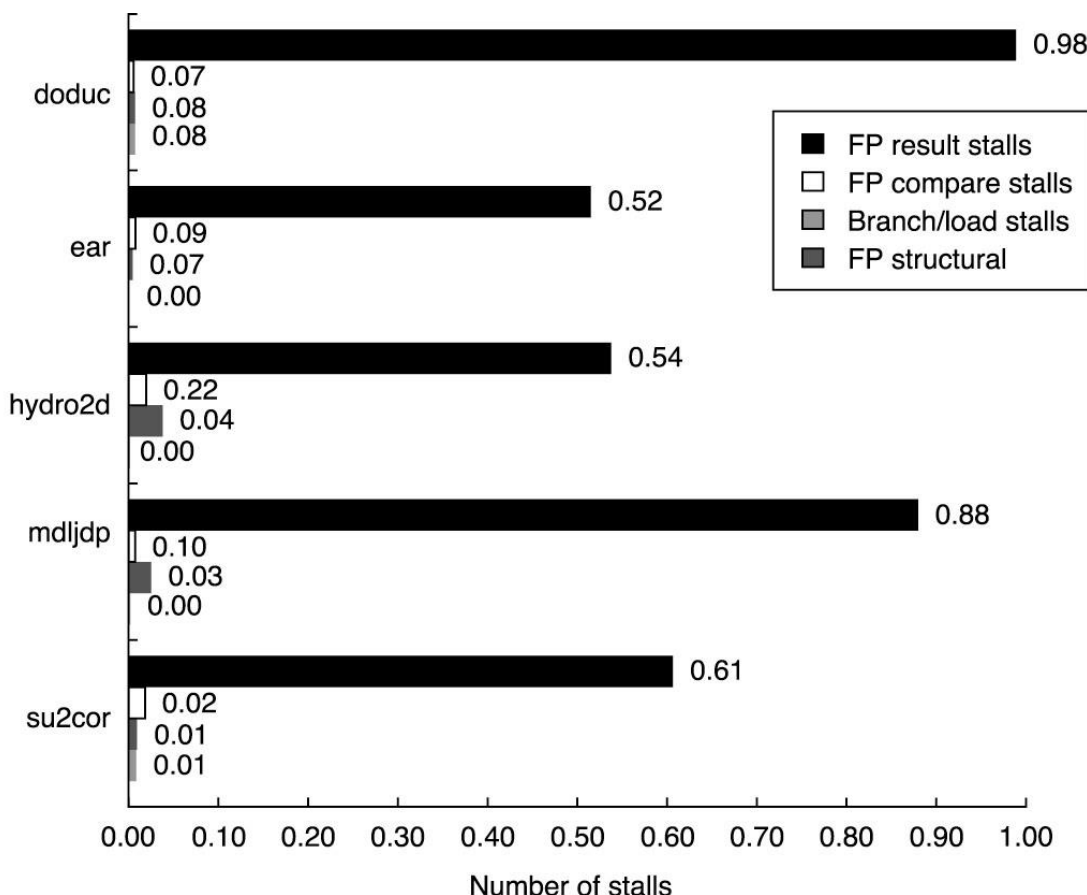
- stall cycles per FP-instruction
- RAW hazard stalls «follow» the functional unit latency, e.g.:
 - Avg #stalls due to MUL.D=2.8 (46% of FP-Mult latency)
 - Avg #stalls due to DIV.D=14.2 (59% of FP-Div latency)
- Structural hazards are rare because divides are infrequent



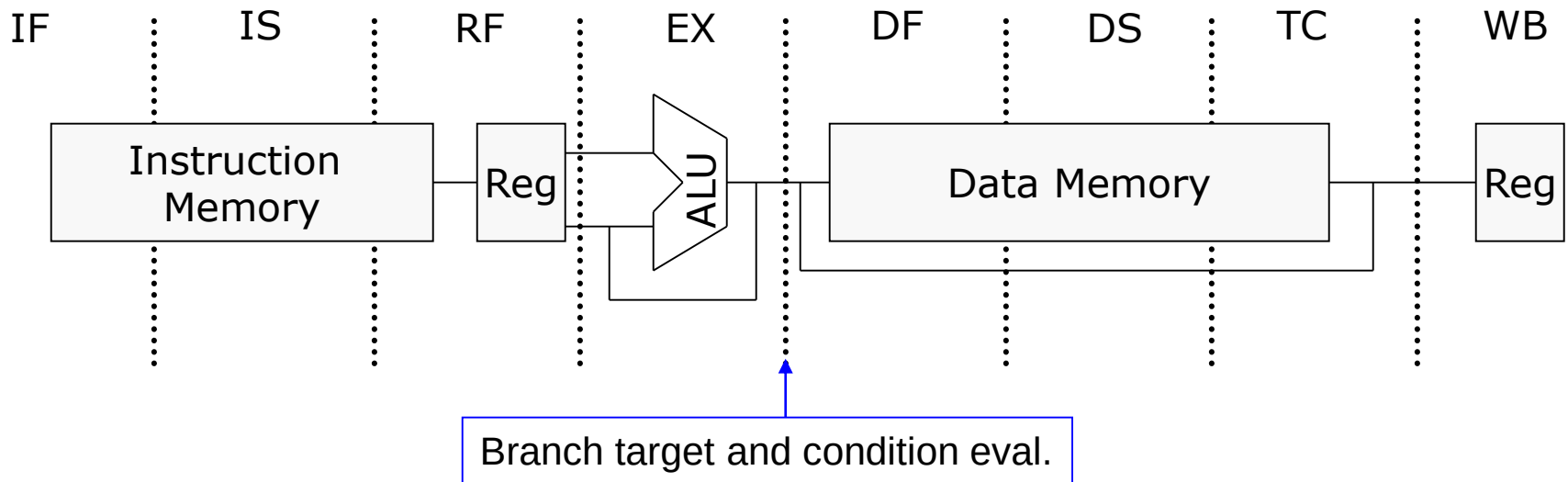
Multi-cycle FP pipeline performance

- stalls per instruction + breakdown
- between 0.65 and 1.21 stalls per instruction
- RAW hazard stalls dominate («FP result stalls»)

FP SPEC
benchmarks

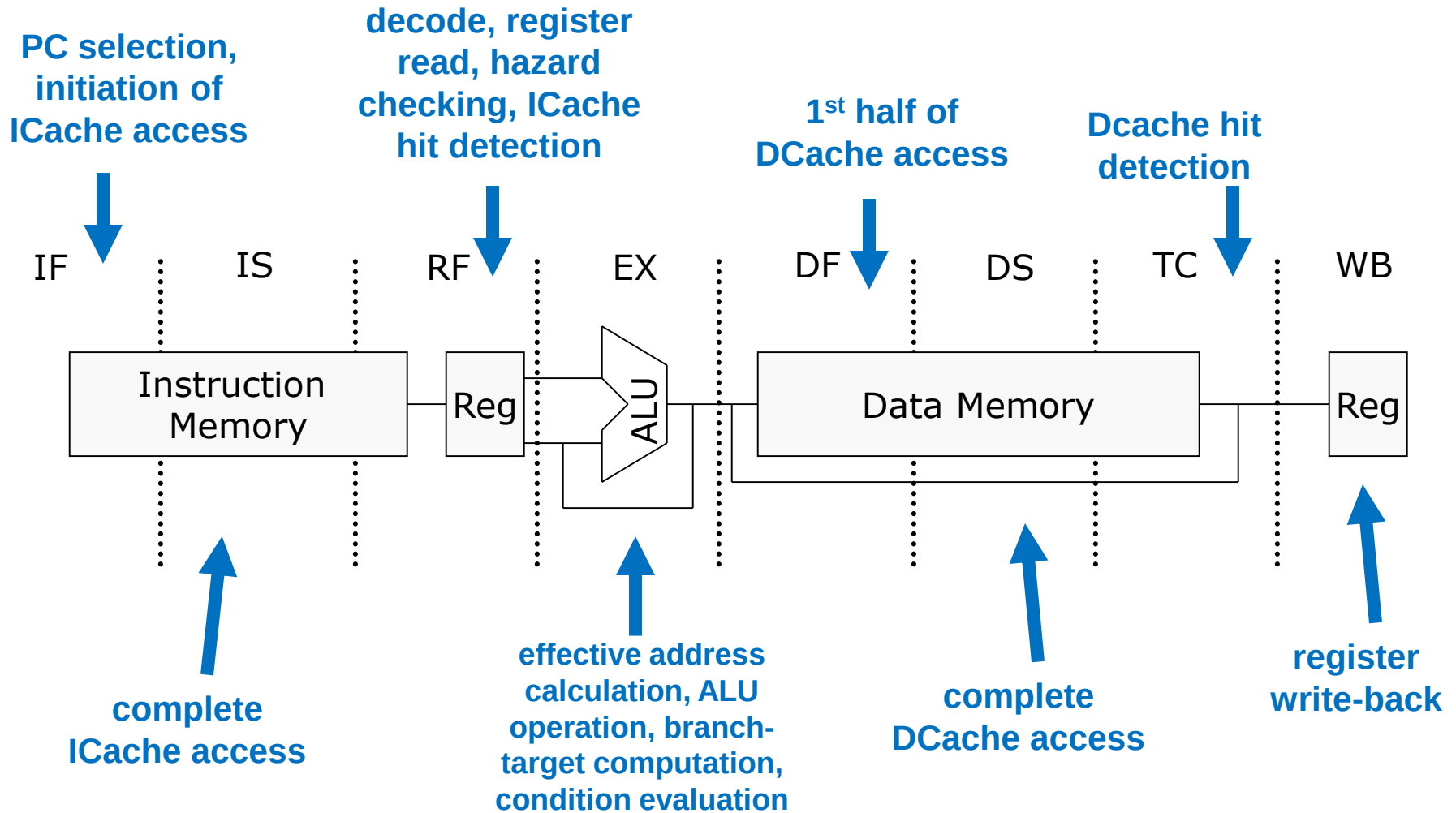


Case-study: MIPS R4000 Pipeline

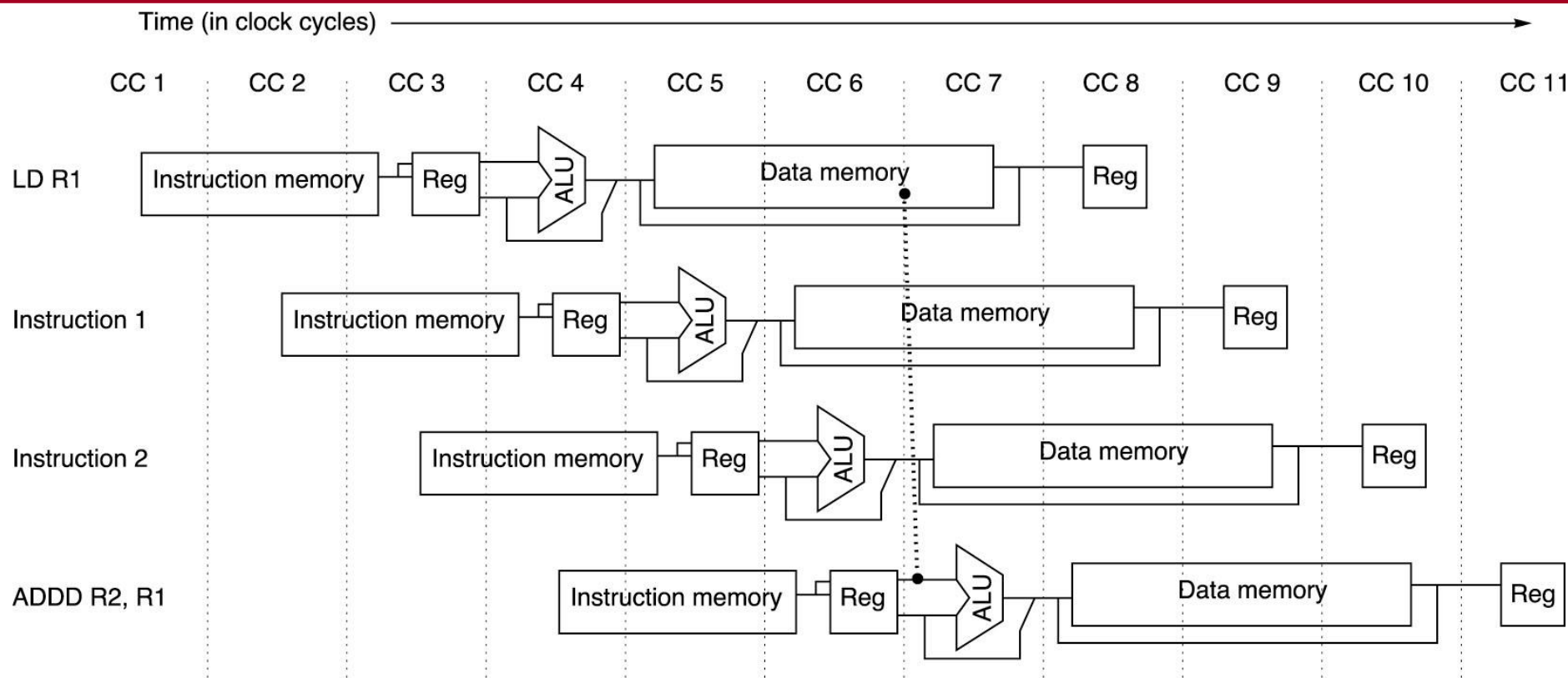


- Deeper Pipeline (superpipelining): επιτρέπει υψηλότερα clock rates
- Fully pipelined memory accesses (2 cycle delays για loads)
- Predicted-Not-Taken πολιτική
 - Not-taken (fall-through) branch : 1 delay slot
 - Taken branch: 1 delay slot + 2 idle cycles

Case-study: MIPS R4000 Pipeline



Load delay (2 cycles)



- Actually, the pipeline can forward the data from the cache to the dependent instruction before determining if the access is hit ή miss!

Branch delay (3 cycles)

